



**Universidad Tecnológica Nacional
Facultades Regionales Córdoba
Cátedra de Diseño de Sistemas**

Patrones de Diseño

**Judith Meles
Cecilia Massano
Laura Covaro
Elizabeth Jeinson**

Principios de Diseño que vimos...

- ◎ Don't Repeat Yourself
- ◎ Tell don't ask
- ◎ SOC (Separation of concerns)
- ◎ Principle of least knowledge (No hables con extraños)
- ◎ Composite Reuse Principle
- ◎ Program to Interface Principle (PIP)
- ◎ YAGNI
- ◎ KISS

◎ SOLID

- ◎ (SRP) Single Responsibility
- ◎ (OCP) Open Closed (Encapsular lo que varia)
- ◎ (LSP) Liskov Sustitution
- ◎ (ISP) Interface Segregation
- ◎ (DIP) Dependency Inversion

**Todos
corolarios de
los principios
de diseño
básicos:**





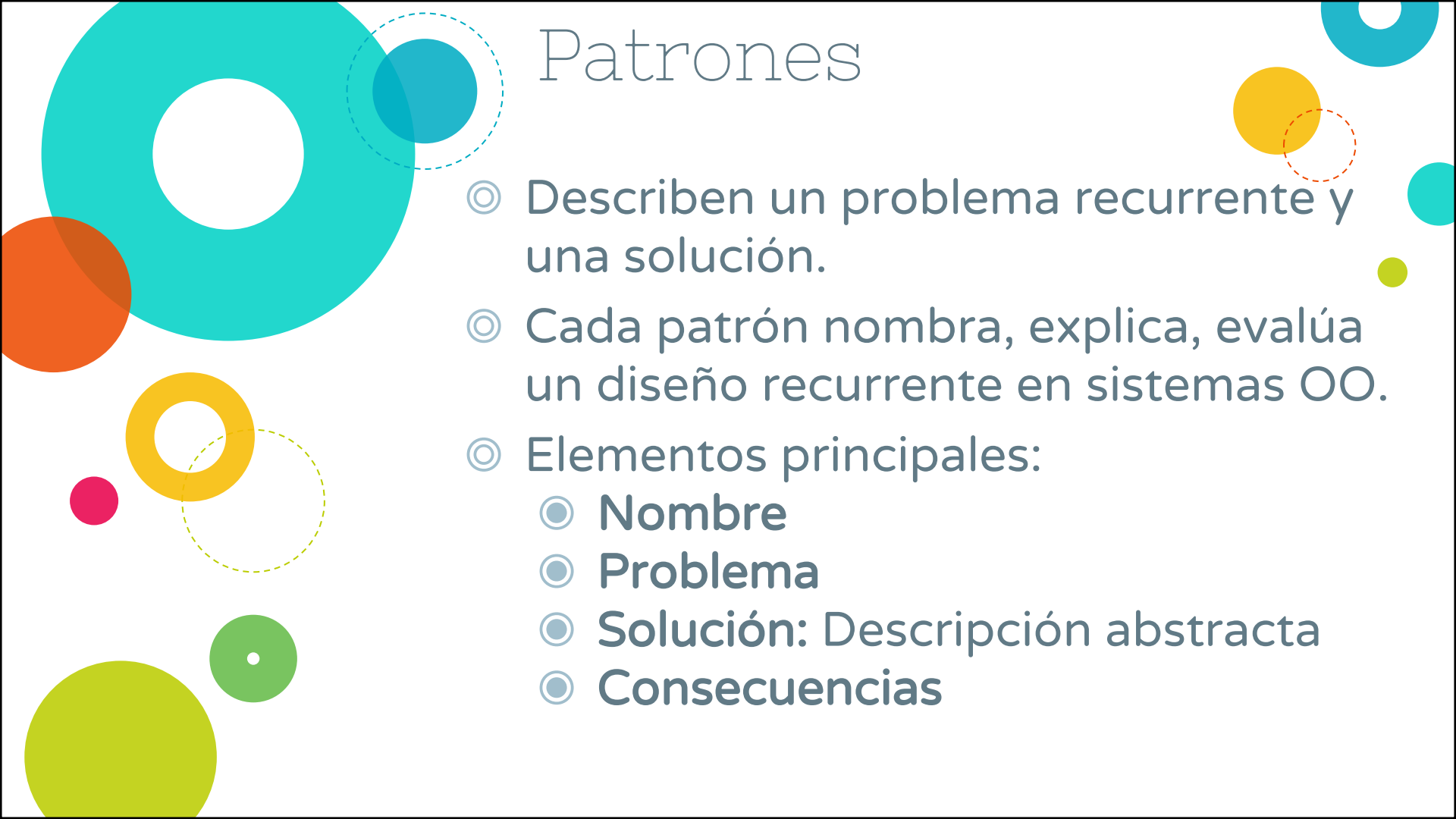
Patrones de Diseño

- © Refinan un subsistema o componente de un sistema software.
- © Describen una estructura a la cual muchas veces recurrimos para resolver problemas de diseño.
- © Patrones de escala media.
- © No afectan a la arquitectura de un sistema
- © No dependen del lenguaje de implementación.
- © Permiten resolver problemas complejos y direccionan la cooperación efectiva entre componentes

Patrones vs. Idiomas

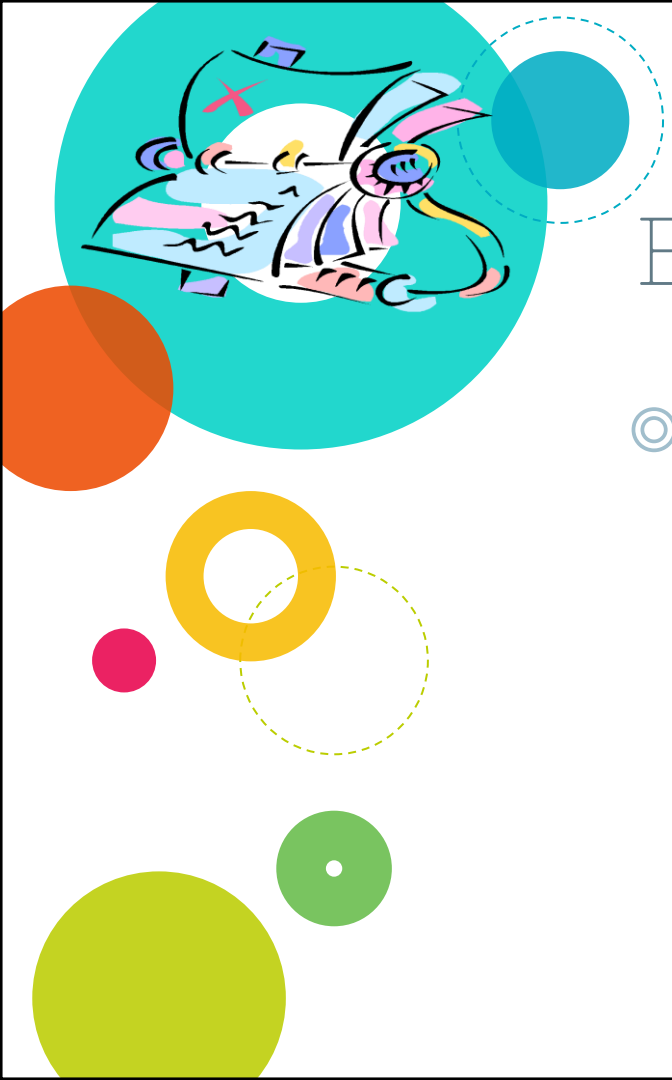


- ¿Cuál es la diferencia entre un patrón de diseño y un idioma?
- La diferencia es el alcance.
- En general, tanto los patrones de diseño como los idiomas describen soluciones a problemas de diseño recurrentes.
- En el **patrón de diseño**, tanto el problema como la solución son lo suficientemente genéricos como para ser independientes del lenguaje de programación.
- Un **idioma**, por el contrario, es un patrón de bajo nivel que es específico a un lenguaje de programación.



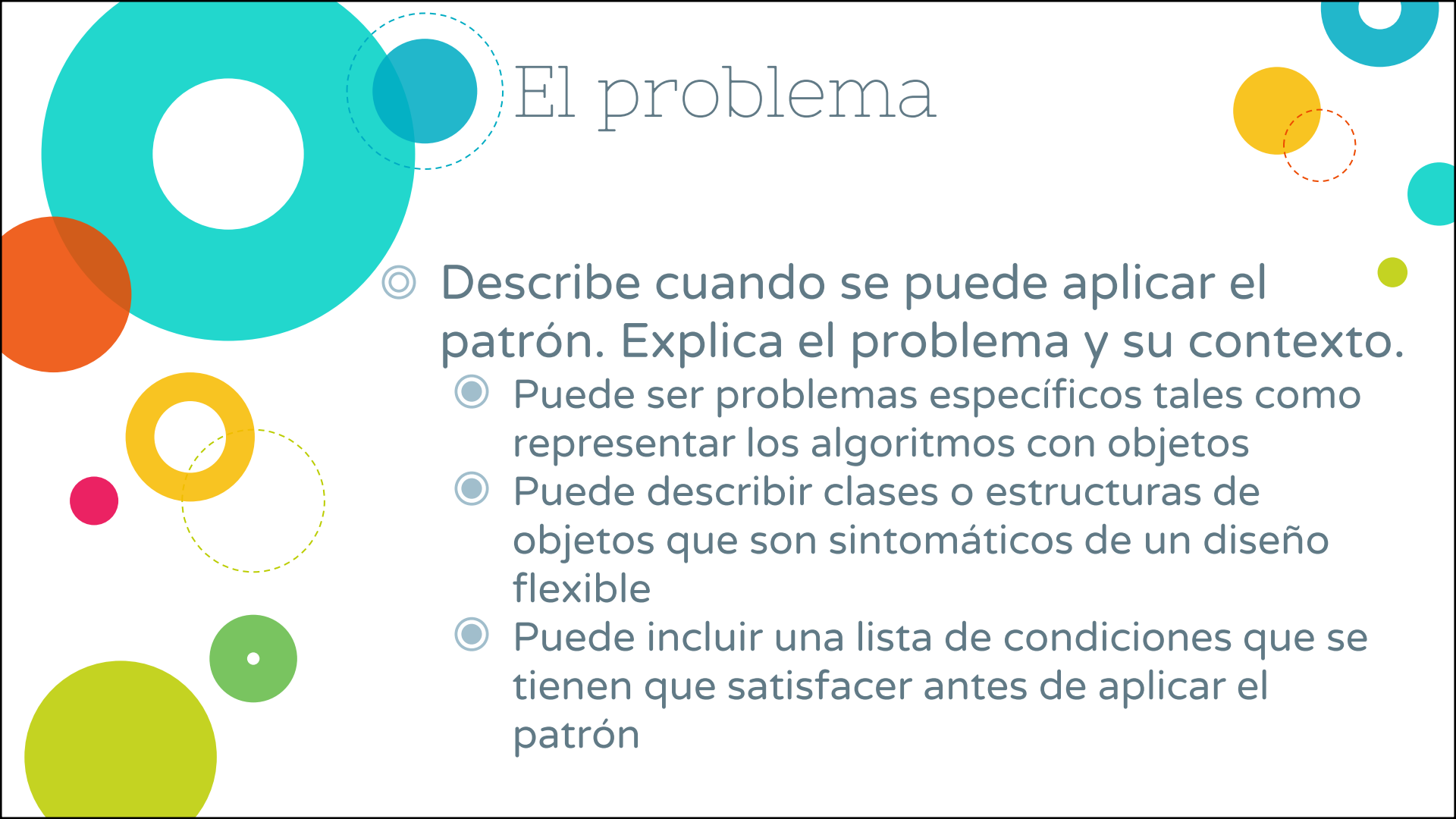
Patrones

- © Describen un problema recurrente y una solución.
- © Cada patrón nombra, explica, evalúa un diseño recurrente en sistemas OO.
- © Elementos principales:
 - © **Nombre**
 - © **Problema**
 - © **Solución:** Descripción abstracta
 - © **Consecuencias**



El nombre

- ◎ Se debe poder describir con una o dos palabras un problema, sus soluciones y la consecuencia.
 - ◎ Nominar un patrón incrementa los vocabularios de diseño
 - ◎ Nos permite diseñar con abstracción de un nivel más alto



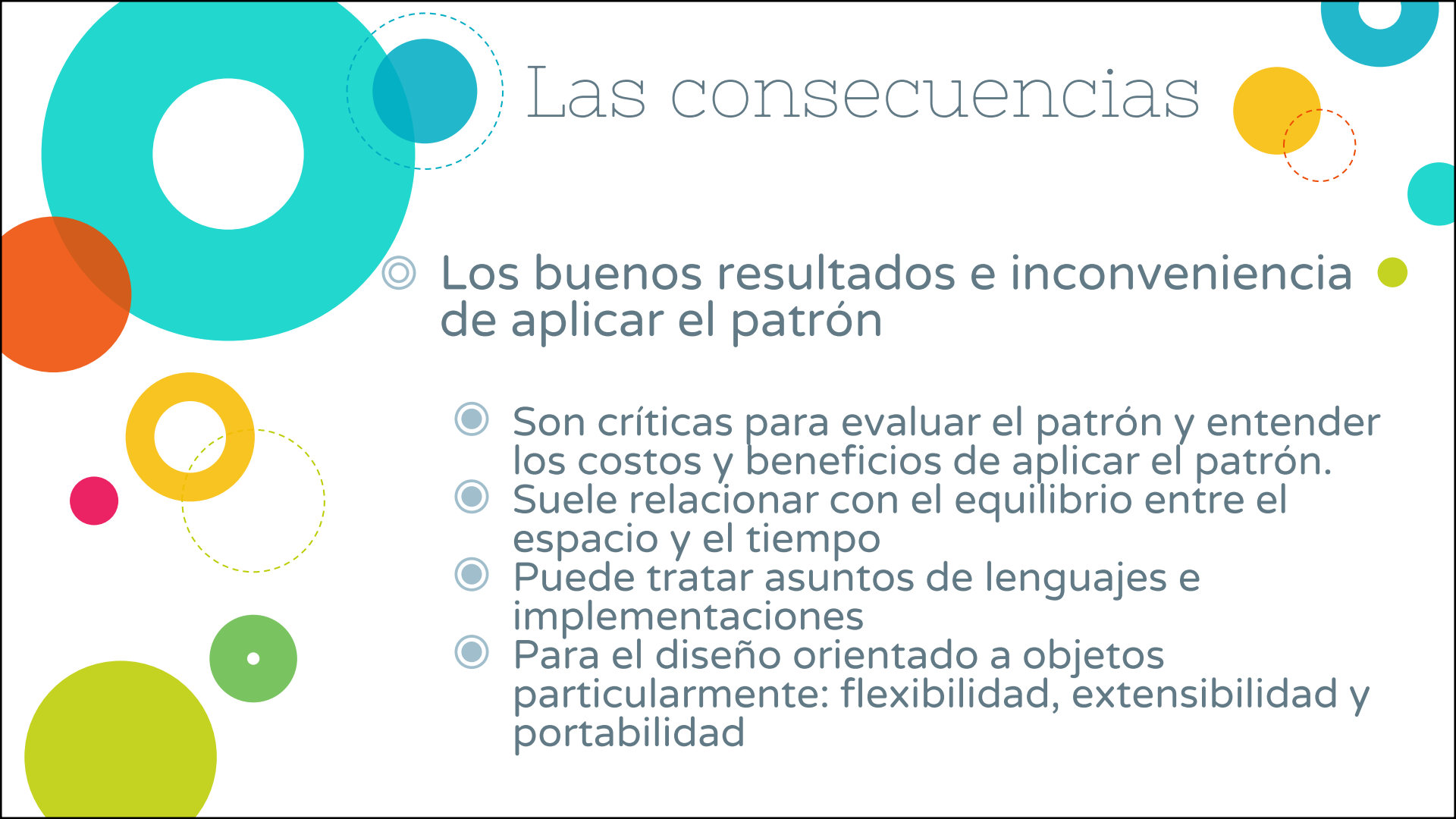
El problema

- ⦿ Describe cuando se puede aplicar el patrón. Explica el problema y su contexto.
 - ⦿ Puede ser problemas específicos tales como representar los algoritmos con objetos
 - ⦿ Puede describir clases o estructuras de objetos que son sintomáticos de un diseño flexible
 - ⦿ Puede incluir una lista de condiciones que se tienen que satisfacer antes de aplicar el patrón



La Solución

- ◎ Describe los elementos que forman el diseño, sus relaciones, responsabilidades, y cooperaciones.
 - ◎ La solución no describe ningún diseño concreto
 - ◎ Un patrón es una plantilla para muchas situaciones diferentes
 - ◎ Un patrón dispone la descripción abstracta de un problema de diseño y cómo se resuelve con elementos (clases y objetos) generales



Las consecuencias

- ◎ Los buenos resultados e inconveniencia de aplicar el patrón
 - ◎ Son críticas para evaluar el patrón y entender los costos y beneficios de aplicar el patrón.
 - ◎ Suele relacionar con el equilibrio entre el espacio y el tiempo
 - ◎ Puede tratar asuntos de lenguajes e implementaciones
 - ◎ Para el diseño orientado a objetos particularmente: flexibilidad, extensibilidad y portabilidad

Plantilla de definición

- **Nombre**
- **Propósito**
 - ¿Qué hace? ¿Cuál es su razón y propósito?
 - ¿Qué cuestión de diseño particular o problema aborda?
- **Sinónimos**
- **Motivación**
 - Escenario que ilustra un problema particular y cómo el patrón lo resuelve.



Plantilla de definición

- **Aplicabilidad**
 - ¿En qué situaciones puede aplicarse?, ¿Cómo las reconoces?, ejemplos de diseños que pueden mejorarse.
- **Estructura**
 - Diagramas de clases que ilustran la estructura
- **Participantes**
 - Clases que participan y sus responsabilidades
- **Colaboraciones**
 - Diagramas de interacción que muestran cómo colaboran los participantes.



Plantilla de definición

◎ Consecuencias

- ◎ ¿Cómo alcanza el patrón sus objetivos?, ¿Cuáles son los compromisos y resultados de usar el patrón?, Alternativas, Costes y Beneficios

◎ Implementación

- ◎ Técnicas, heurísticas y consejos para la implementación
- ◎ ¿Hay cuestiones dependientes del lenguaje?

◎ Ejemplo de Código

◎ Usos conocidos

◎ Patrones relacionados



¿Cómo seleccionar un patrón?

- © Considera de que forma los patrones resuelven problemas de diseño
- © Lee la sección que describe el propósito de cada patrón
- © Estudia las interrelaciones entre patrones
- © Analiza patrones con el mismo propósito
- © Examina las causas de rediseñar
- © Considera que debería ser variable en tu diseño

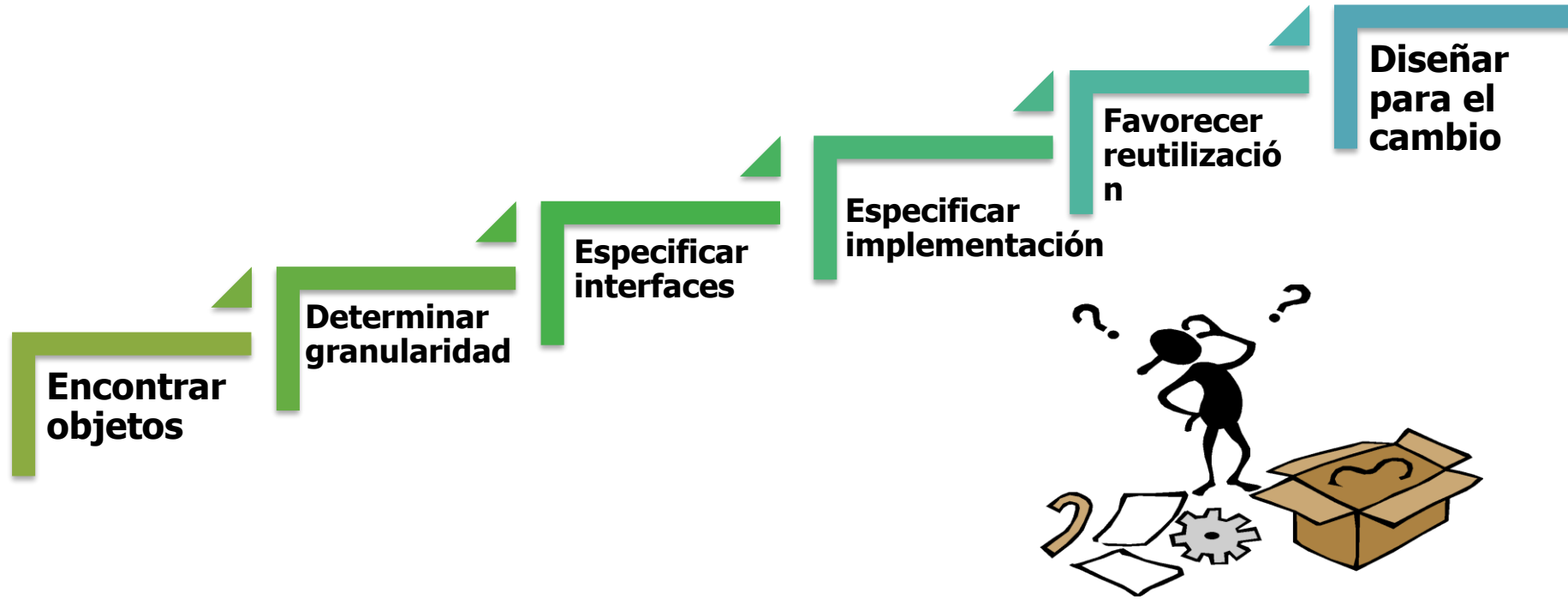


¿Cómo usar un patrón?

- © Lee el patrón, todos sus apartados.
- © Obtenga una visión global.
- © Estudia la *Estructura, Participantes y Colaboraciones*
- © Mira el ejemplo de código
- © Asocia a cada participante del patrón un elemento software de tu aplicación.
- © Implementa las clases y métodos relacionados con el patrón.



¿A qué ayudan los patrones?



¿A qué ayudan los patrones?

◎ Encontrar los Objetos

- ◎ Tarea difícil
- ◎ Debemos considerar: encapsulamiento, flexibilidad, dependencias, reutilización, extensibilidad, rendimiento,...
- ◎ Clases del *análisis* vs. Clases del *diseño*
- ◎ Clases del diseño no tienen contrapartida en el dominio ⇒ Fabricación Pura
- ◎ Clases del diseño son claves para hacer el sistema más flexible.

¿A qué ayudan los patrones?

- ◎ **Determinar granularidad de los objetos**
 - *Facade, Flyweight, Builder, Visitor,..*
- ◎ **Especificar interfaces**
 - *Memento, Decorator, Proxy,..*
- ◎ **Especificar implementación de clases**
 - Muchos patrones dependen de la distinción entre herencia de implementación y herencia de interfaces
 - *Observer, State, Chain of Responsibility,...*

¿A qué ayudan los patrones?

- ◎ **Especificar implementación de clases**

“programar hacia la interfaz, no hacia la implementación”

- ◎ No declarar variables de clases concretas sino abstractas.
- ◎ Patrones de creación permiten que un sistema esté basado en términos de interfaces y no en implementaciones.
- ◎ Favorecer la reutilización
- ◎ Delegación

¿A qué ayudan los patrones?

© Favorecer la reutilización

“Favorecer la composición sobre la herencia de clases”

Herencia y composición trabajan juntas

“Se suele abusar de la herencia. Los diseños suelen ser más reutilizables si dependen de la composición”.

¿A qué ayudan los patrones?

- © La clave para la reutilización es anticiparse a los nuevos requisitos y cambios, de modo que los sistemas evolucionen de forma adecuada.
- © Cada patrón permite que algunos aspectos de la estructura del sistema puedan cambiar independientemente de otros aspectos.
- © Facilitan reuso interno, extensibilidad y mantenimiento.



Tipos de patrones de diseño (Gamma)

- © Patrones de Creación
- © Patrones de Estructura
- © Patrones de Comportamiento



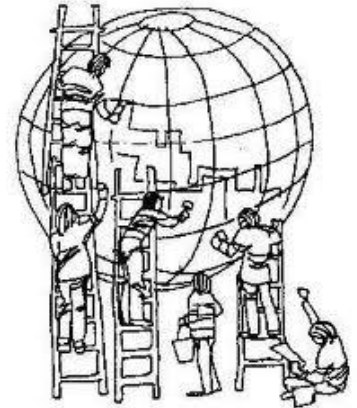
¿A qué ayudan los patrones?

- ◎ **Se encapsula:**
 - ◎ qué clases concretas usa el sistema
 - ◎ cómo se crean las instancias de esas clases
- ◎ **El sistema conoce las clases abstractas**
- ◎ **Flexibilidad en qué se crea, quién lo crea, cómo se crea y cuándo se crea.**

- **Patrones de Creación**
- **Patrones de Estructura**
- **Patrones de Comportamiento**

Patrones de Creación

- ◎ Abstraen el proceso de creación de objetos.
- ◎ Ayudan a crear sistemas independientes de cómo los objetos son creados, compuestos y representados.
- ◎ Dos tipos:
 - Clase: usa **herencia** para variar la clase que es instanciada.
 - Objeto: **delega** instanciación a otro objeto.



- Patrones de Creación
- Patrones de Estructura
- Patrones de Comportamiento

Patrones de Creación

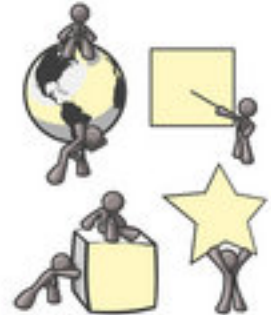
- ◎ **Abstract Factory (Fábrica Abstracta):** Proporciona una interfaz para crear familias de objetos relacionados o que dependen entre sí, sin especificar sus clases concretas.
- ◎ **Factory Method (Método de Fabricación):** Define una interfaz para crear un objeto, pero deja que las subclases deciden que clase instanciar. Delega en las subclases la creación de objetos.
- ◎ **Singleton (Único):** Garantiza que una clase tenga una única instancia y proporciona un punto de acceso global a ella.
- ◎ **Builder (Constructor):** Separa la construcción de un objeto complejo de su representación, de forma que el mismo proceso de construcción pueda crear diferentes representaciones.
- ◎ **Prototype (Prototipo):** Especifica los tipos de objeto a crear por medio de una instancia prototípica, y crea nuevos objetos copiando de este prototipo.

- **Patrones de Creación**
- **Patrones de Estructura**
- **Patrones de Comportamiento**

Patrones de Estructura

- © Determinan cómo combinar objetos y clases para definir estructuras complejas.
 - © Comunicar dos clases incompatibles,
 - © Añadir funcionalidad a objetos

- Patrones de Creación
- Patrones de Estructura
- Patrones de Comportamiento



Patrones de Estructura

- ◎ **Adapter (Adaptador):** Convierte la interfaz de una clase en otra distinta que es la que esperan los clientes. Permite que cooperen las clases que de otra manera tendrían interfaces incompatibles.
- ◎ **Bridge (Puente):** Desacopla una abstracción de su implementación, de manera que ambas puedan variar de forma independiente.
- ◎ **Composite (Compuesto):** Combina objetos en estructuras de árbol para representar jerarquías de todo-parte. Permite que los clientes traten de manera uniforme a los objetos individuales y a los compuestos.
- ◎ **Decorator (Decorador):** Añade dinámicamente nuevas responsabilidades a un objeto, proporcionando una alternativa flexible a la herencia para extender funcionalidad.
- ◎ **Facade (Fachada):** Proporciona una interfaz unificada para un conjunto de interfaces de un subsistema. Define una interfaz de alto nivel que hace que el subsistema sea más fácil de utilizar.
- ◎ **Flyweight (Peso Ligero):** Usa el comportamiento para permitir un gran número de objetos de granularidad fina de forma eficiente.
- ◎ **Proxy (Apoderado):** Proporciona un sustituto o representante de otro objeto para controlar el acceso a éste.

- Patrones de Creación
- Patrones de Estructura
- Patrones de Comportamiento

Patrones de Comportamiento

- © Se ocupan de los algoritmos y reparto de responsabilidades.
- © Describen los patrones de comunicación entre objetos y clases.
- © Podremos definir abstracciones de algoritmos (Template Method)
- © Cooperaciones entre objetos para realizar tareas complejas, reduciendo las dependencias entre objetos (Iterator, Observer, etc.)
- © Asociar comportamiento a objetos e invocar su ejecución (Command, Strategy, etc.)



- **Patrones de Creación**
- **Patrones de Estructura**
- **Patrones de Comportamiento**

Patrones de Comportamiento

- ◎ **Chain of Responsibility (Cadena de Responsabilidad):** Evitar acoplar el emisor de una petición a su receptor, al dar a más de un objeto la posibilidad de responder a la petición. Crea una cadena con los objetos receptores y pasa la petición a través de la cadena hasta que ésta sea tratada por algún objeto.
- ◎ **Command (Orden):** Encapsula una petición en un objeto, permitiendo así parametrizar a los clientes con distintas peticiones, encolar o llevar un registro de las peticiones y poder deshacer las operaciones.
- ◎ **Interpreter (Intérprete):** Define para un lenguaje dado, una representación de su gramática junto con un intérprete que usa dicha representación para interpretar sentencias del lenguaje.
- ◎ **Iterator (Iterador):** Proporciona un modo de acceder secuencialmente a los elementos de un objeto agregado sin exponer su representación interna.
- ◎ **Mediator (Mediador):** Define un objeto que encapsula cómo interactúan los objetos. Promueve un bajo acoplamiento al evitar que los objetos se refieran uno a otros explícitamente, y permite variar la interacción entre ellos de forma independiente.

- Patrones de Creación
- Patrones de Estructura
- Patrones de Comportamiento

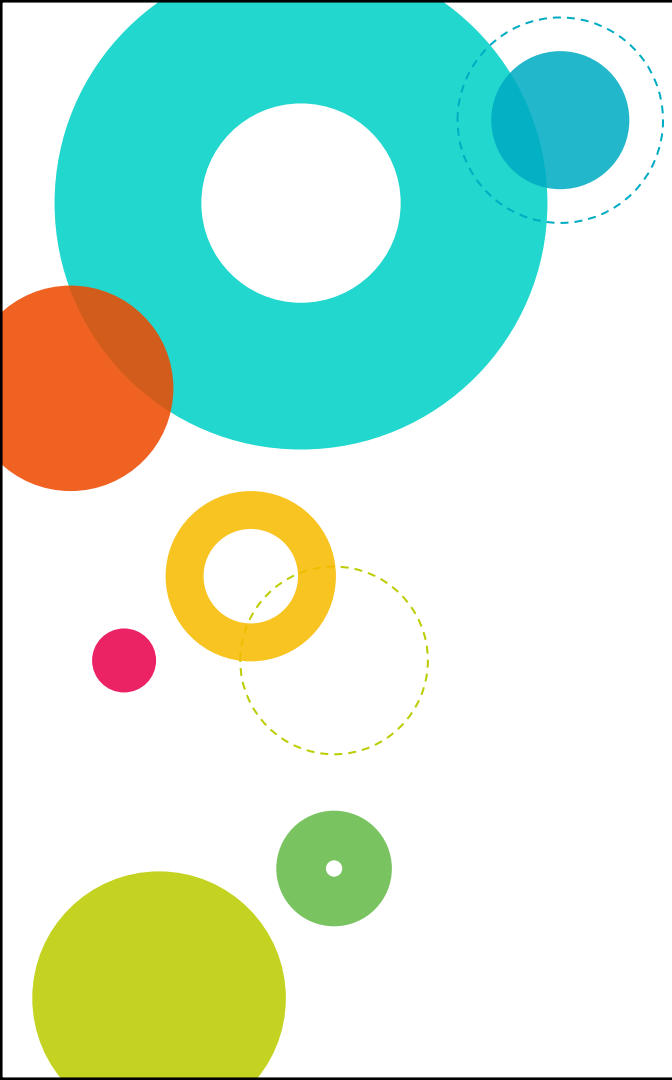
Patrones de Comportamiento (cont.)

- ◎ **Memento (Recuerdo):** Representa y externaliza el estado interno de un objeto sin violar el encapsulamiento, de forma que éste puede volver a dicho estado más tarde.
- ◎ **Observer (Observador):** Define una dependencia de uno a muchos entre objetos, de forma que cuando un objeto cambie de estado se notifica y se actualizan automáticamente todos los objetos que dependen de él.
- ◎ **State (Estado):** Permite que un objeto modifique su comportamiento cada vez que cambie su estado interno. Parecerá que cambia la clase del objeto.
- ◎ **Strategy (Estrategia):** Define una familia de algoritmos, encapsula cada uno de ellos y los hace intercambiables. Permite que un algoritmo varíe independientemente de los clientes que lo usan.
- ◎ **Template Method (Método Plantilla):** Define en una operación el esqueleto de algún algoritmo, delegando en las subclasses algunos de sus pasos. Permite que las subclasses redefinan ciertos pasos del algoritmo sin cambiar su estructura.
- ◎ **Visitor (Visitante):** Representa una operación sobre los elementos de una estructura de objetos. Permite definir una nueva operación sin cambiar las clases de los elementos sobre los que opera.

- **Patrones de Creación**
- **Patrones de Estructura**
- **Patrones de Comportamiento**

Clasificación

		Propósito		
		Creación	Estructura	Comportamiento
Ámbito	Clase	Factory Method	Adapter	Interpreter Template Method
	Objeto	Abstract Factory Builder Prototype Singleton	Adapter Bridge Composite Decorator Facade Flyweight Proxy	Chain of Responsibility Command Iterator Mediator Memento Observer State Strategy Visitor



Patrones que se pueden evaluar desde Noviembre de 2025

- ◎ 1. State
- ◎ 2. Strategy
- ◎ 3. Observer
- ◎ 4. Adapter
- ◎ 5. Iterator

Caso de Estudio

Objetivo:

Dar soporte a los procesos de venta de artículos, cobranza, preparación y despacho de los pedidos a través de las empresas de transporte contratadas, brindando información vinculada a la gestión y a la toma de decisiones.

Alcances:

- **Administrar artículos y kits de artículos con sus tipos y medios de soporte**
- **Administrar catálogos de artículos**
- **Gestionar pedidos en todo su ciclo de vida, incluida su preparación y envío, notificando cambios de estados del pedido**

Caso de Estudio

Alcances (continuación):

- Administrar clientes y las tarjetas de crédito con las que operan
- Administrar marcas de tarjetas de créditos
- Gestionar la cobranza por tarjeta de créditos
- Gestionar comentarios sobre artículos o pedidos
- Administrar empresas de transportes y sus puntos de distribución
- Administrar medios de transportes y formas de envío
- Administrar usuarios y la personalización de sus perfiles
- Administrar las ubicaciones geográficas y localizaciones
- Gestionar la entrega de pedidos y la devolución de pedidos no entregados

Caso de Estudio

Alcances (continuación):

- **Administrar motivos de no entrega**
- **Brindar información sobre pedidos y su estado**
- **Brindar información sobre los despachos de pedidos por forma de envío y por empresa de transporte**
- **Brindar información de ventas por cliente y por tipo de artículo**
- **Brindar información sobre frecuencia de compra por cliente y volumen de la operación**
- **Brindar información sobre los artículos más vendidos**

Caso de Estudio

No Contempla:

- **Contratación y Pago a proveedores**
- **Contabilización de las transacciones**
- **Logística de distribución de los pedidos, está a cargo de las empresas de transporte contratadas**
- **Gestión de Stock, sólo se descuenta del stock lo vendido.**

Caso de Estudio

Reglas de Negocio (de la funcionalidad afectada a este taller)

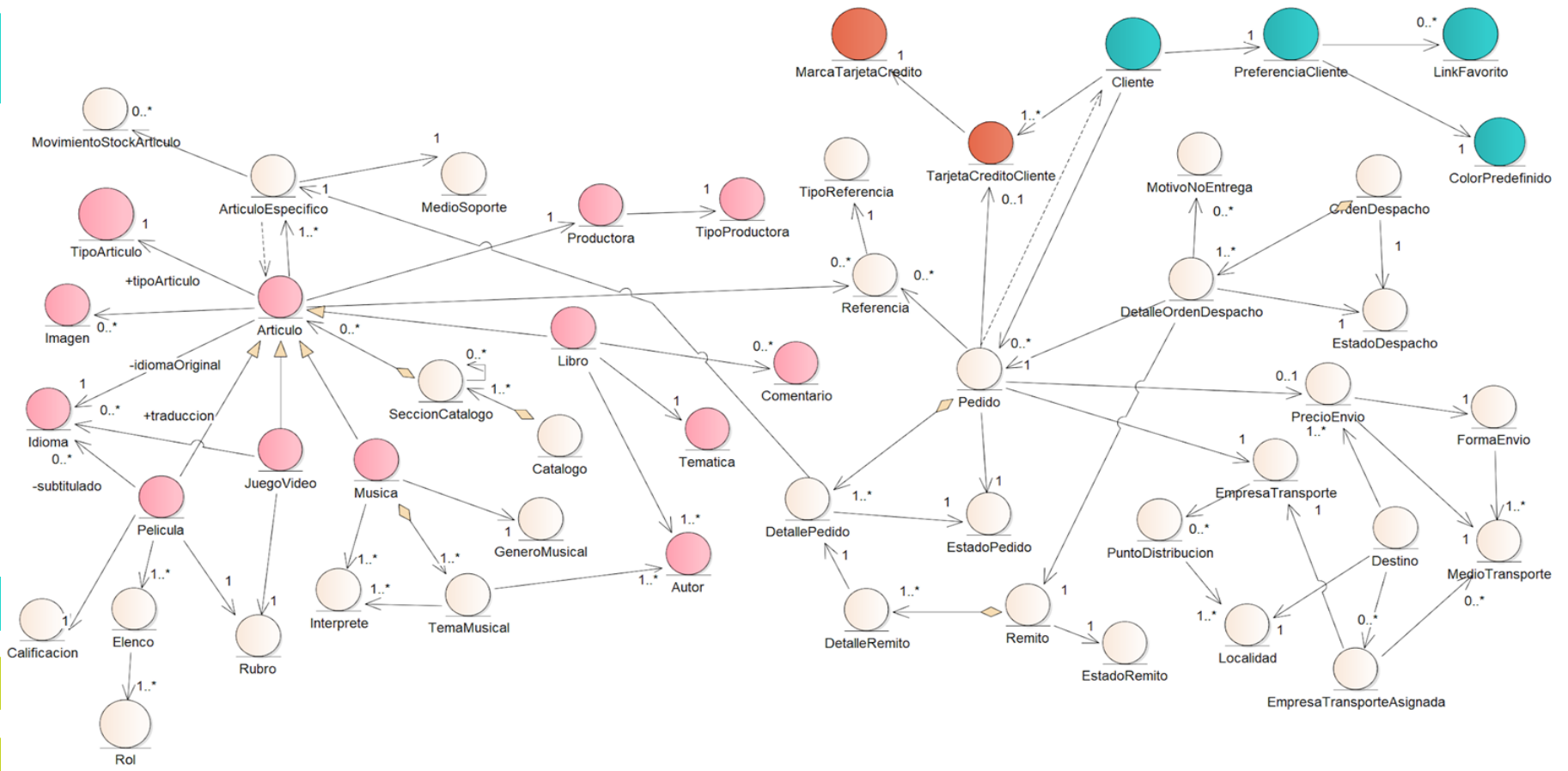
- **Costo de Envío:** El precio de la entrega se determina para cada destino y varía según el medio de transporte y la forma de envío.
- **Opciones de Envío:** Se dispone de 3 opciones de envío: un solo envío, los mínimos posibles, y tan pronto como tenga los artículos.
- **Único Catálogo Vigente:** Sólo puede existir un único catálogo vigente.
- **Kits de artículos:** Los kits sólo se pueden armar con artículos del mismo tipo.
- **Forma de Pago:** La única forma de pago aceptada es tarjeta de crédito en un solo pago.

Caso de Estudio

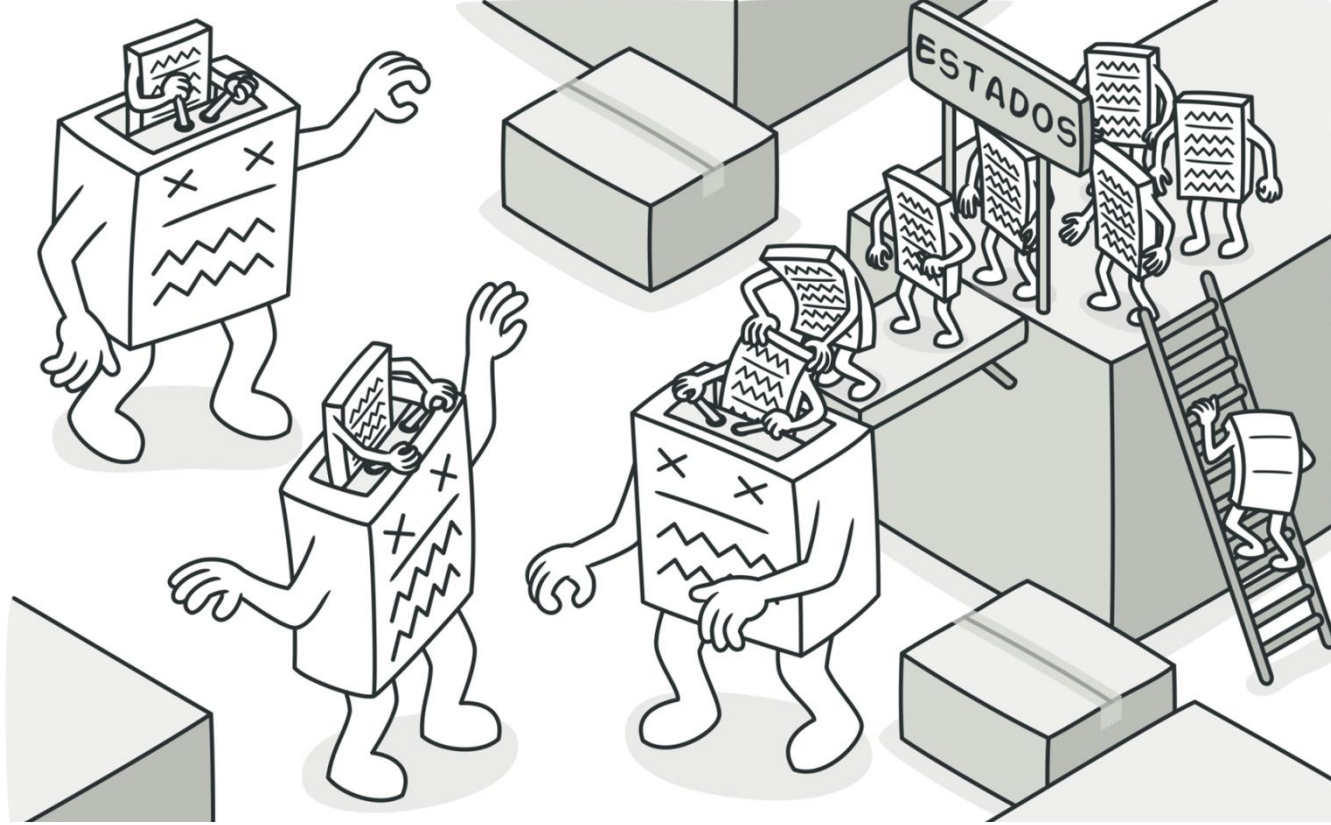
Reglas de Negocio (de la funcionalidad afectada a este taller)

- **Notificación del Estado del Pedido:** Se notificará al usuario en cada cambio de estado que defina o afecte la fecha estimada de entrega
- **Cobertura de Entrega de Pedidos:** Sólo se distribuirán a aquellos destinos donde alguna empresa de transporte tiene un Punto de Distribución.
- **Empresa de Transporte Asignada:** Existirá una única empresa de transporte activa para un Destino, Medio de Transporte y Forma de Envío.

Caso de Estudio Venta Digital: Modelo de Dominio



State



State (Estado)

◎ Propósito

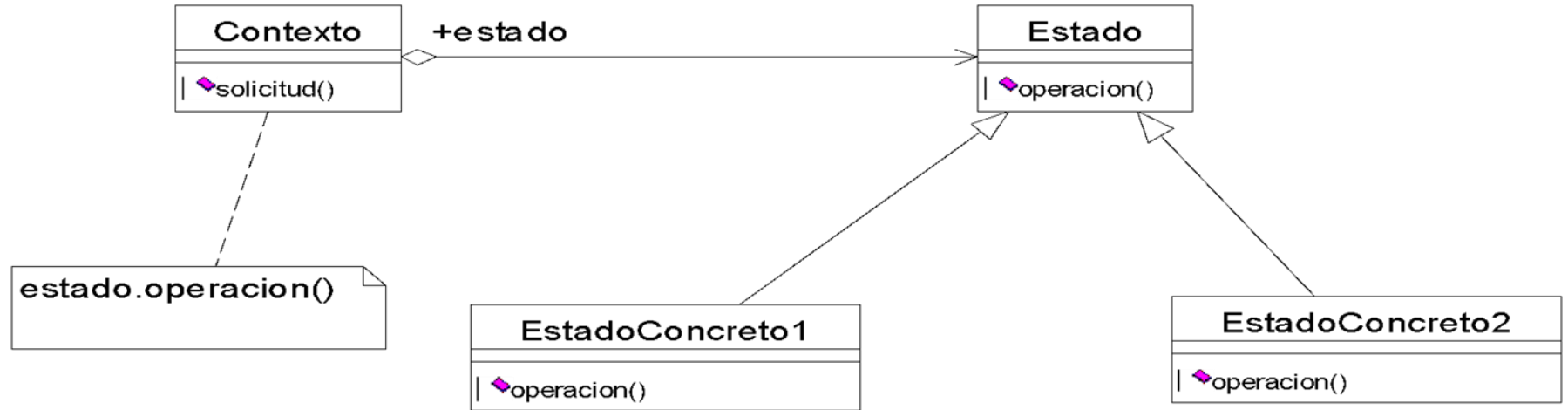
- Permite a un objeto cambiar su comportamiento cuando cambia su estado. El objeto parece cambiar de clase.

◎ Motivación

- Una conexión TCP puede encontrarse en uno de varios estados, y dependiendo del estado responderá de un modo diferente a los mensajes de otros objetos para solicitudes tales como abrir, cerrar o establecer conexión.

- Patrones de Creación
- Patrones de Estructura
- Patrones de Comportamiento

State (Estado)



State (Estado)

◎ Aplicabilidad

- ◎ El comportamiento del objeto depende de su estado, y debe cambiar su comportamiento en tiempo de ejecución dependiendo de su estado.
- ◎ Las operaciones tienen grandes estructuras CASE que dependen del estado del objeto, que es representado por uno o más constantes de tipo enumerado.

- Patrones de Creación
- Patrones de Estructura
- Patrones de Comportamiento

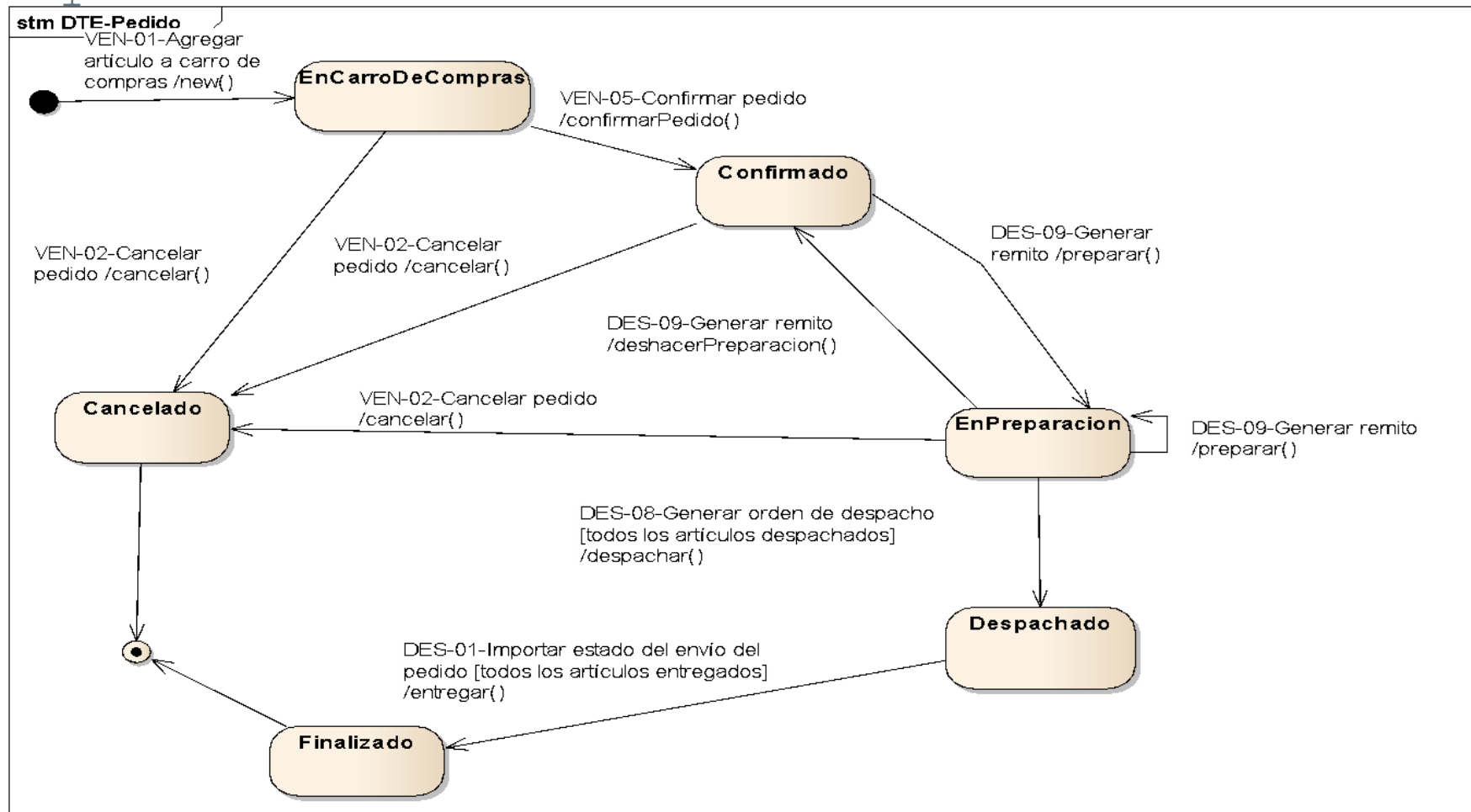
State (Estado)

◎ Consecuencias

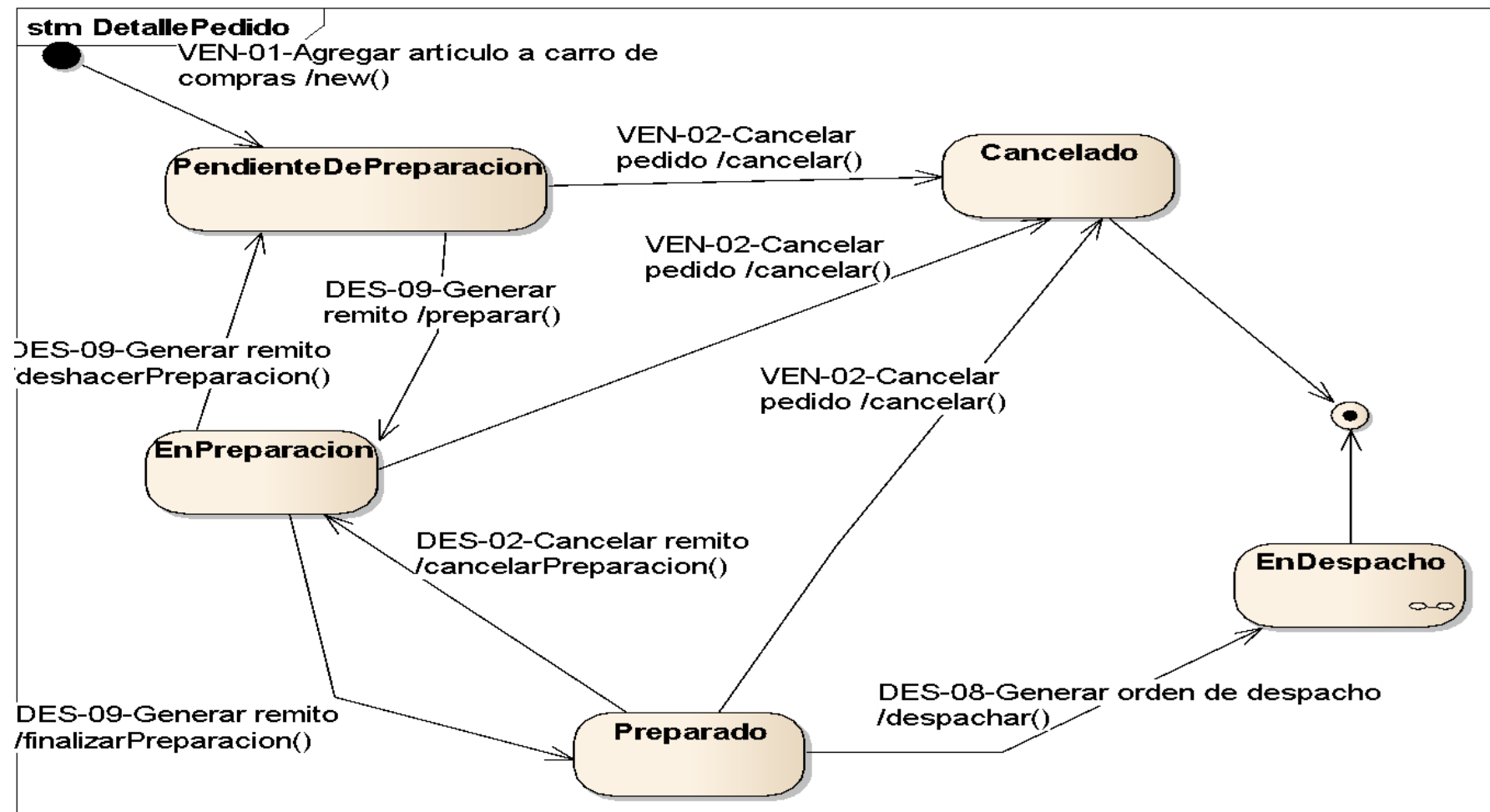
- Coloca todo el comportamiento asociado a un estado particular en una clase.
- Subclases vs. Sentencias CASE.
- Ayuda a evitar estados inconsistentes
- Transiciones de estado son más explícitas.
- Incrementa el número de objetos
 - Posible solución: Los objetos *ESTADO* pueden ser compartidos.

- Patrones de Creación
- Patrones de Estructura
- Patrones de Comportamiento

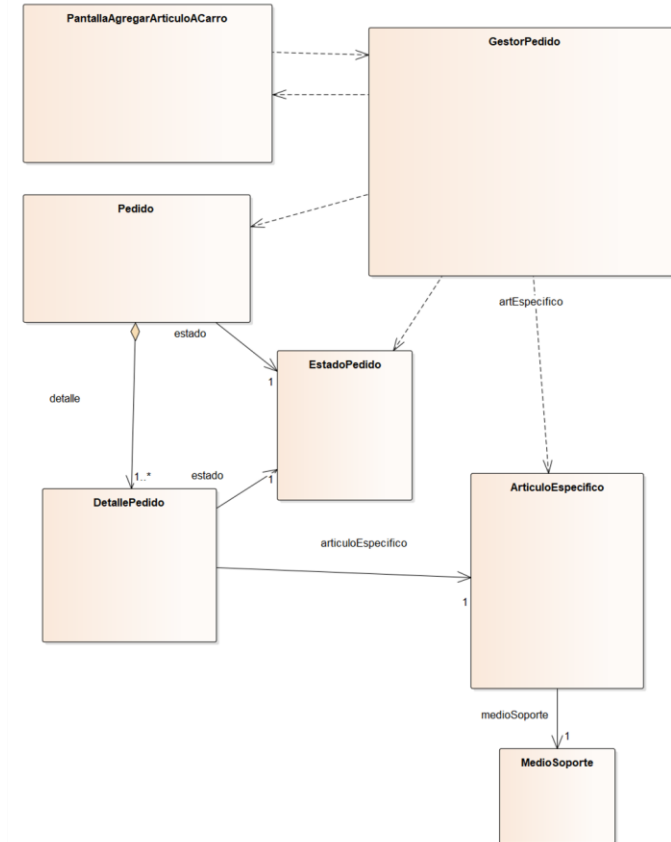
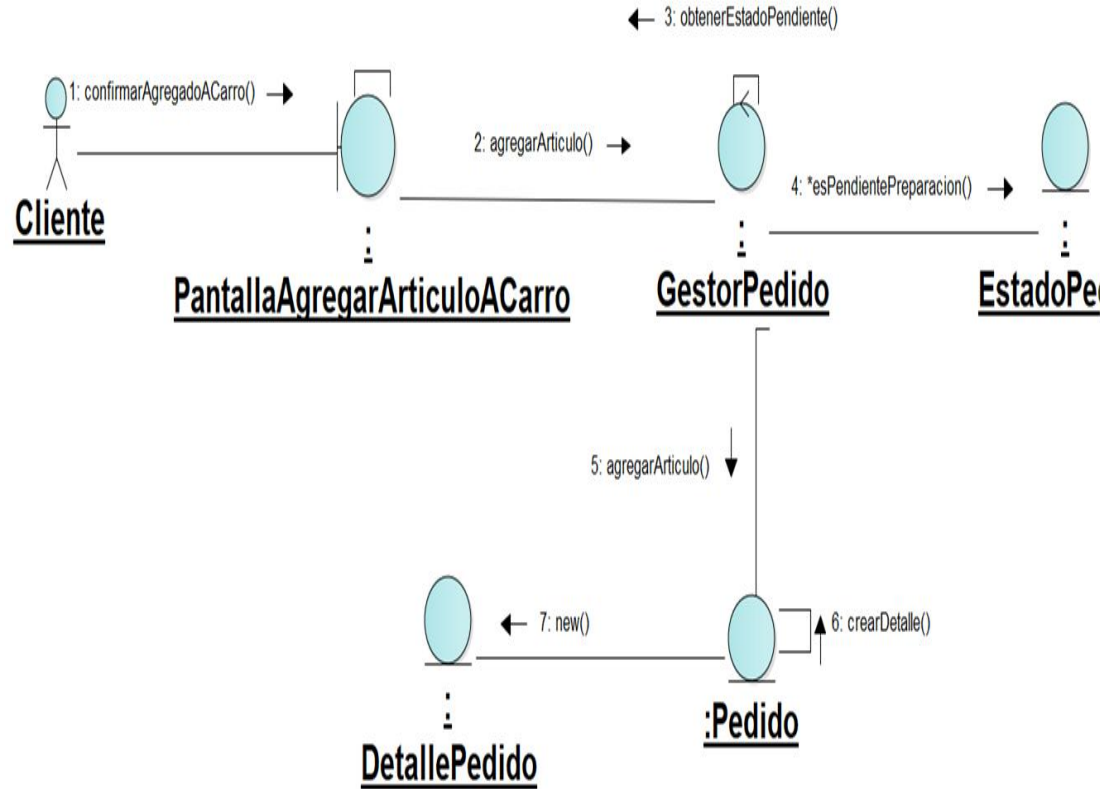
Máquina de Estado de la Clase Pedido



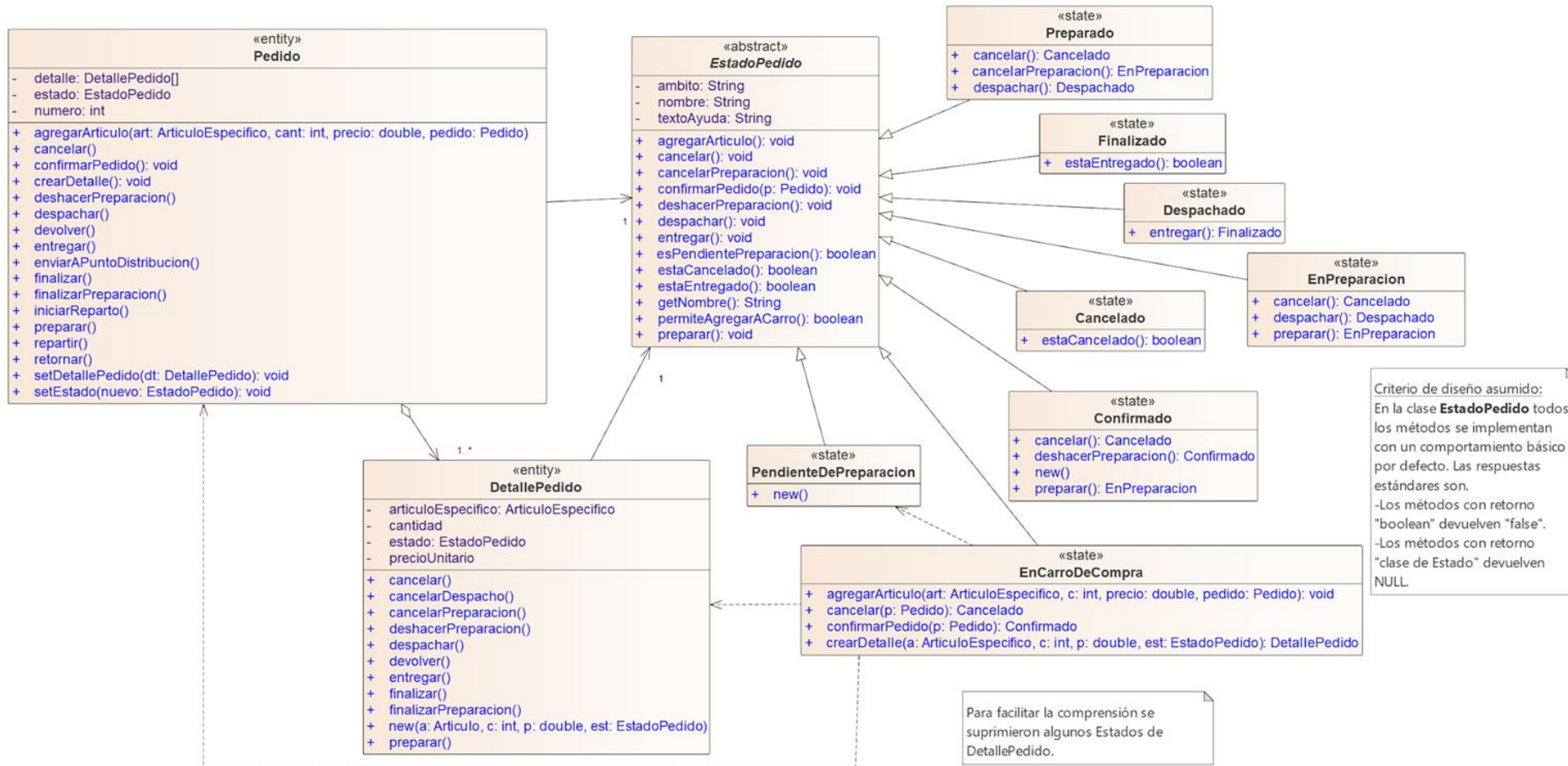
DetallePedido



Vista parcial de la Realización de CU de Análisis del Caso de Uso Agregar Artículo al Carro de Compras



State: Aplicación



State: Aplicación

◎ Participantes

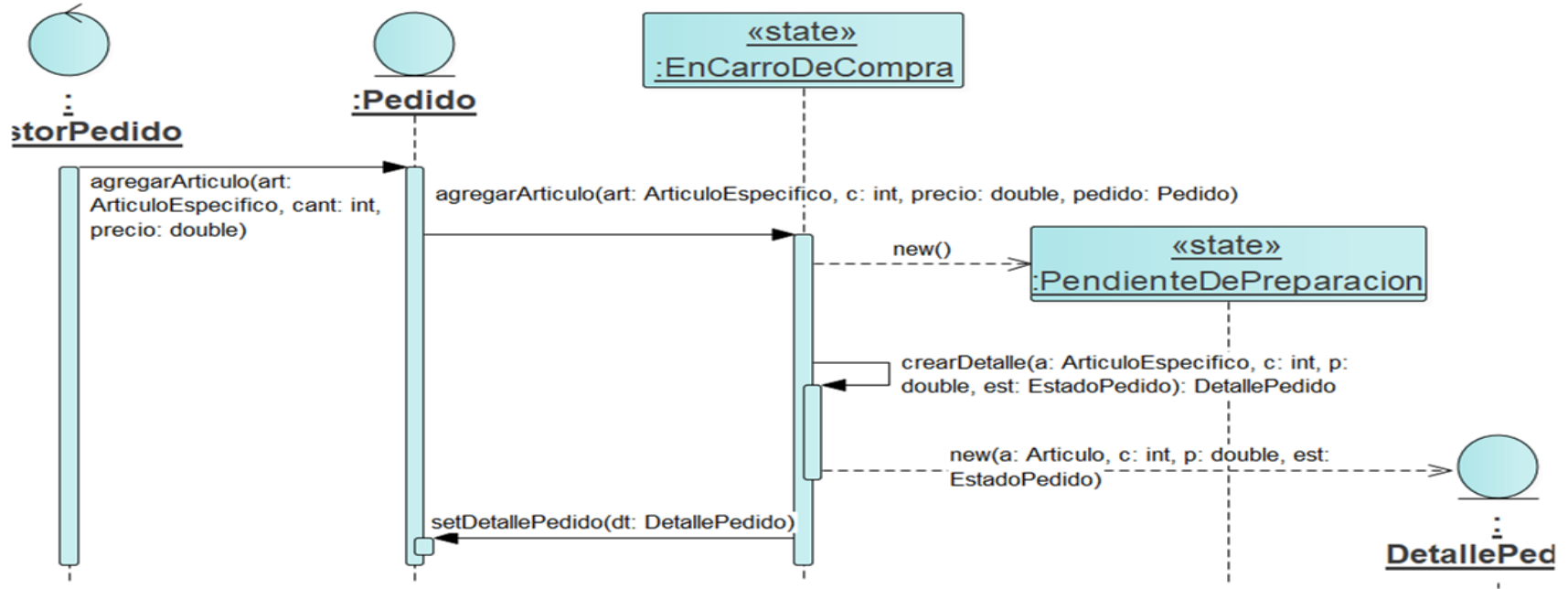
- ◎ **Contexto** [**Pedido**]: recibe el método para agregar un producto al pedido.
- ◎ **Estado Abstracto** [**EstadoPedido**]: define los métodos abstractos para cada estado diferente.
- ◎ **Estados Concretos** [**EnCarroDeCompras**; **PendienteDePreparación**; **Confirmado**; **Cerrado**; **Despachado...**]: resuelven la petición del método en función del estado concreto que poseen.

◎ Procedimiento

- ◎ El **GestorPedido** le pide al **Pedido**, que agregue un producto a la lista de los productos que contiene el pedido.
- ◎ El **Pedido** delega la petición, en función de su estado actual a la instancia de estado concreto que corresponda: **EnCarroDeCompras**.
- ◎ La Instancia de **EstadoConcreto** que corresponda resuelve la petición.

State: Aplicación

⦿ Dinámica:



⦿ Ventajas:

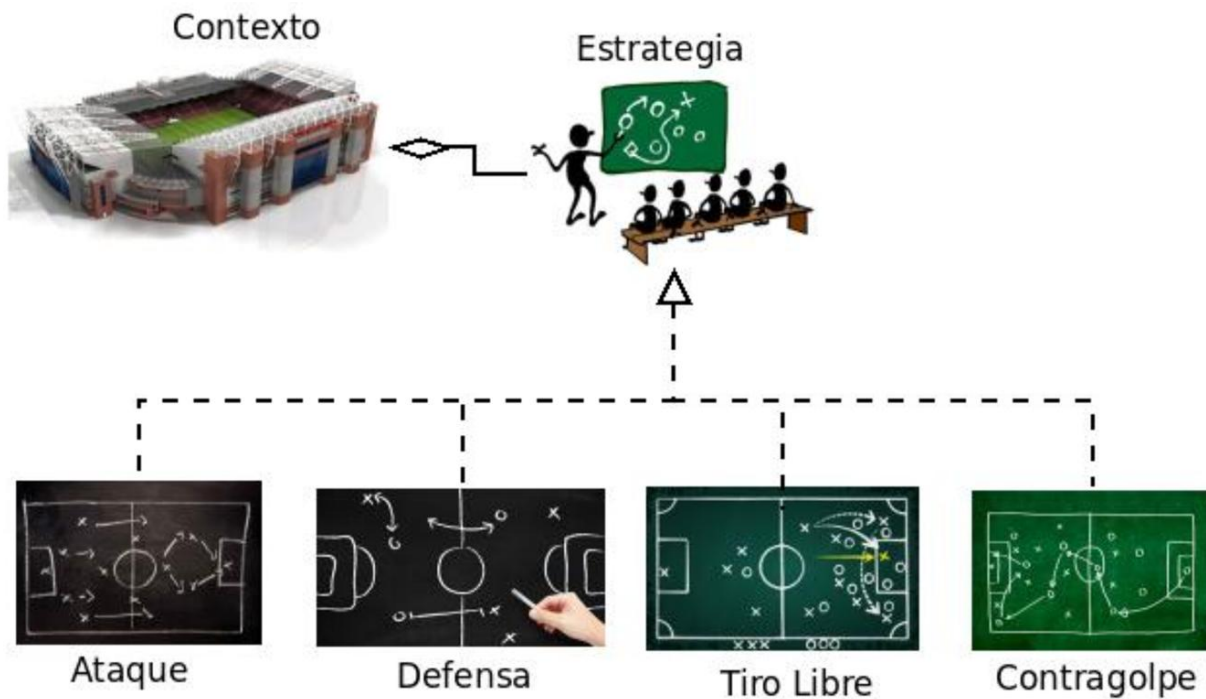
- Se eliminan las sentencias CASE utilizando Polimorfismo.
- Permite crecimiento en la variación de la lógica de negocio en función de los cambios de estado.

Strategy



Strategy

Strategy



Strategy/Policy (Estrategia)

◎ Propósito

- ◎ Define una familia de algoritmos, encapsula cada uno, y permite intercambiarlos. Permite variar los algoritmos de forma independiente a los clientes que los usan

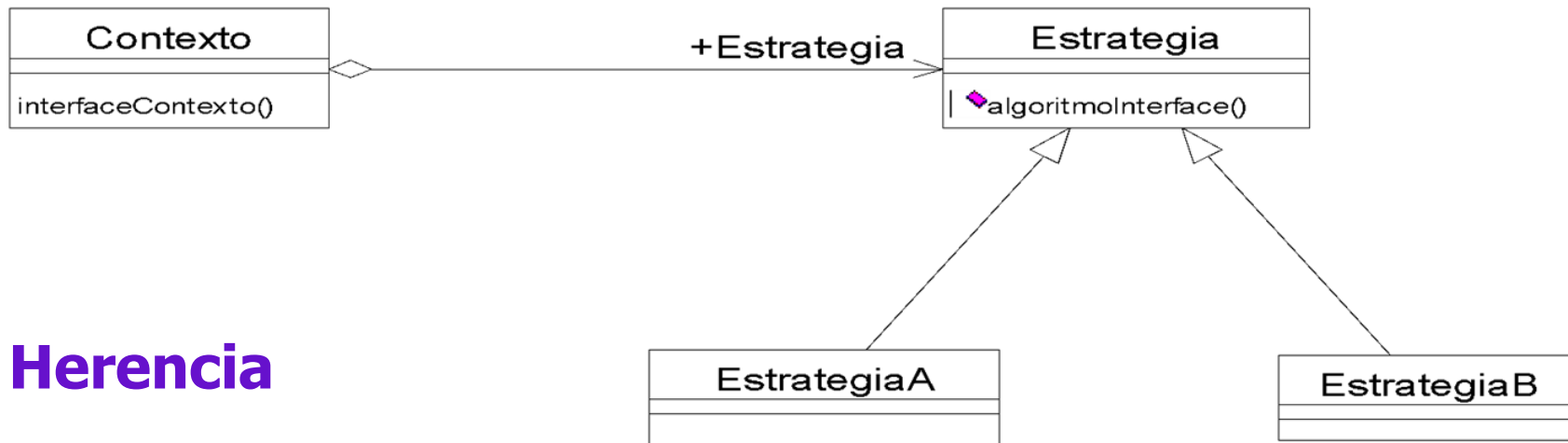
◎ Motivación

- ◎ Existen muchos algoritmos para justificación de texto, ¿debe implementarlo el cliente que lo necesita?

- Patrones de Creación
- Patrones de Estructura
- Patrones de Comportamiento

Strategy

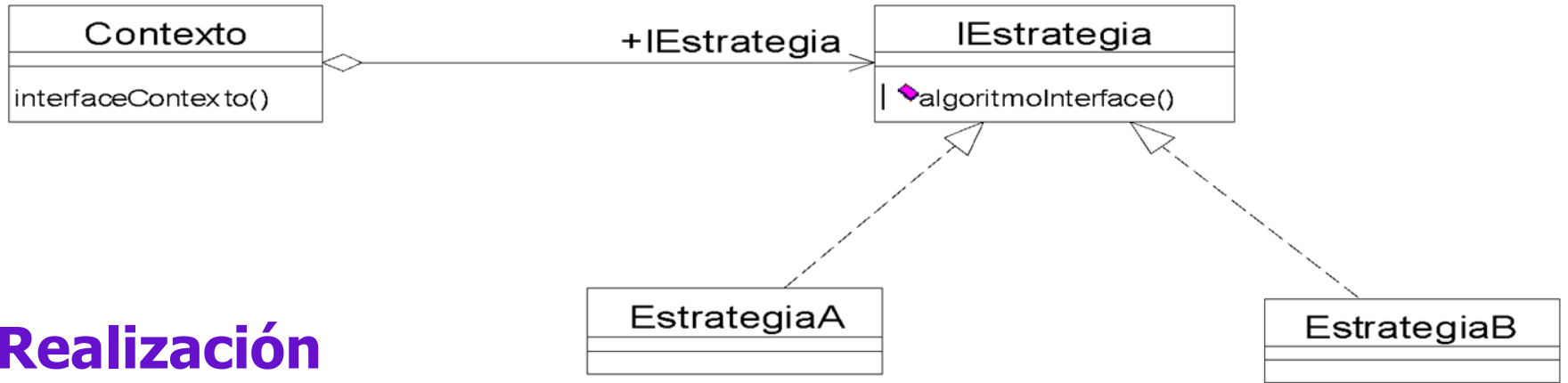
Estructura



Herencia

Strategy

Estructura



Realización

Strategy

◎ Aplicabilidad

- ◎ Configurar una clase con uno de varios comportamientos posibles.
- ◎ Se necesitan diferentes variantes de un algoritmo.
- ◎ Una clase define muchos comportamientos que aparecen como sentencias CASE en sus métodos.

- Patrones de Creación
- Patrones de Estructura
- Patrones de Comportamiento

Strategy

◎ Consecuencias

- ◎ Define una familia de algoritmos relacionados.
- ◎ Una alternativa a crear subclases de la clase *Contexto*.
- ◎ Elimina sentencias CASE
- ◎ El cliente puede elegir entre diferentes estrategias o implementaciones: debe conocer detalles
- ◎ Se incrementa el número de objetos:
 - Solución: usar *Flyweight*
- ◎ *State* y *Strategy* son similares, cambia el *Propósito*

- Patrones de Creación
- Patrones de Estructura
- Patrones de Comportamiento

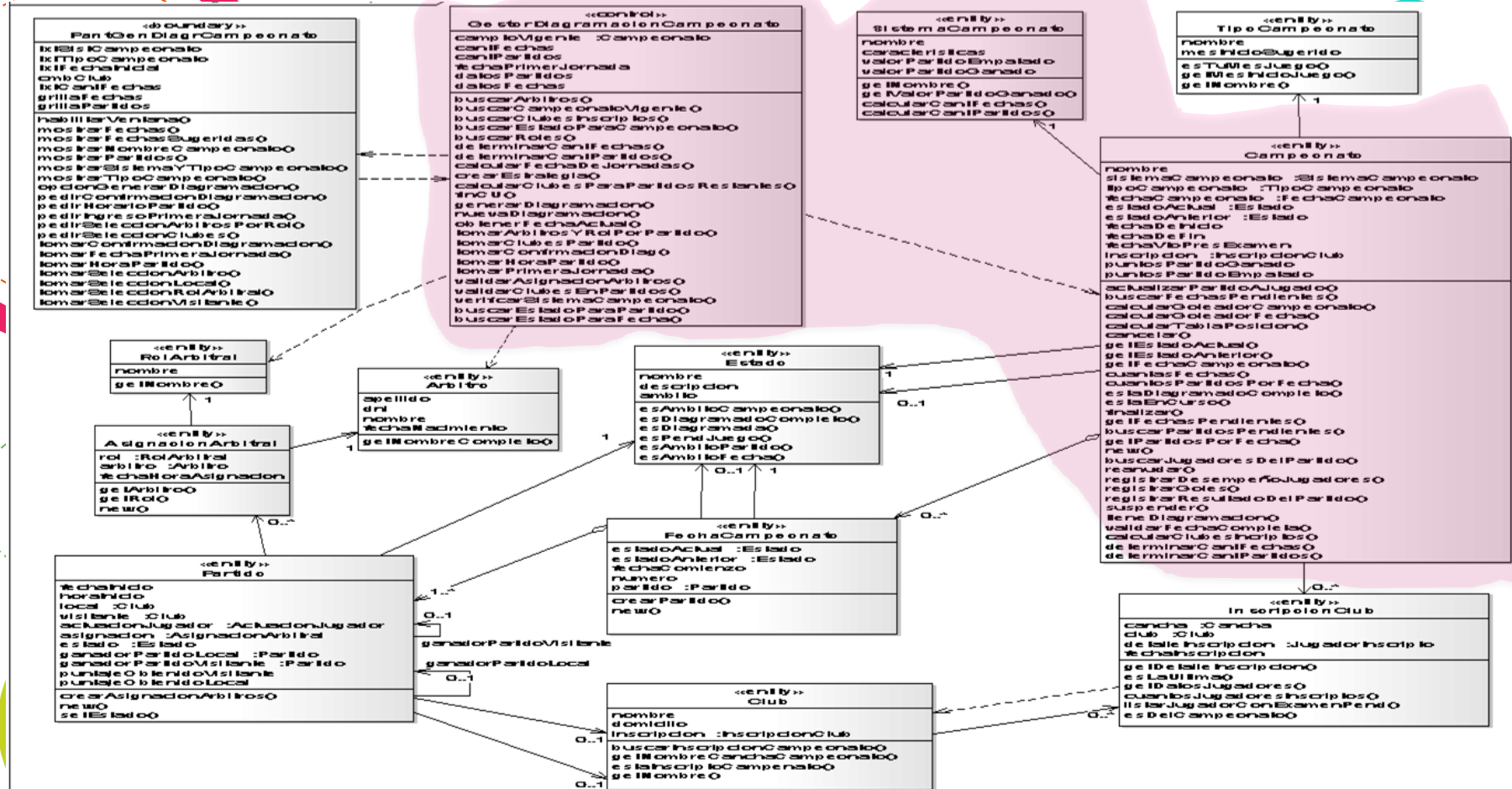
Flujo Básico

- 1- **AC:** selecciona la opción “**Generar Diagramación de Campeonato**”
- 2- **Sistema:** busca para el mes actual el campeonato que corresponde, que esté vigente y no tenga diagramación asociada y lo muestra, mostrando también el nombre del tipo de campeonato y el sistema de campeonato asociado.
- 3- **Sistema:** Busca los clubes inscriptos para el campeonato y calcula la cantidad de fechas y la cantidad de partidos a realizarse según el sistema de campeonato asociado, y las muestra.
- 4- **Sistema:** solicita que se indique la fecha de la primera jornada (fecha).
- 5- **AC:** Ingresa la fecha.
- 6- **Sistema:** Valida el sistema de campeonato seleccionado y **es todos contra todos**. Calcula las fechas de todas las jornadas sumando siete días a cada fecha y permite modificarlas.
- 7- **AC:** No desea modificar las fechas.
- 8- **Sistema:** Solicita que se ingrese la hora en que se jugará cada partido en las fechas.
- 9- **AC:** Ingresa la hora en que se jugará cada partido.
- 10- **Sistema:** El sistema de campeonatos seleccionado y **es todos contra todos**. *Para cada partido de cada fecha solicita* que se seleccione el club que será local y el club que será visitante, para la primera fecha y en función de eso determina las rondas restantes.
- 11- **Sistema:** Solicita se seleccionen los árbitros y el rol de cada uno (árbitro principal, juez de línea, cuarto árbitro)
- 12- **AC:** Selecciona los datos solicitados.
- 13- **Sistema:** Solicita se confirme la diagramación del campeonato.
- 14- **AC:** Confirma la diagramación del campeonato completa.
- 15- **Sistema:** valida que no exista más de un partido dónde se enfrenten los mismos equipos y no es así.
- 16- **Sistema:** Valida para cada partido que un mismo árbitro no esté seleccionado más de una vez para un partido (un árbitro puede cumplir un único rol en un partido) y que no se superpongan partidos para un árbitros y es así.
- 17- **Sistema:** El sistema de campeonato seleccionado y **es todos contra todos**. Registra la diagramación del campeonato, registrando para cada fecha los partidos a jugar con sus respectivas horas, clubes (local y visitante) y árbitros y asignando el estado “Diagramada” a cada fecha y partido.

Flujos Alternativos

- A.1. No hay ningún campeonato sin diagramación.
- A.2. El Sistema de campeonatos es **de eliminatorias**, debe calcular las fechas de todas las rondas sumando siete días entre cada ronda.
- A.3. Se modifican las fechas propuestas para cada jornada.
- A.4. El sistema de campeonatos seleccionado es **por eliminatorias**. Se debe definir clubes que jugarán, los árbitros y el rol de cada uno (árbitro principal, juez de línea, cuarto árbitro) para cada partido.
- A.5 La diagramación del campeonato queda incompleta.

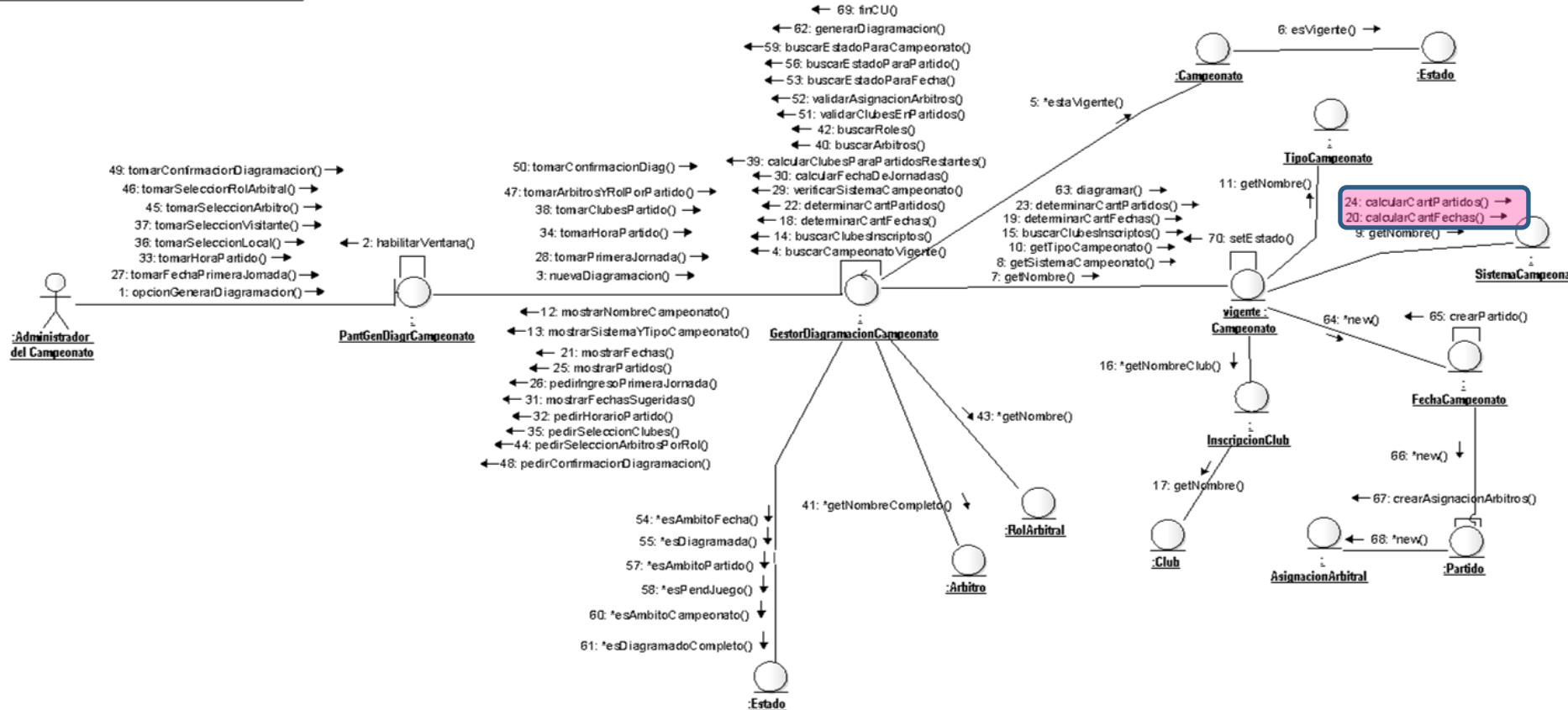
Caso Liga de Fútbol – Realización de CU de Análisis del Caso de Uso Generar Diagramación del Campeonato

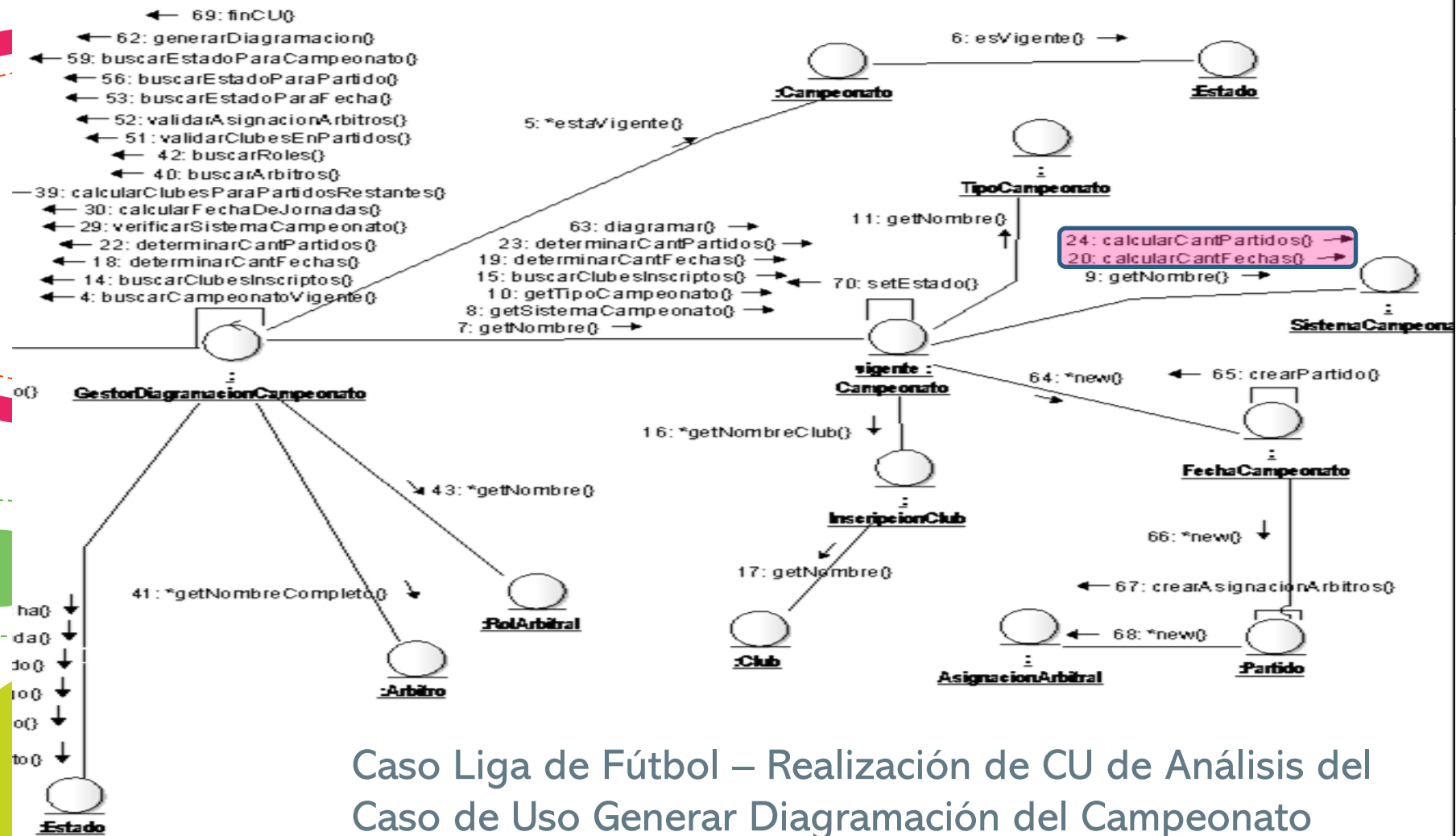


Caso Liga de Fútbol – Realización de CU de Análisis del

Caso de Uso Generar Diagramación del Campeonato

Realización de Diagramación del Campeonato

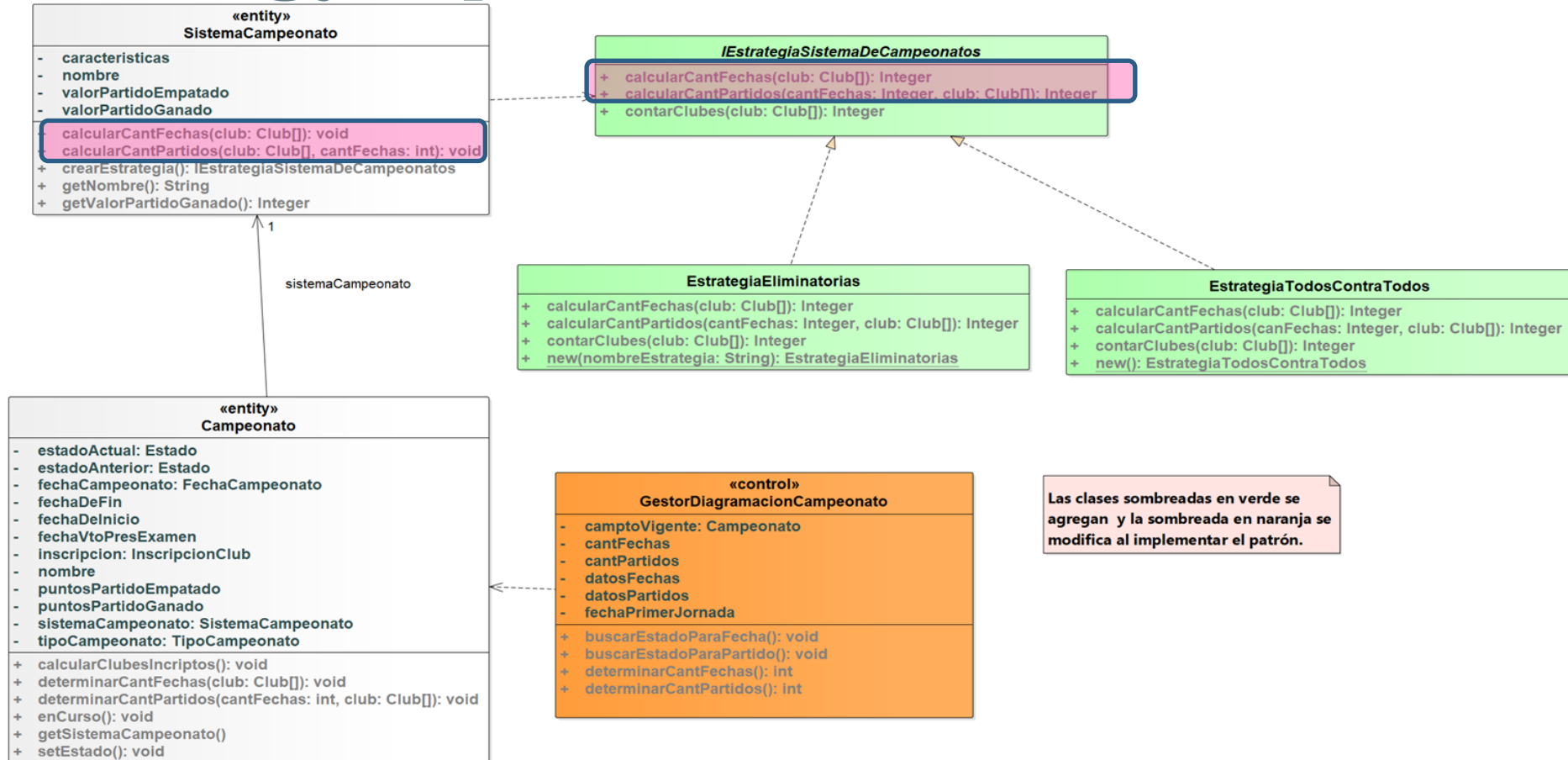




Caso Liga de Fútbol – Realización de CU de Análisis del Caso de Uso Generar Diagramación del Campeonato

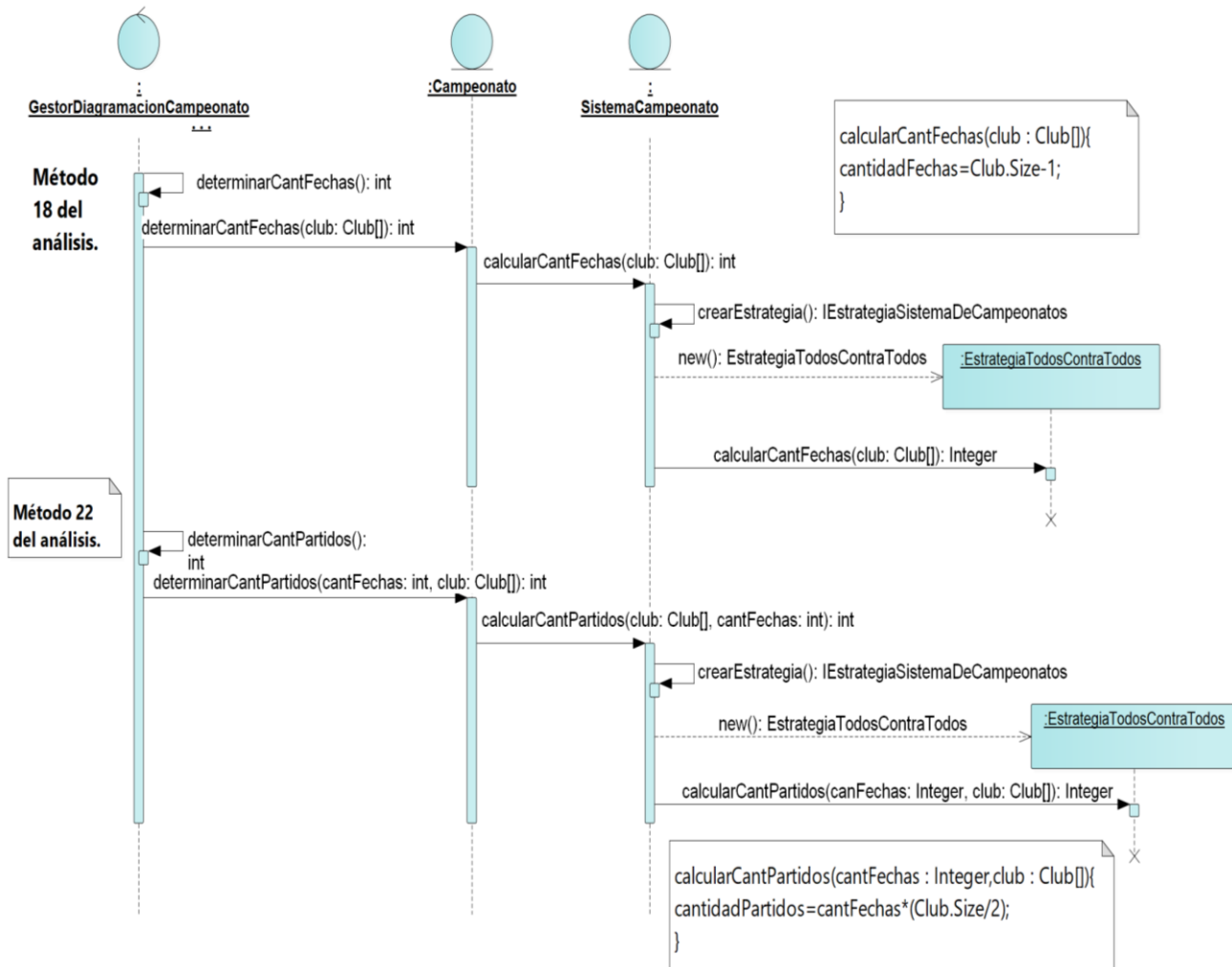
Strategy: Aplicación

Estructura



Strategy: Aplicación

• Dinámica



Strategy: Aplicación

◎ Participantes

- ◎ **Estrategia** [**IEstrategiaSistemadeCampeonato**]:
 - Declara una interfaz común a todos los algoritmos permitidos que define los métodos disponibles para ser utilizados por el cliente.
- ◎ **Estrategia Concreta** [**Eliminatorias y TodosContraTodos**]:
 - Implementan las variantes de los algoritmos de las operaciones definidas en la interfaz estrategia: **calcularCantFechas()** y **calcularCantPartidos()**.
- ◎ **Contexto** [**SistemaCampeonato**]
 - Es la clase que utiliza las diferentes estrategias para ciertas tareas. Mantiene una referencia a una o más instancias Estrategia Concreta, que podrán ser configurables en tiempo de ejecución.
 - La configuración puede ser de una o varias estrategias, y debe realizarse antes de invocarse el método de variación algorítmica de cada una de las Estrategias Concretas.
 - El Contexto es responsable establecer la estrategia a utilizar y de ejecutar el o los métodos polimórficos. En este caso

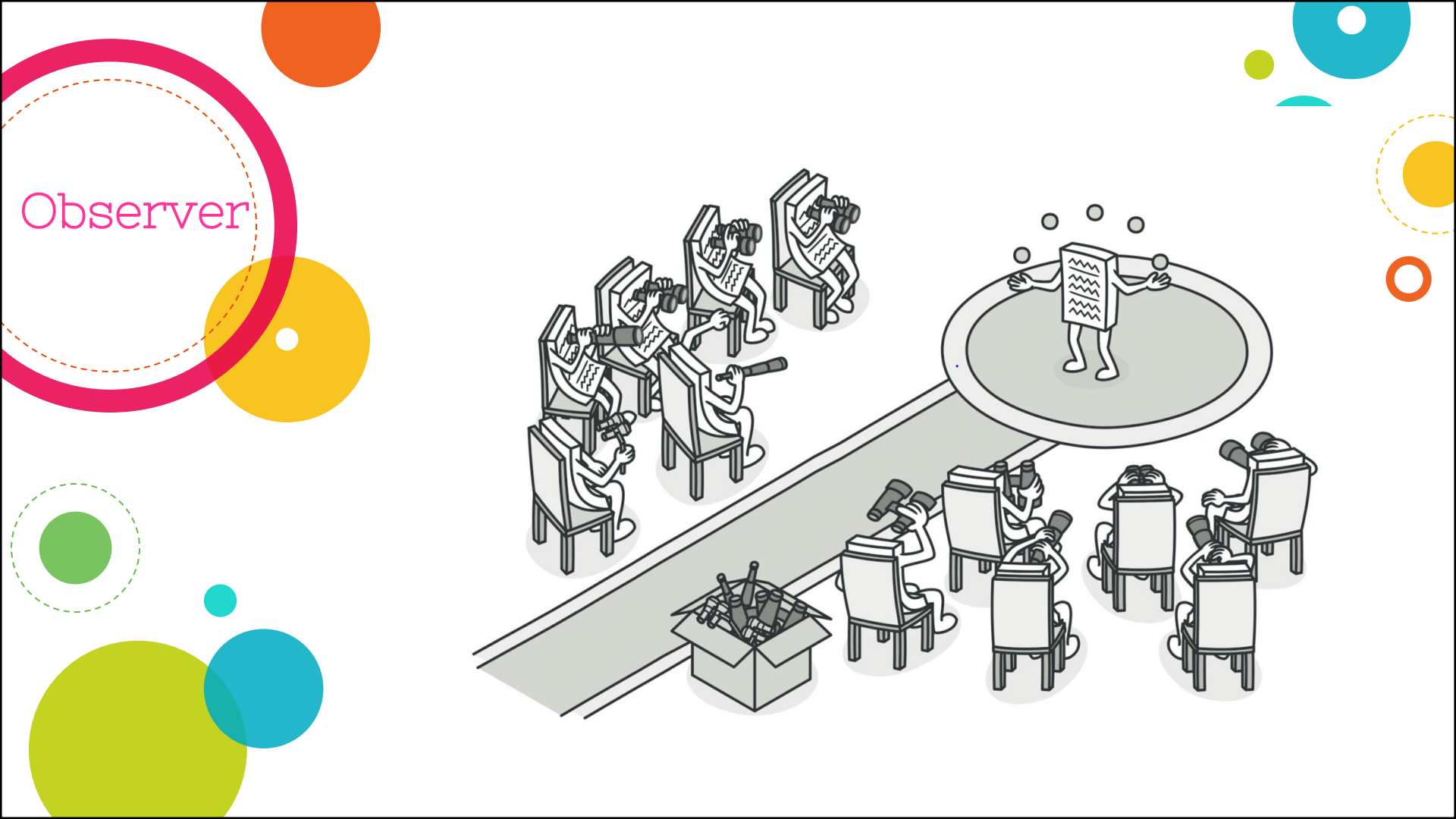
Strategy: Aplicación

◎ Procedimiento

- ◎ Identificar los métodos cuyo algoritmo posee diferentes variantes o implementaciones [`calcularCantFechas()` y `calcularCantPartidos()`].
- ◎ Definir la interfaz Estrategia Abstracta `IEstrategiaSistemadeCampeonato` que define la interfaz común para las variantes.
- ◎ Establecer en el Contexto [`SistemaCampeonato`] la referencia hacia la Estrategia Abstracta.
- ◎ Definir como se configurará el Contexto con la/s estrategia/s deseada/s.
- ◎ Reconocer las diferentes variantes de los algoritmos e implementar las clases EstrategiaConcreta [`Eliminatorias` y `TodosContraTodos`] para cada comportamiento específico.

State/Strategy

- ⦿ El Patrón State permite a un objeto tener varios comportamientos diferentes que están basados en su estado interno.
- ⦿ A diferencia de una máquina de estados procedural, el patrón State representa un estado como una clase completa.
- ⦿ El “contexto” obtiene su comportamiento delegando al estado actual del objeto por el que está compuesto.
- ⦿ Encapsulando cada estado en una clase, localizamos cualquier cambio que sea necesario hacer.
- ⦿ Los patrones STATE/STRATEGY tienen el mismo diagrama de clases, pero con una intención diferente.
- ⦿ El patrón STRATEGY comúnmente configura la clase contexto con un comportamiento o algoritmo.
- ⦿ Las transiciones de los estados pueden ser controladas por las clases “estado” o por las clases “contexto”.
- ⦿ La utilización del patrón STATE resultará en un incremento en el número de clases de su diseño.
- ⦿ Las clases estado pueden compartirse entre instancias de contexto.



Observer

Observer / Publish-Subscribe

◎ Propósito

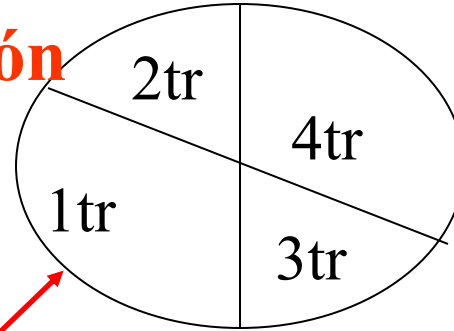
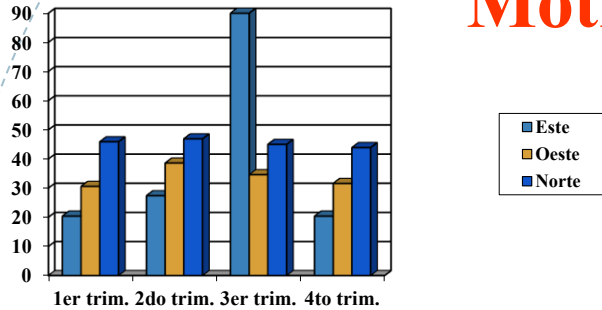
- Define una dependencia uno-a-muchos entre objetos, de modo que cuando cambia el estado de un objeto, todos sus dependientes automáticamente son notificados y actualizados.

◎ Motivación

- ¿Cómo mantener la consistencia entre objetos relacionados, sin establecer un acoplamiento fuerte entre sus clases?
- Ejemplo: Separación *Modelo-Vista*
 - Patrones de Creación
 - Patrones de Estructura
 - Patrones de Comportamiento

Motivación

Vistas



Modelo

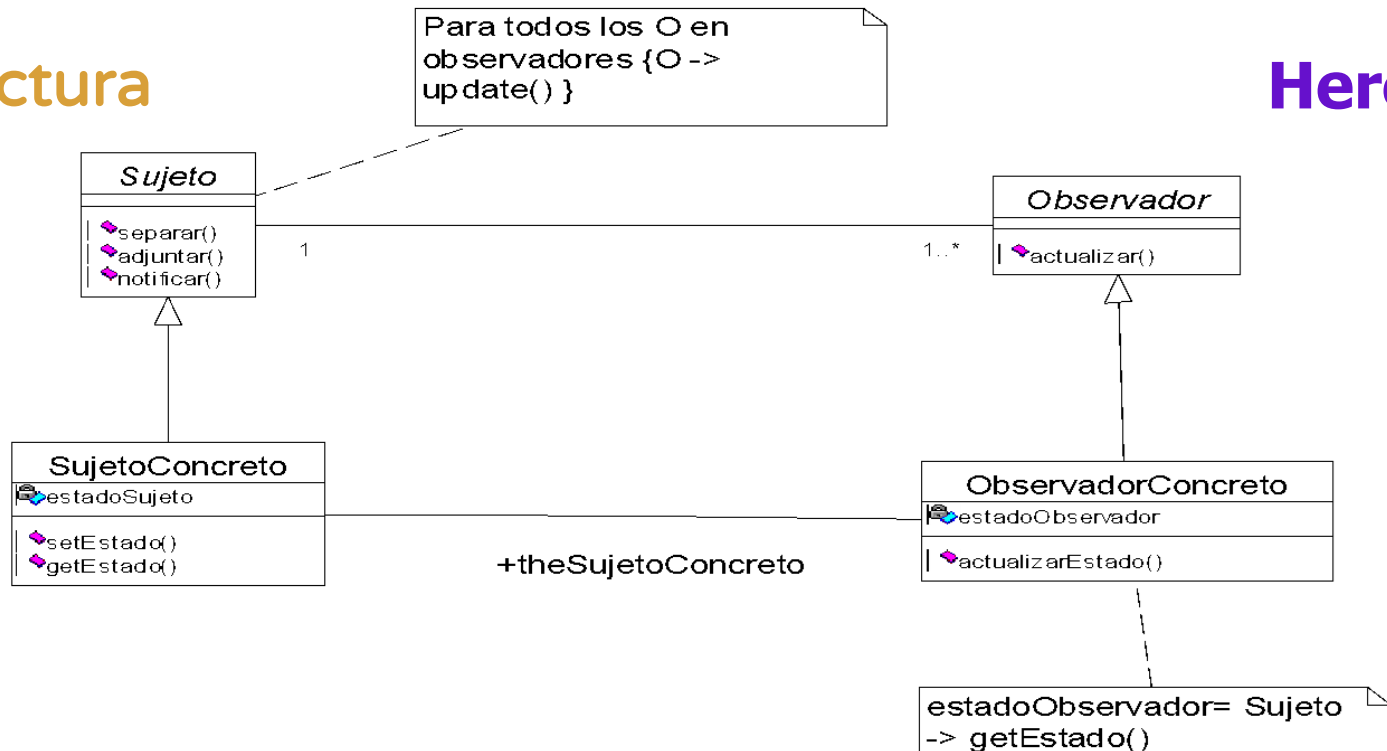
t1=20
t2=25
t3=90
t4=20

	<u>1tr.</u>	<u>2tr.</u>	<u>3tr.</u>	
<u>4tr.</u>				
Este	20	25	90	20
Oeste	30	38	32	32
Norte	47	47	45	45

Observer

Estructura

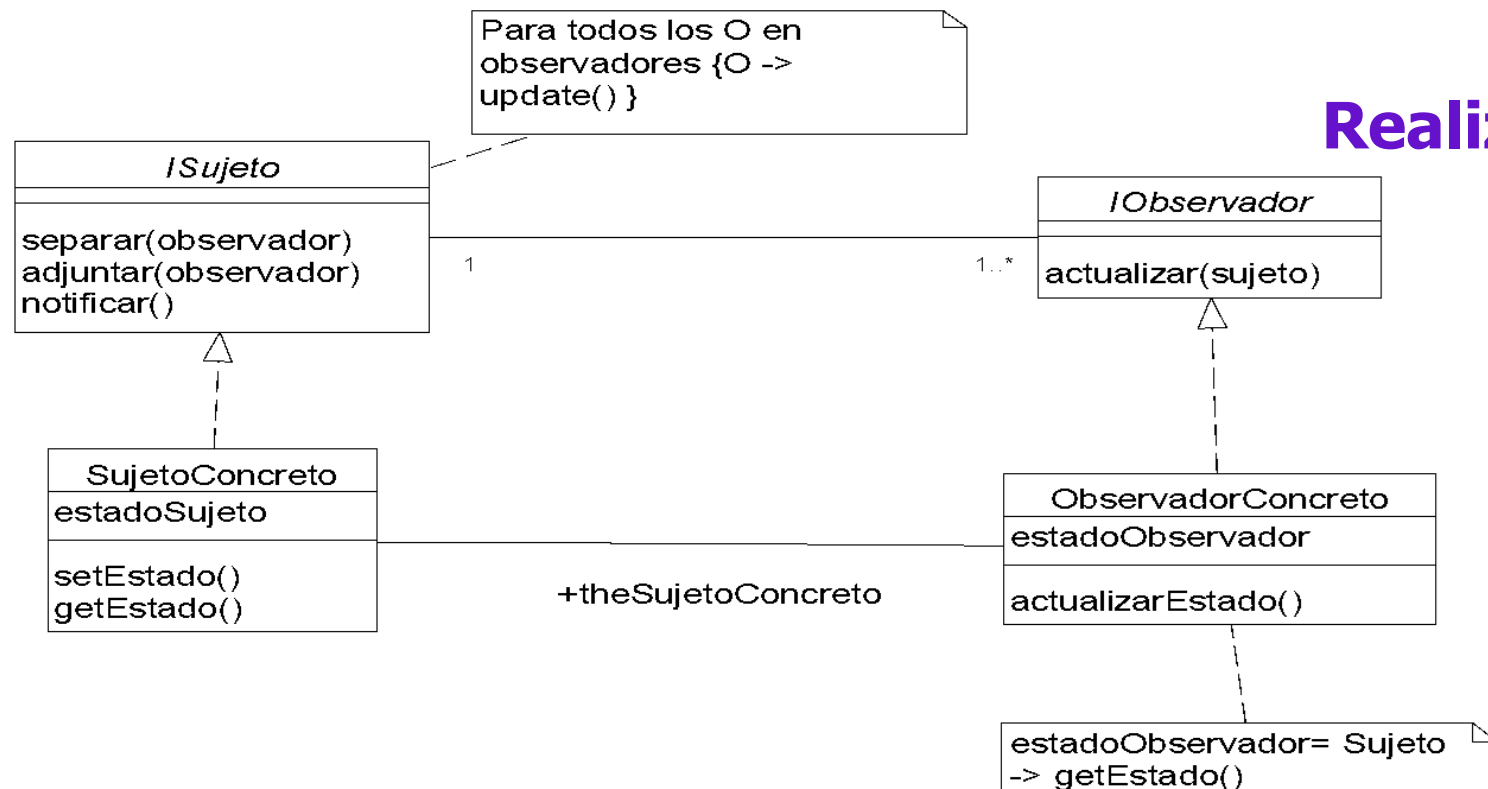
Herencia



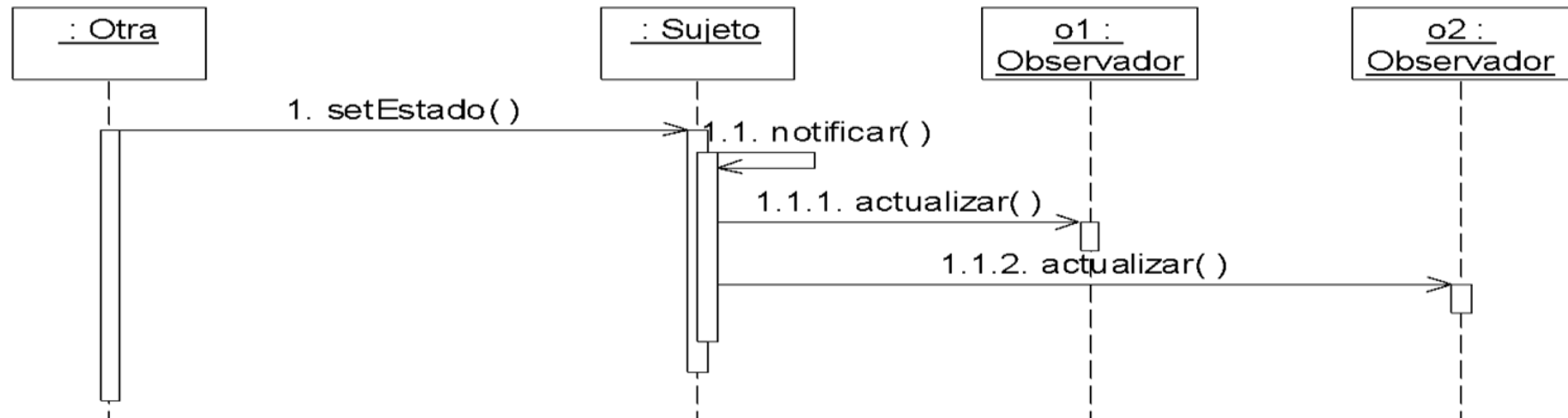
Observer

Estructura

Realización



Colaboración



Observer

◎ Aplicabilidad

- ◎ Cuando un cambio de estado en un objeto requiere cambios en otros objetos, y no sabe sobre cuántos objetos debe aplicarse el cambio.
- ◎ Cuando un objeto debe ser capaz de notificar algo a otros objetos, sin hacer asunciones sobre quiénes son estos objetos.

- Patrones de Creación
- Patrones de Estructura
- Patrones de Comportamiento

Observer

◎ Consecuencias

- Permite modificar independientemente *subjects* y *observers*.
- Acoplamiento abstracto y mínimo entre *Subject* y *Observer*:
 - pueden reutilizarse por separado
 - *observers* pueden añadirse sin modificar el *subject*
 - *Subject* no necesita conocer las clases concretas de *Observers*

- Patrones de Creación
- Patrones de Estructura
- Patrones de Comportamiento

Observer

◎ Consecuencias

- ◎ Soporte para comunicación de tipo *broadcast*:
 - el *subject* no especifica al receptor concreto de la notificación
 - es posible añadir y eliminar *observers* en cualquier instante.
- ◎ Actualizaciones inesperadas
- ◎ Interfaz de *Observer* simple requiere que los *observers* tengan que deducir el ítem cambiado

- Patrones de Creación
- Patrones de Estructura
- Patrones de Comportamiento

Observer

- Cuando la semántica de las actualizaciones es compleja, puede ser necesario que un objeto mantenga esas relaciones, GestorDeCambios.
- El propósito del GestorDeCambios es minimizar el trabajo necesario para lograr que los observadores reflejen el cambio en su sujeto.
- GestorDeCambios tiene las siguientes responsabilidades:
 - Corresponder a un sujeto con sus observadores.
 - Definir una determinada estrategia de actualización
 - Actualiza todos los observadores dependientes a petición de un sujeto.
- El GestorDeCambios es una instancia del patrón Mediator y como sólo hay un único GestorDeCambios el patrón Singleton es de utilidad aquí.
 - Patrones de Creación
 - Patrones de Estructura
 - Patrones de Comportamiento

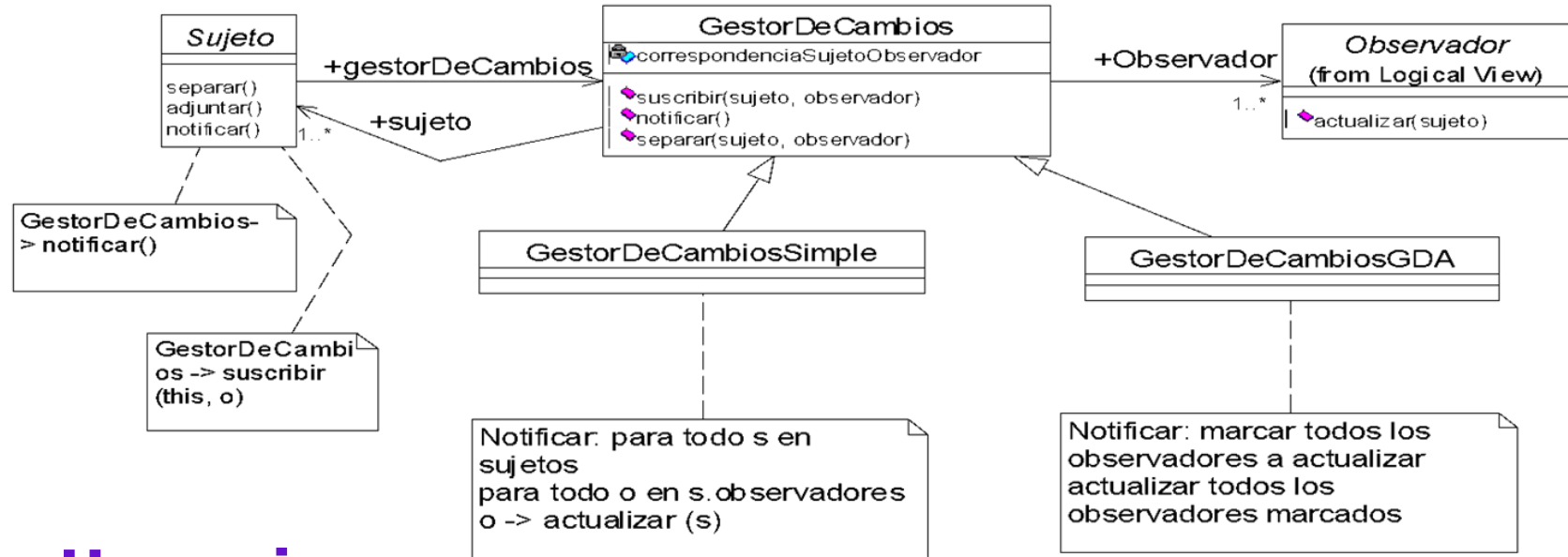
Observer: Aplicación

◎ Consecuencias

- ◎ Reusabilidad: Al separar el Sujeto de sus Observadores se permite reutilizar sujetos y observadores independientemente.
- ◎ Flexibilidad: Se pueden agregar y quitar observadores en cualquier momento sin afectar al sujeto ni ha otros observadores, ya que el Sujeto siempre ejecuta el método notificar() para todos los observadores existentes.
- ◎ Disminuye el Acoplamiento: el sujeto conoce a sus observadores en las clases Madre, lo cual reduce el acoplamiento entre las clases instanciables, ya que la relación es abstracta.
- ◎ Es posible asegurar la consistencia entre Sujeto y Observadores aplicando el patrón Template Method, haciendo que al finalizar la actualización del Sujeto se llame al método notificar(), evitando el riesgo de llamarlo cuando aún no se ha producido la actualización.

Observer: Aplicación

Estructura con Gestor de Cambios

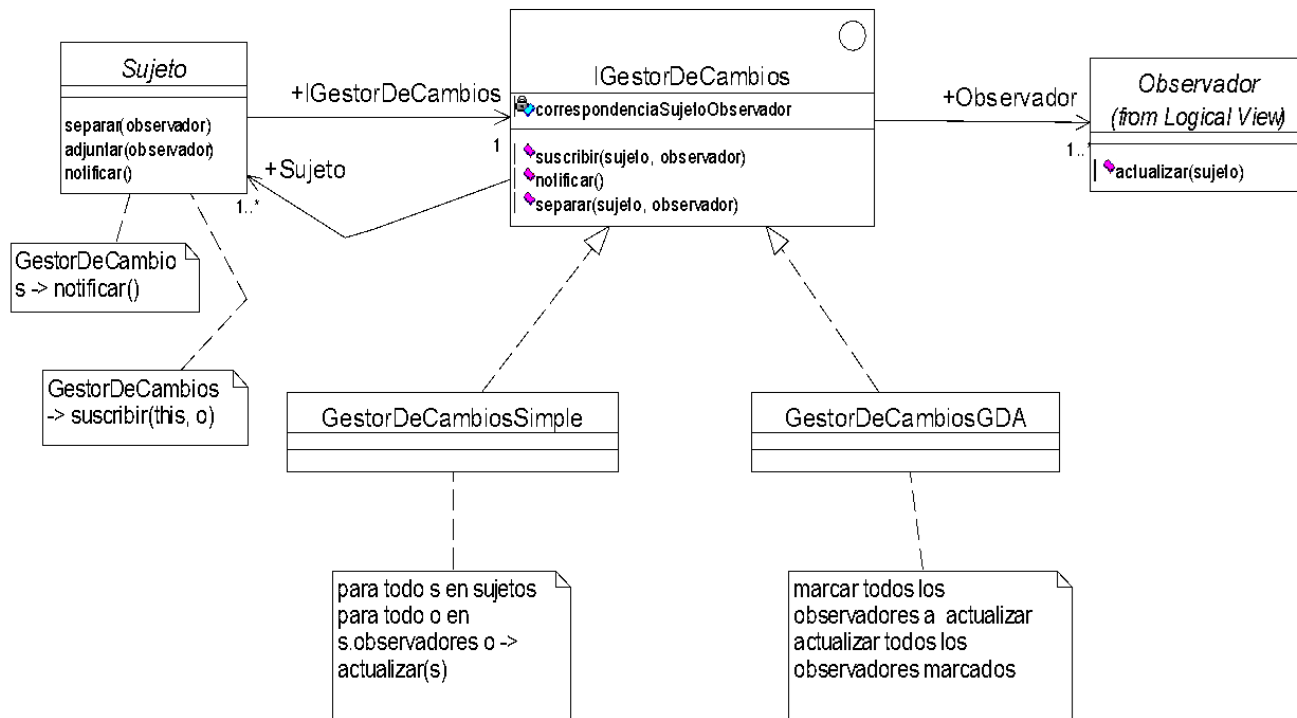


Herencia

Observer: Aplicación

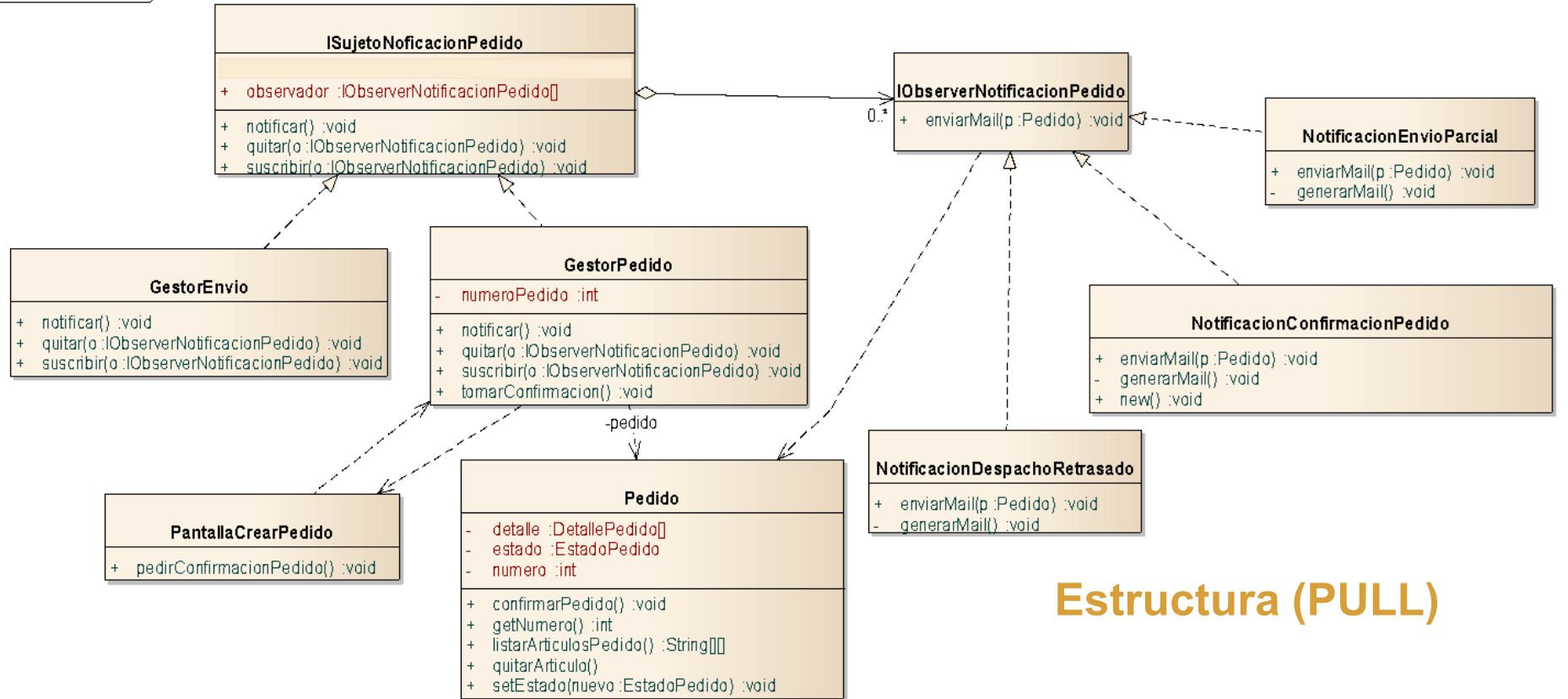
Estructura con Gestor de Cambios

Realización



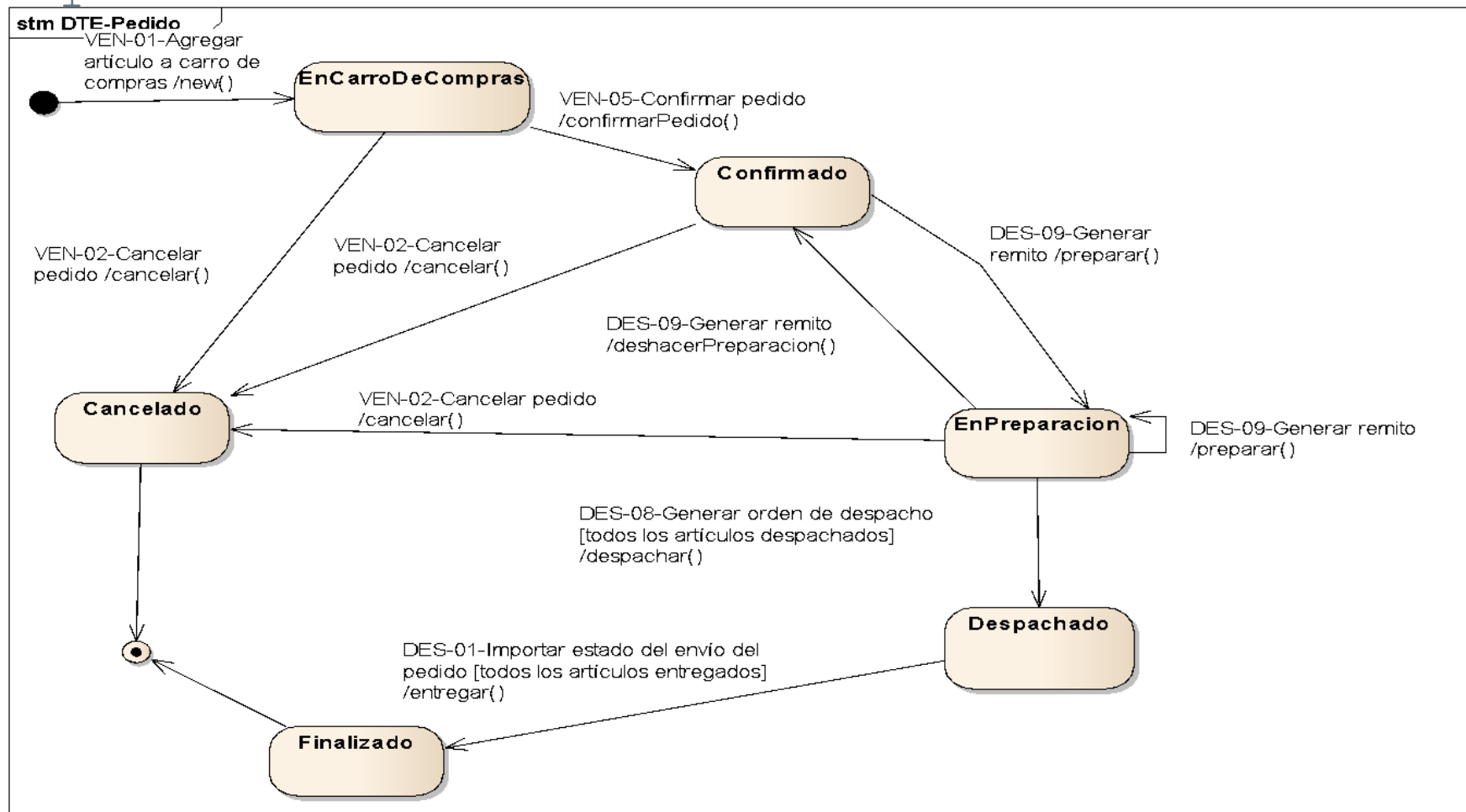
Observer: Aplicación

class Observer-Pull



Estructura (PULL)

Máquina de Estado de la Clase Pedido



Observer: Aplicación

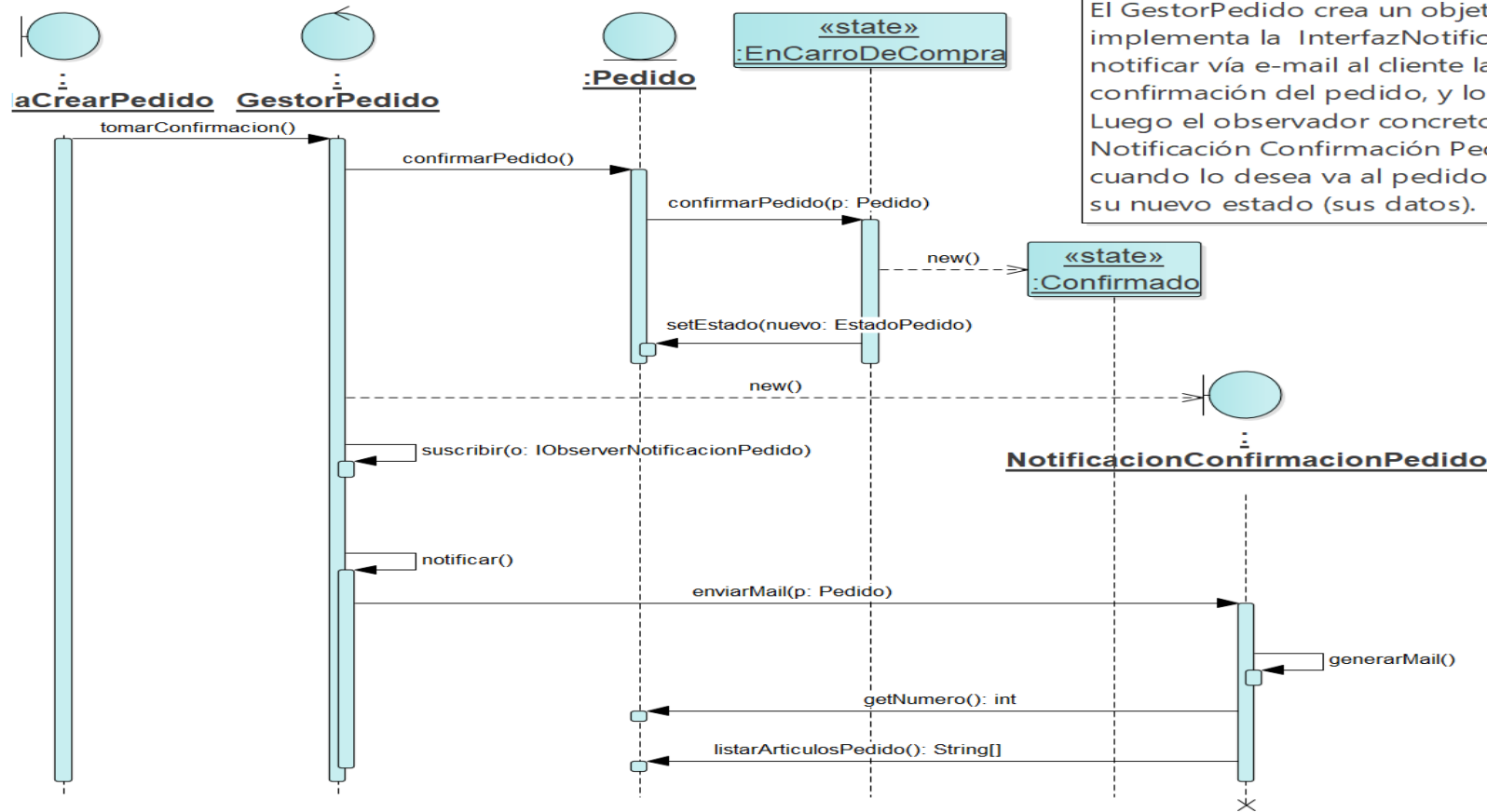
⊙ Participantes

- ⊙ **Sujeto** [**ISujetoNotificacionPedido**]: Proporciona una interfaz para notificar a sus Observadores de sus modificaciones.
- ⊙ **Observador** [**IObserverNotificacionPedido**]: Proporciona una interfaz para actualizar los objetos que deben enterarse de cambios en el Sujeto.
- ⊙ **SujetoConcreto** [**GestorEnvio y GestorPedido**]: Almacena el estado (valor de los atributos) que importan a los Observadores y notifica a los mismos sobre cambios ocurridos.
- ⊙ **ObservadorConcreto** [**NotificacionConfirmacionPedido, NotificacionDespachoRetrasado, NotificacionEnvioParcial**]

Observer : Aplicación

Dinámica (Pull)

sd Observer-Pull



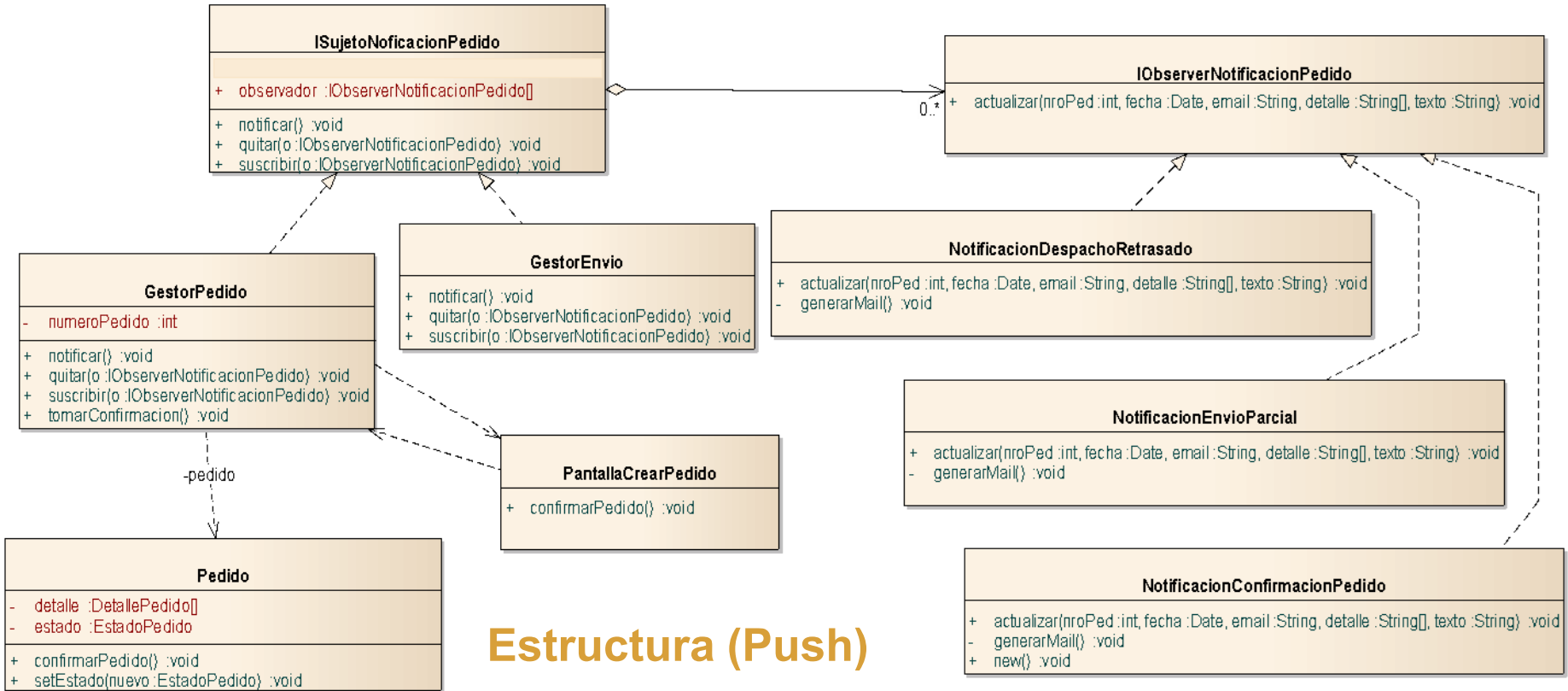
Observer: Aplicación

◎ Procedimiento

- ◎ Cuando se produce un cambio en el estado del pedido, el se deberá notificar a todos los suscriptos (InterfazNotificacion), para que realicen la notificación correspondiente en función del estado del Pedido.
- ◎ El **GestorPedido** corrobora que el estado del pedido se ha modificado, por lo que llama a su método **notificar()** e informa a las **Notificaciones**.
- ◎ En función del estado del pedido se analiza que **Notificación** corresponde que ejecute el método **enviarMail()** declarado en la interfaz **IObserverNotificacionPedido**.
- ◎ El método **generarMail()** que se implementa en cada **Notificación** concreta que corresponda, pide las actualizaciones que corresponden, en este caso al objeto Pedido.

Observer: Aplicación

class Observer-Push

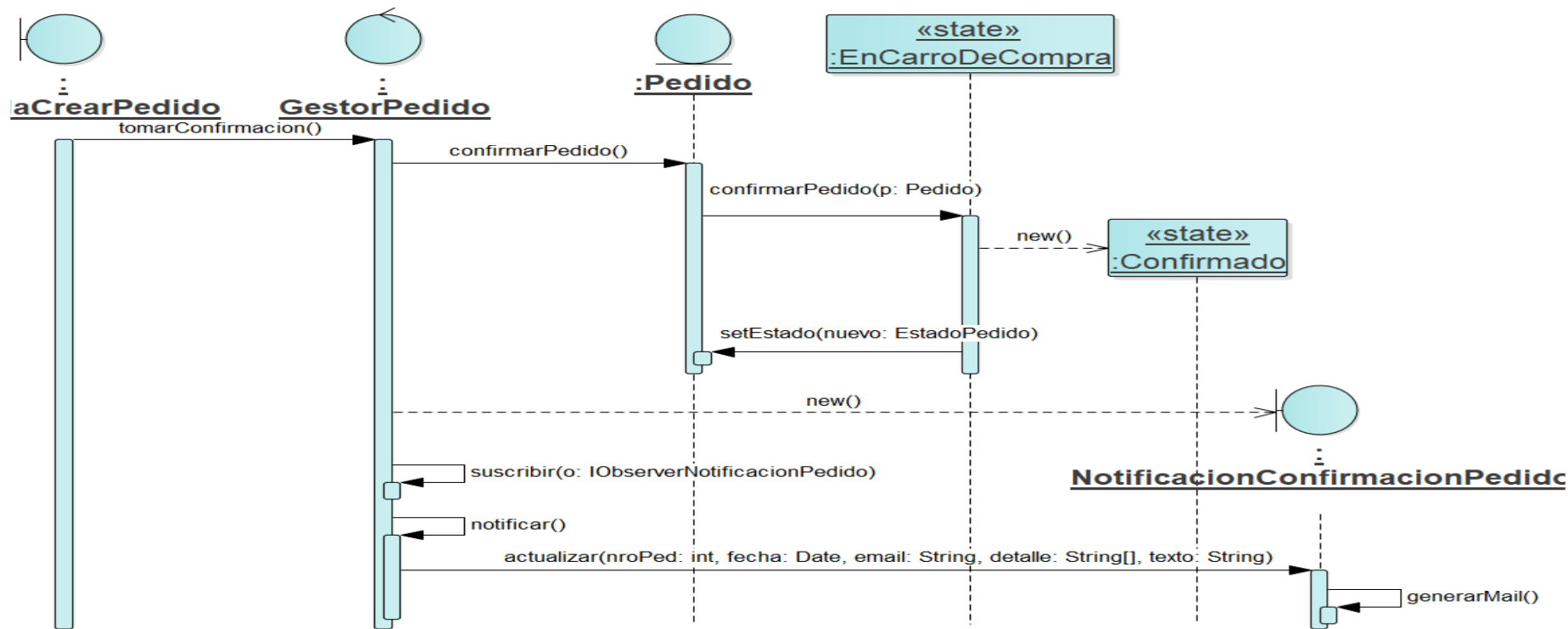


Estructura (Push)

Observer : Aplicación

Dinámica (Push)

Observer-Push



Tras producirse el cambio de estado del Pedido, se notifica y se envía datos forzosamente a los observadores suscriptos al sujeto GestorPedido. Luego los Observadores disponen de esos datos para realizar sus operaciones.

Observer: Aplicación

◎ Participantes

- ◎ **Sujeto** [**ISujetoNotificacionPedido**]: Proporciona una interfaz para notificar a sus Observadores de sus modificaciones.
- ◎ **Observador** [**IObserverNotificacionPedido**]: Proporciona una interfaz para actualizar los objetos que deben enterarse de cambios en el Sujeto.
- ◎ **SujetoConcreto** [**GestorEnvio y GestorPedido**]: Almacena el estado (valor de los atributos) que importan a los Observadores y notifica a los mismos sobre cambios ocurridos.
- ◎ **ObservadorConcreto** [**NotificacionConfirmacionPedido, NotificacionDespachoRetrasado, NotificacionEnvioParcial**]: Debe mantener su estado consistente al del sujeto concreto, para lo cual implementa el método **actualizar()**.

Observer: Aplicación

◎ Procedimiento

Notificación de que el pedido ha sido registrado

- ◎ El **GestorPedido** manda la confirmación al **Pedido**, el **Pedido** delega esa confirmación al estado que se encuentra **Carrito**.
- ◎ El **Carrito** crea el nuevo estado **Confirmado** y se lo devuelve al **Pedido**.
- ◎ El **GestorPedido** debe notificar el cambio de estado y llama a su método **notificar()** y actualiza las pantallas de **Notificacion**.
- ◎ En función del estado del pedido se analiza que **Notificacion** corresponde que ejecute el método **actualizar()** declarado en la interfaz **IObserverNotificacionPedido**.
- ◎ La pantalla **NotificacionConfirmacionPedido**, es quien debe hacer algo en este caso, el método **actualizar()** llama al método concreto **generarMail()** de la **Notificacion** concreta que corresponda.

Adapter/ Wrapper

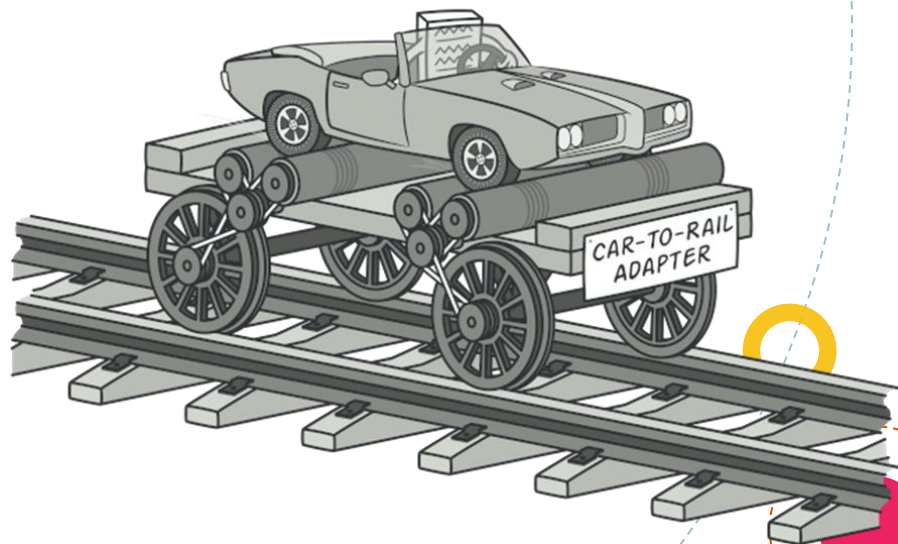
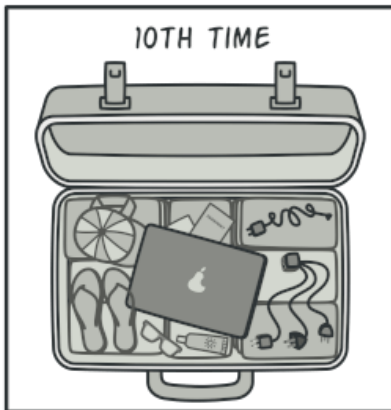
(Adaptador/ Envoltorio)

TRAVELING ABROAD

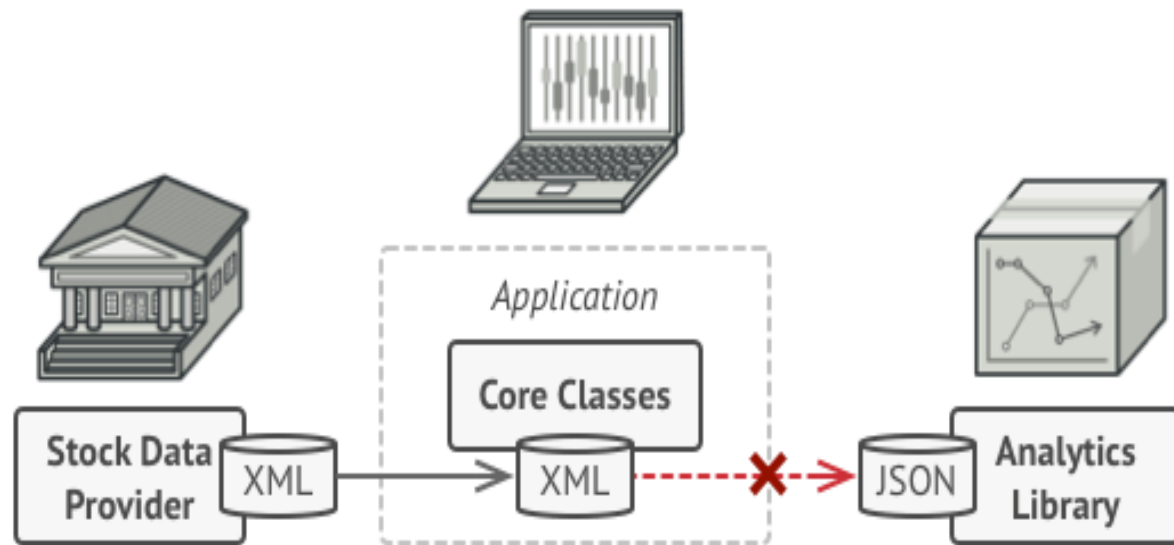
1ST TIME



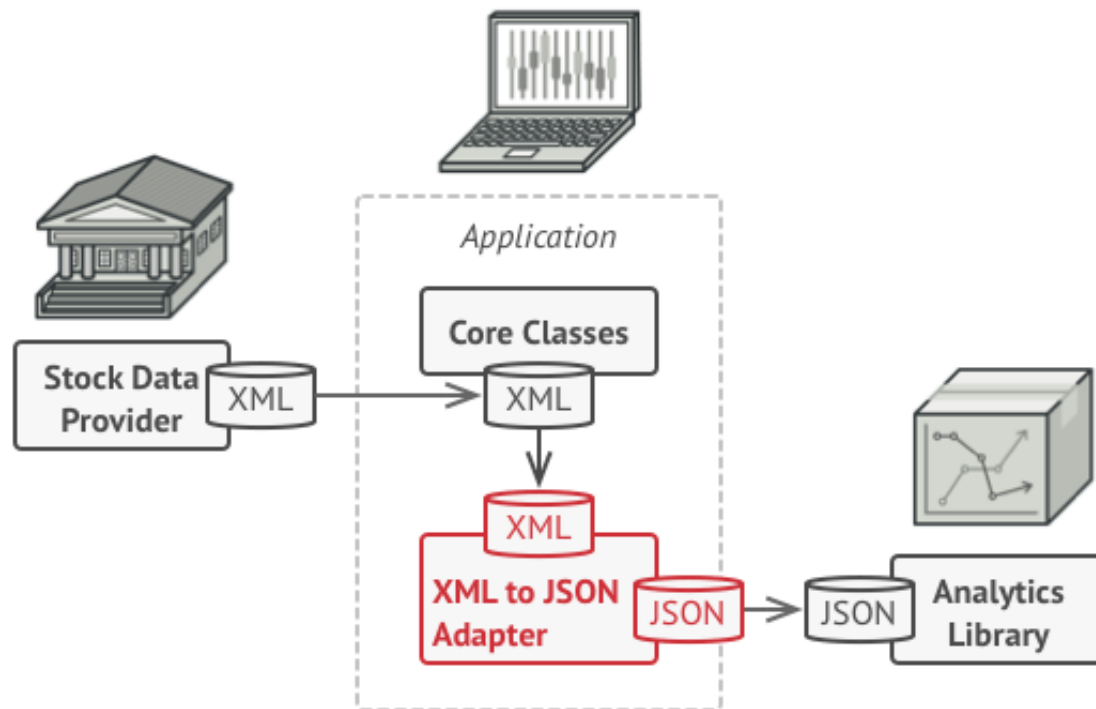
10TH TIME



¿Qué problema resuelve?



Solución: Patrón Adaptador



Solución: Patrón Adaptador

Propósito

Convertir la interfaz de un objeto para que otro pueda comprenderla.

Se dice que el Adaptador “**envuelve**” uno de los objetos para ocultar la complejidad de la conversión. El objeto “envuelto” no necesita conocer que se realiza una conversión.

Ámbito

De clases y de objetos



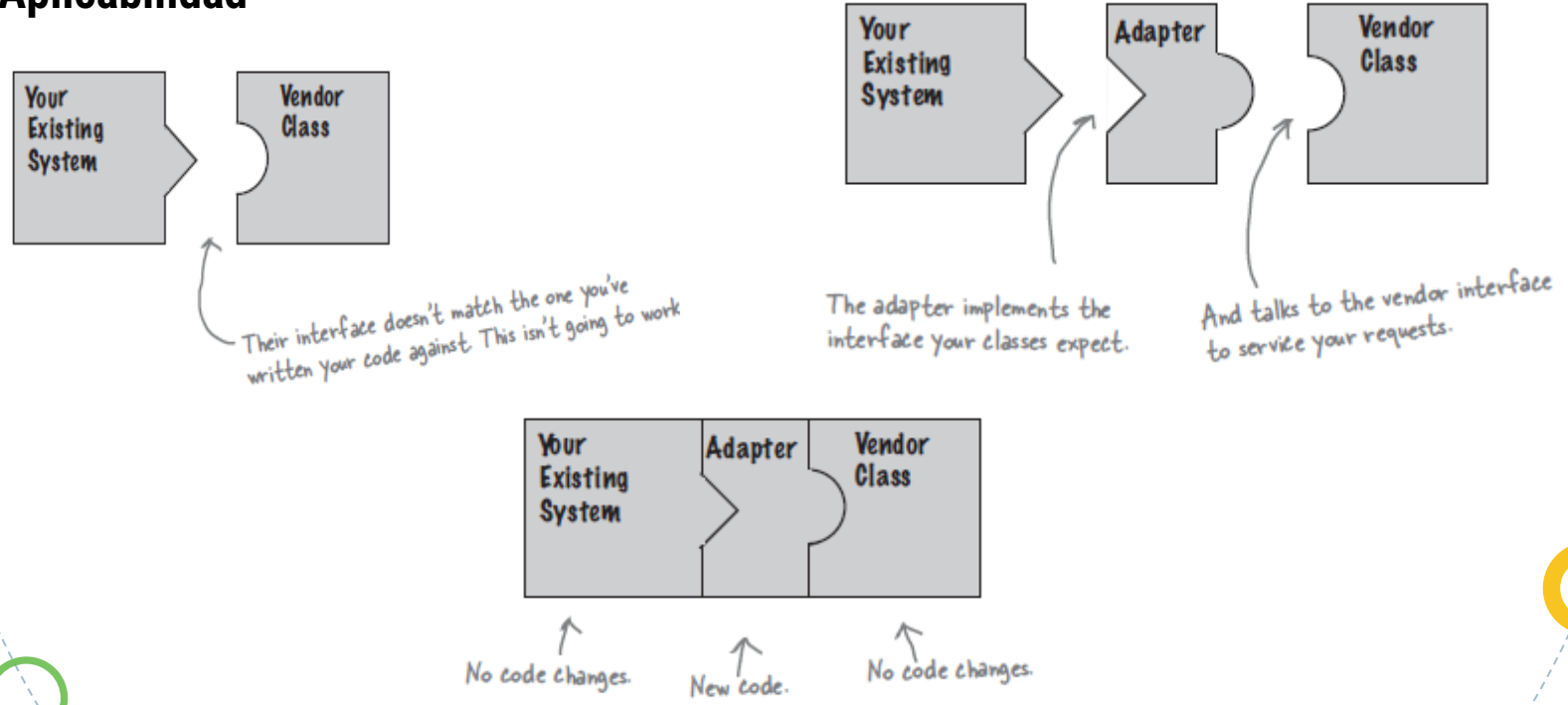
Solución: Patrón Adaptador

Aplicabilidad

- **Cuando queremos usar alguna clase existente, pero su interfaz no es compatible con el resto de su código.**
- Cuando queremos crear una clase reutilizable que coopere con clases no relacionadas o que no han sido previstas (que no tienen porqué tener interfaces compatibles)
- Cuando queremos usar varias subclases existentes, pero no resulta práctico adaptar su interfaz heredando de cada una de ellas.
- **Como buen patrón estructural, el cometido del patrón adapter está relacionado con la forma en la que los objetos interactúan entre ellos.**

Solución: Patrón Adaptador

Aplicabilidad



Solución: Patrón Adaptador

Implementación

1. Identificar a los actores: Esto sería determinar quién es el cliente y quiénes la parte a “adaptar”.
2. Diseñar una interfaz común, una que conozca el cliente y que todos los futuros adaptados puedan implementar.

Solución: Patrón Adaptador

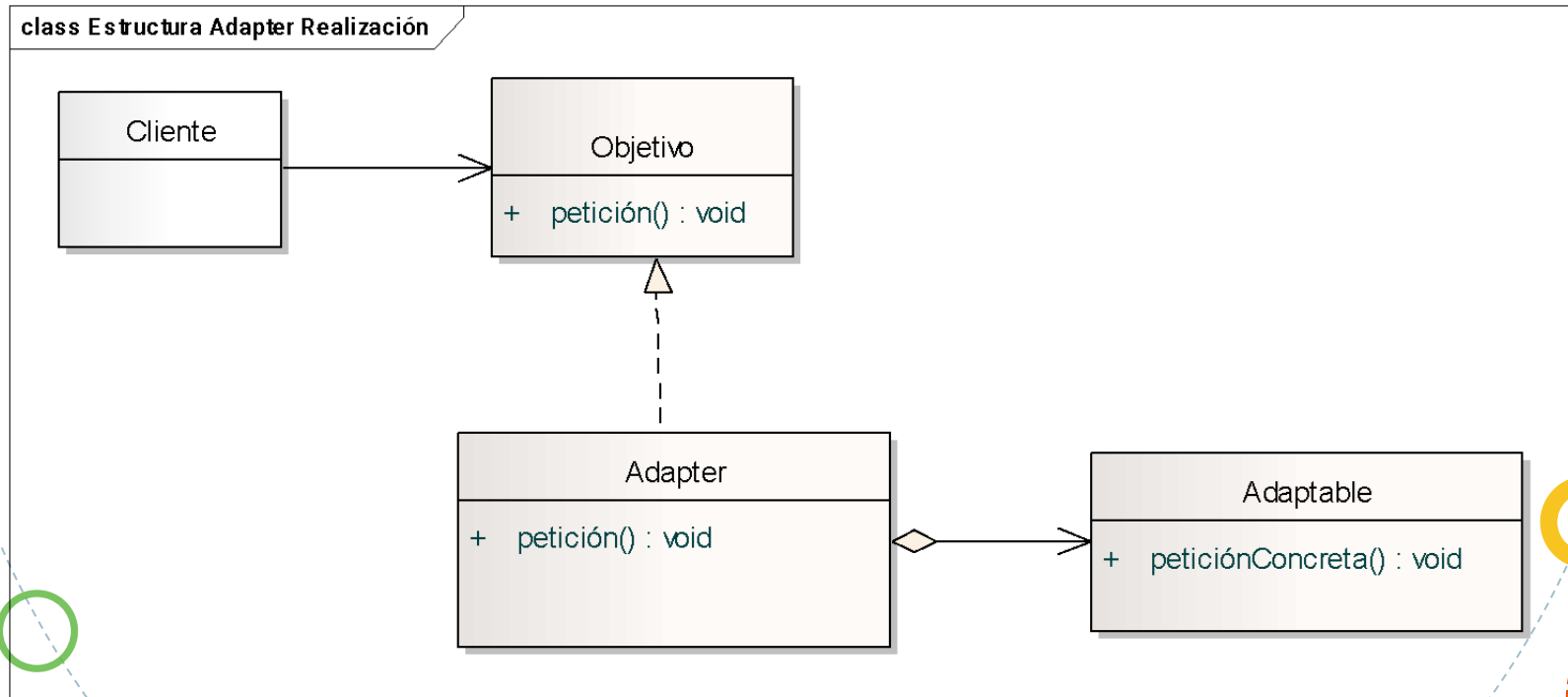
Implementación (Continuación)

3. Crear un adaptador que implemente dicha interfaz. Esto se haría para cada una de las partes a adaptar. Cada uno de estos adaptadores contendrá una instancia de la clase adaptada y “forzará” su uso para que se adapte a la nueva interfaz.
4. Desacoplar el cliente de la clase concreta. En lugar de recibir una clase concreta, a partir de ahora recibirá un objeto cualquiera que implemente la nueva interfaz.

Estructura – Adaptador de Objeto

Adapter 96

Esta implementación utiliza el principio de composición de objetos: el adaptador implementa la interfaz de un objeto y envuelve el otro. Se puede implementar en todos los lenguajes de programación populares.

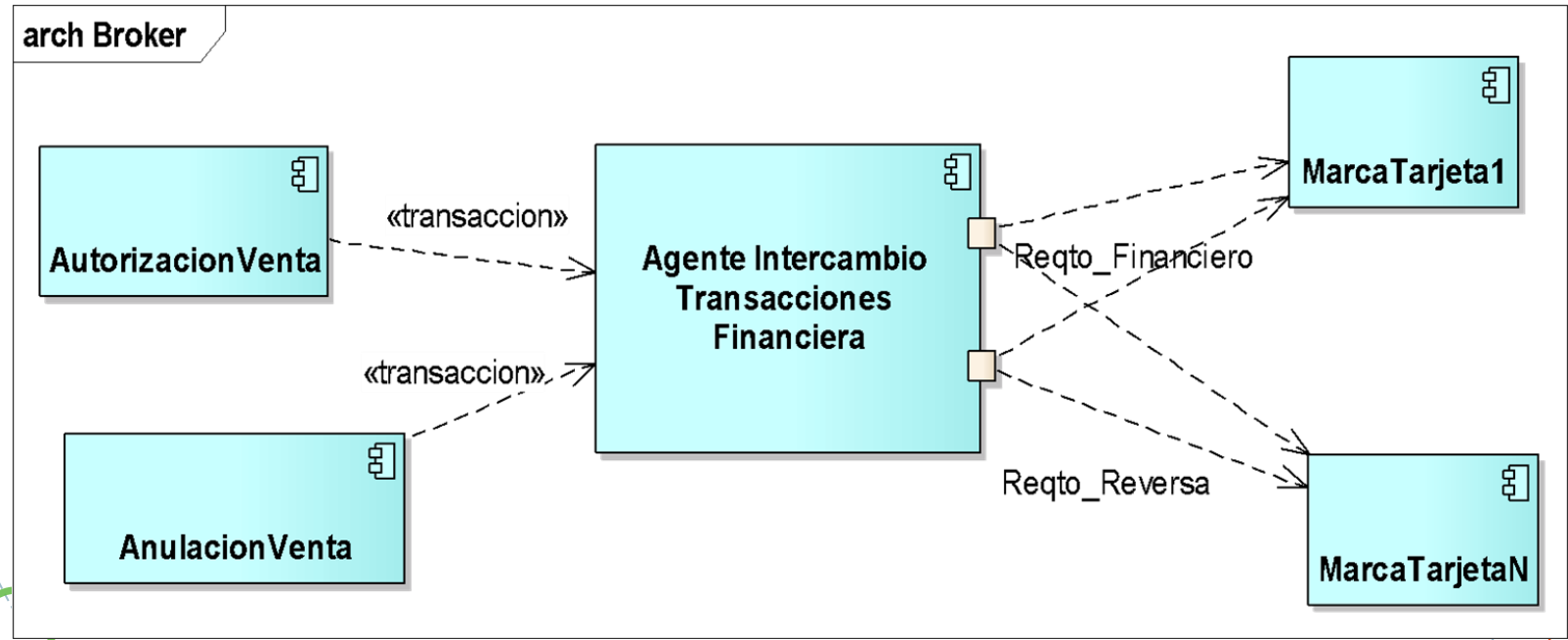


Aplicación Venta Digital

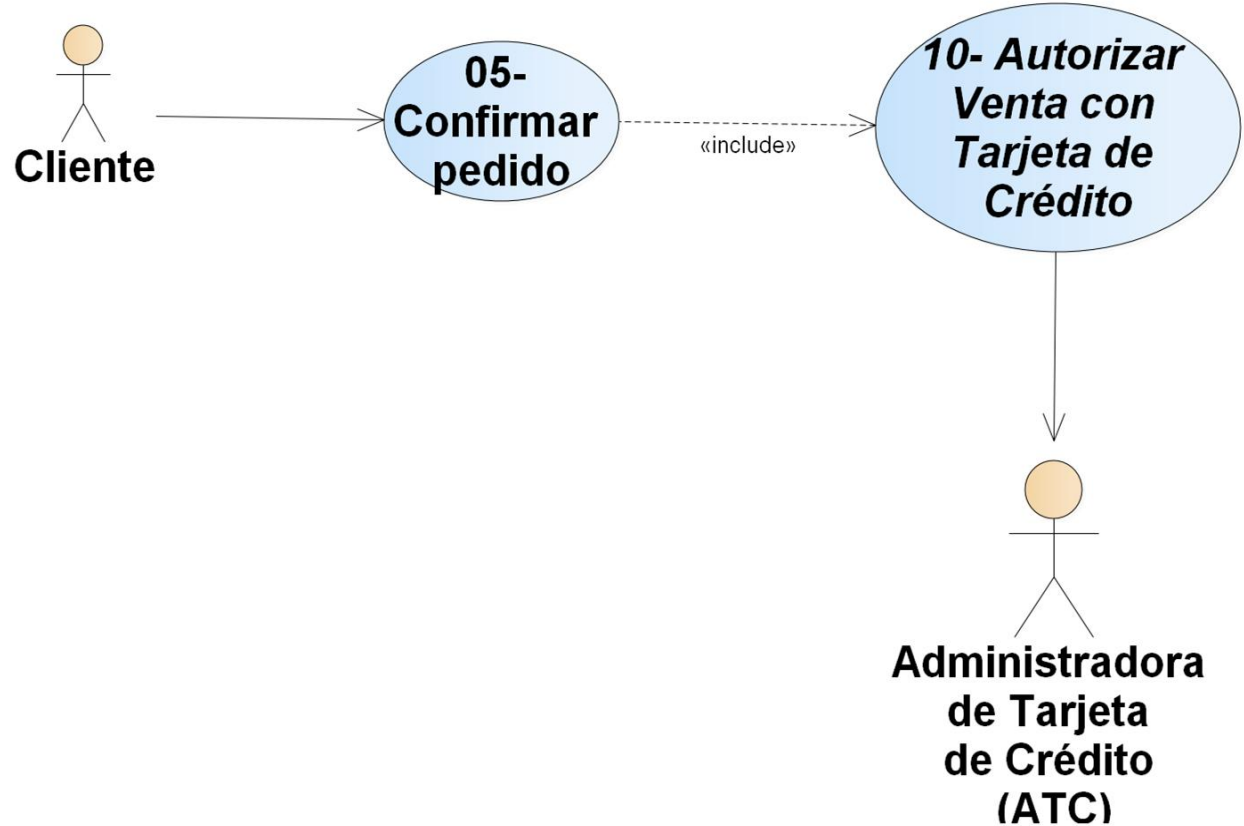
Implementar la integración con los sistemas de autorizaciones de pago propio de cada marca de tarjeta de crédito. Esta integración implica gestionar los mensajes de requerimientos de solicitud financiera (venta) y de reversa de operación (anulación de venta) respetando el estándar de transacciones financiera ISO 8583.

Aplicación Venta Digital

¿Y si miramos la arquitectura?

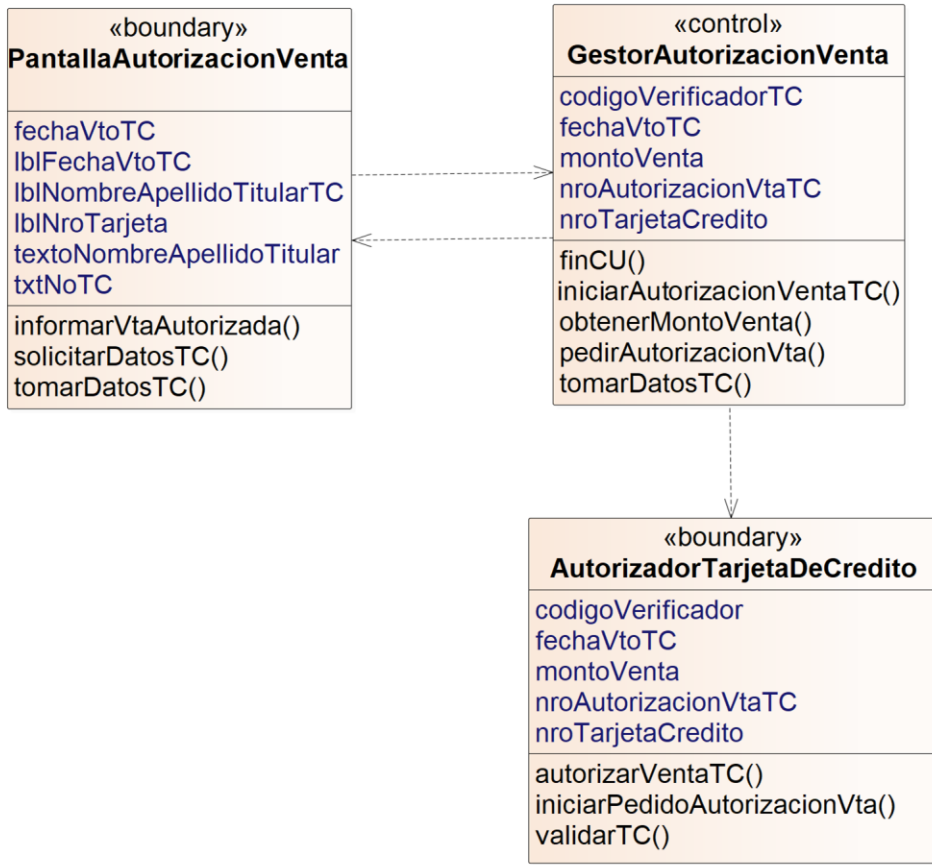


Vista parcial de Casos de uso para la Compra de artículos



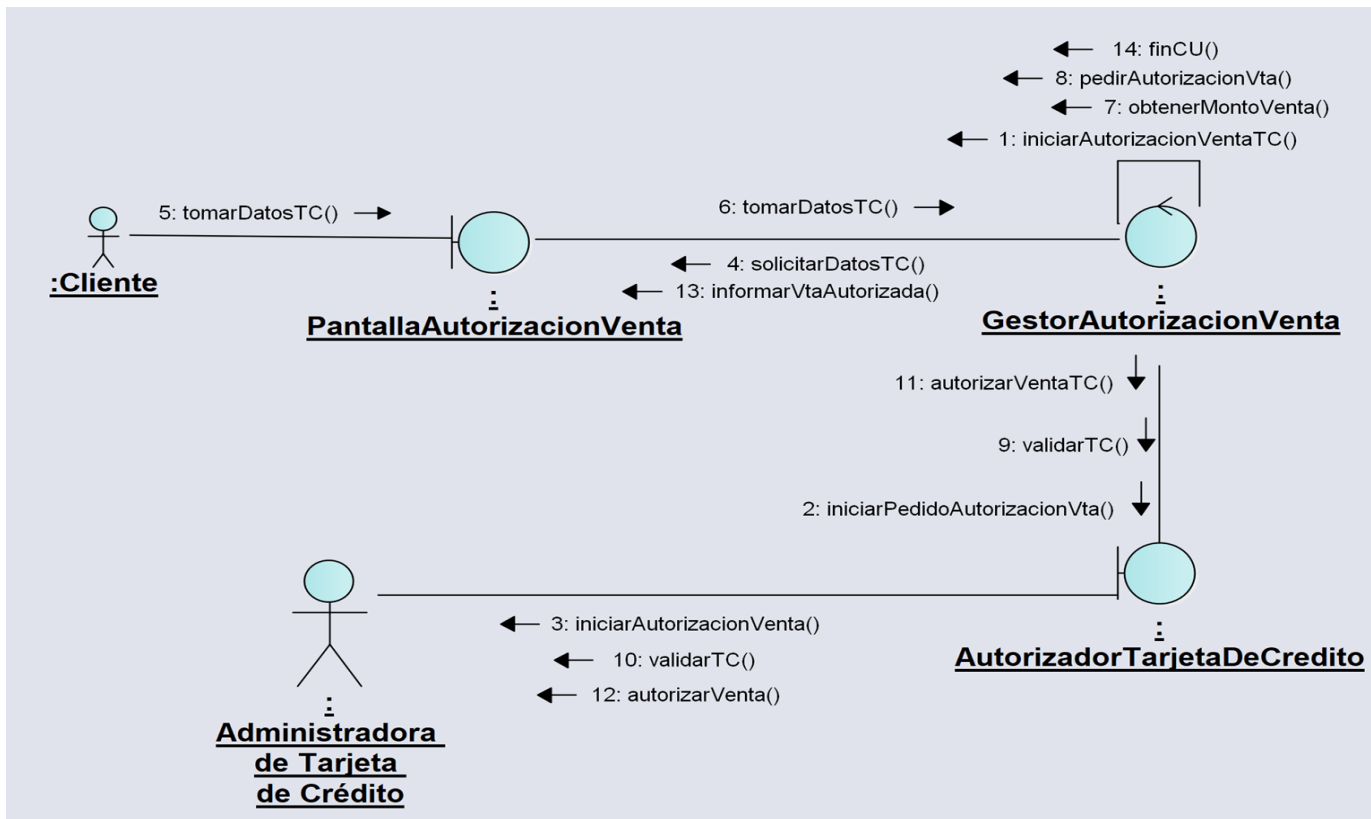
Aplicación Venta Digital

Estructura Análisis – CU Autorizar Venta con Tarjeta de Crédito



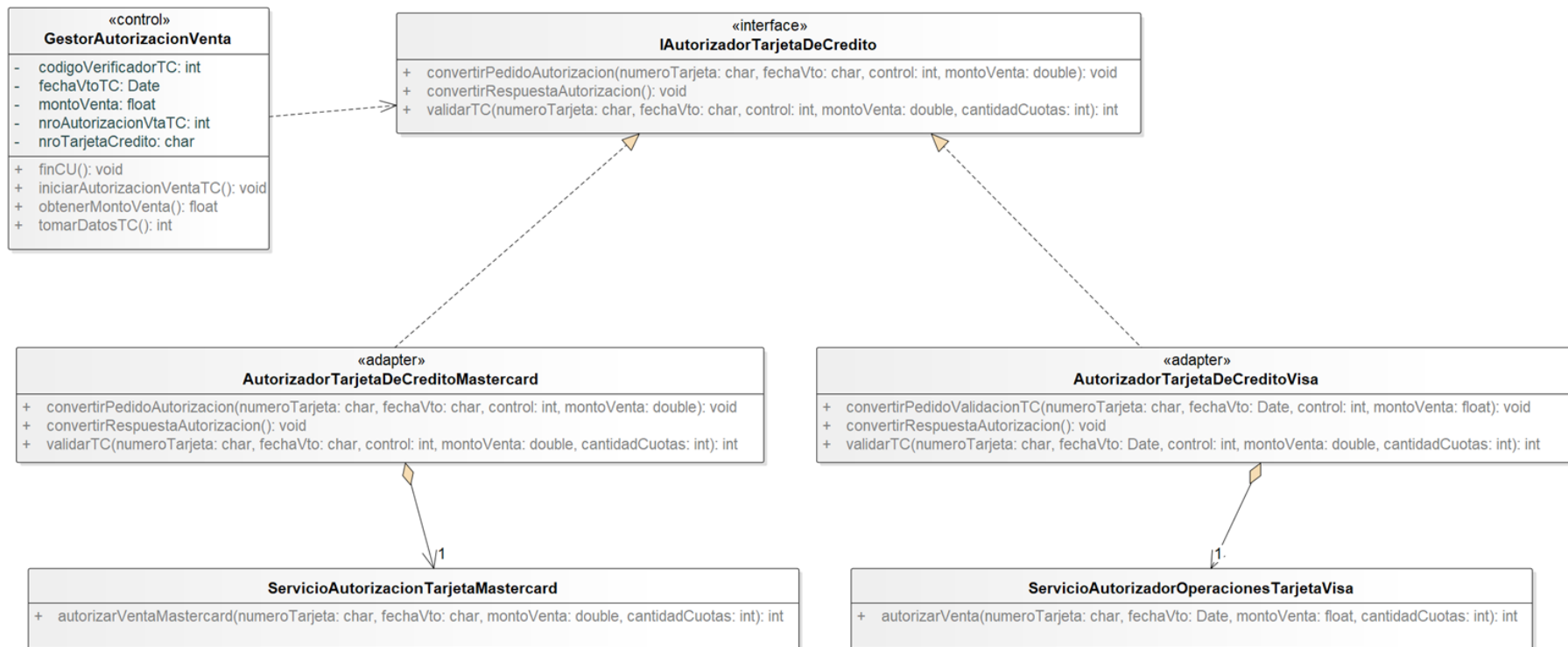
Aplicación Venta Digital

Dinámica Análisis – CU Autorizar Venta con Tarjeta de Crédito



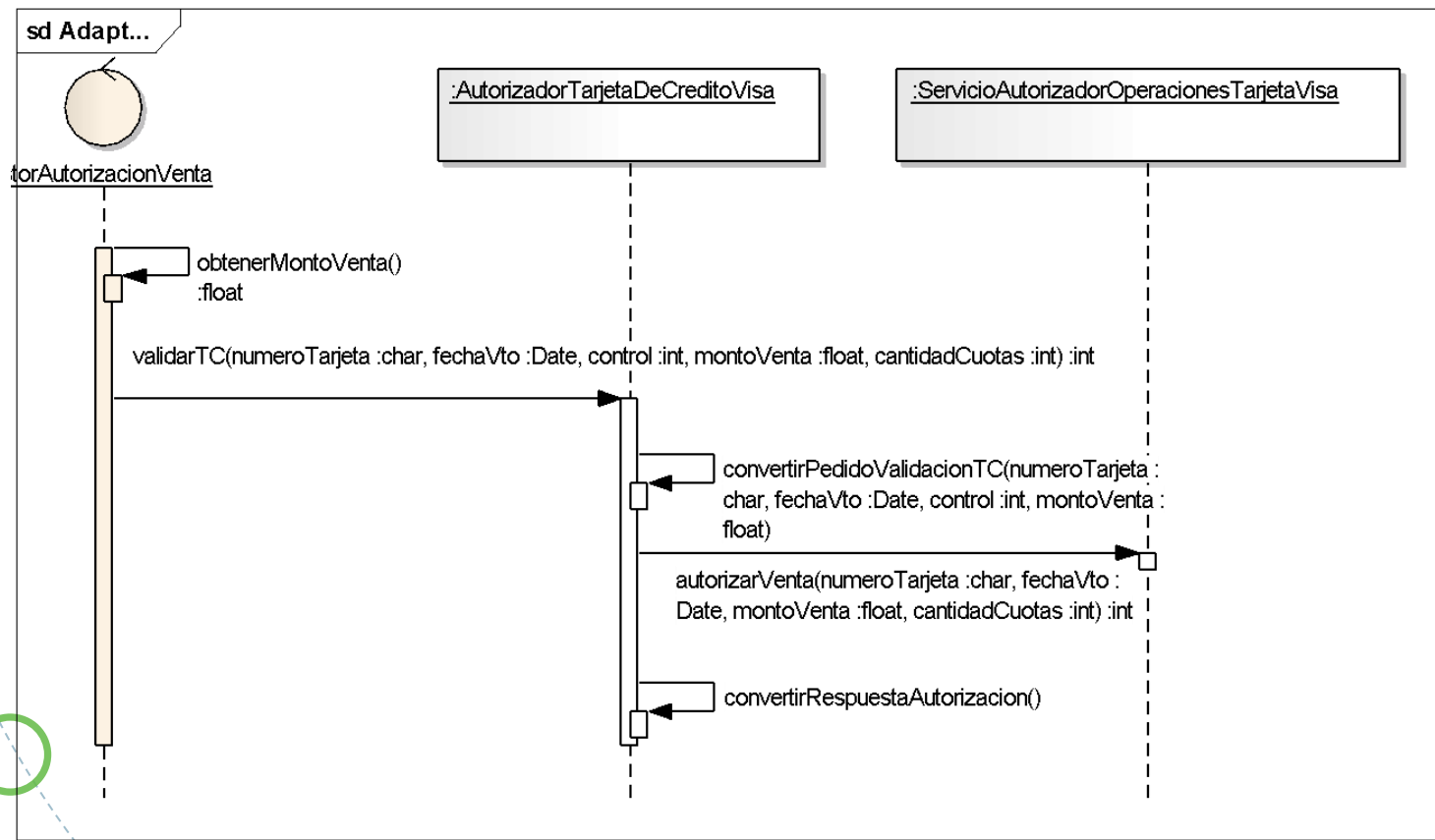
Aplicación Venta Digital

Estructura Diseño – CU Autorizar Venta con Tarjeta de Crédito



Aplicación Venta Digital

Dinámica Diseño – CU Autorizar Venta con Tarjeta de Crédito



Aplicación Venta Digital

Participantes

- **Cliente** – clase que contiene lógica de negocio de la aplicación. En el ejemplo es el **GestorAutorizaciónVenta**.
- **Objetivo** – Interfaz de la clase cliente que deben utilizar otras clases para colaborar con Cliente. En el ejemplo es **IAutorizadorTarjetaDeCrédito**
- **Adaptable** – una clase (normalmente de terceros) que no puede ser utilizada por Cliente por tener una interfaz incompatible. En el ejemplo es la clase, la API o el servicio provisto por la Administradora de la Tarjeta de Crédito (**ServicioAutorizadorOperacionesTarjetaVisa**)
- **Adapter** – implementa la interfaz del Cliente “envolviendo” el Adaptable. Traduce la llamada del cliente de manera que el Adaptable la pueda entender. En el ejemplo es **AutorizadorTarjetaDeCreditoVisa**.

Ventajas – Adaptador de Objeto

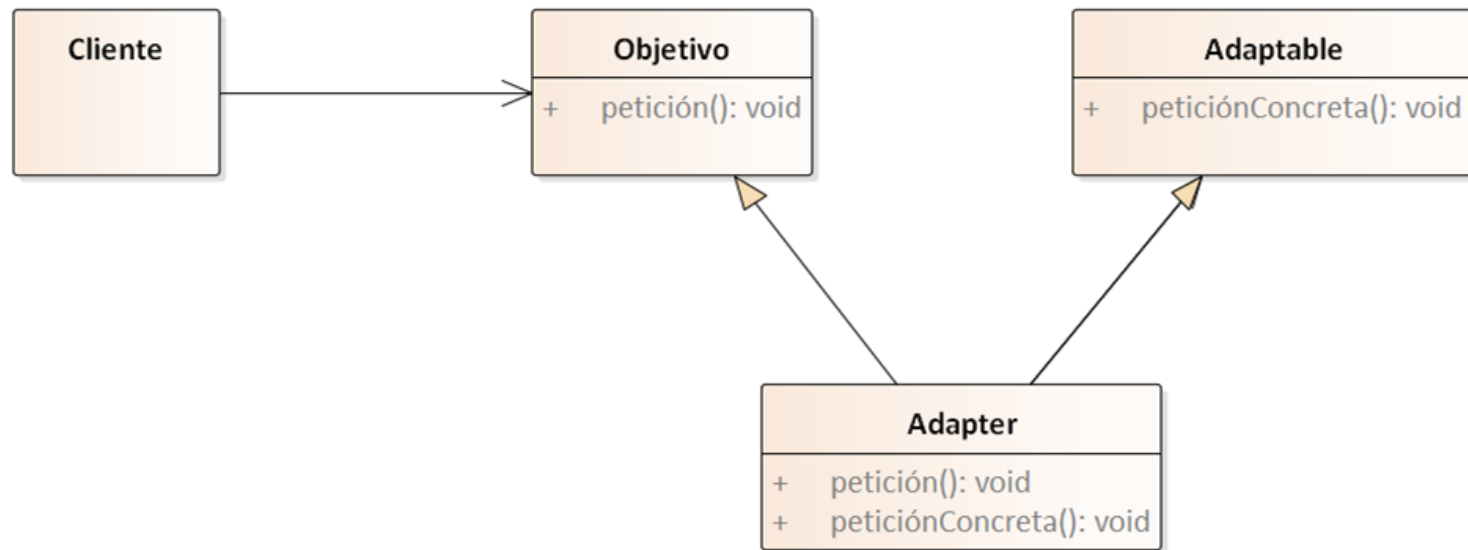
- ❖ Principio de responsabilidad única . Puede separar la interfaz o el código de conversión de datos de la lógica comercial principal del programa.
- ❖ Principio abierto / cerrado . Puede introducir nuevos tipos de adaptadores en el programa sin romper el código del cliente existente, siempre que funcionen con los adaptadores a través de la interfaz del cliente.

Desventajas

- © La complejidad general del código aumenta porque necesita introducir un conjunto de nuevas interfaces y clases. A veces es más simple cambiar la clase de servicio para que coincida con el resto de su código.

Estructura – Adaptador de Clase

Esta implementación utiliza la herencia: el adaptador hereda las interfaces de ambos objetos al mismo tiempo. Tenga en cuenta que este enfoque solo se puede implementar en lenguajes de programación que admitan herencia múltiple.



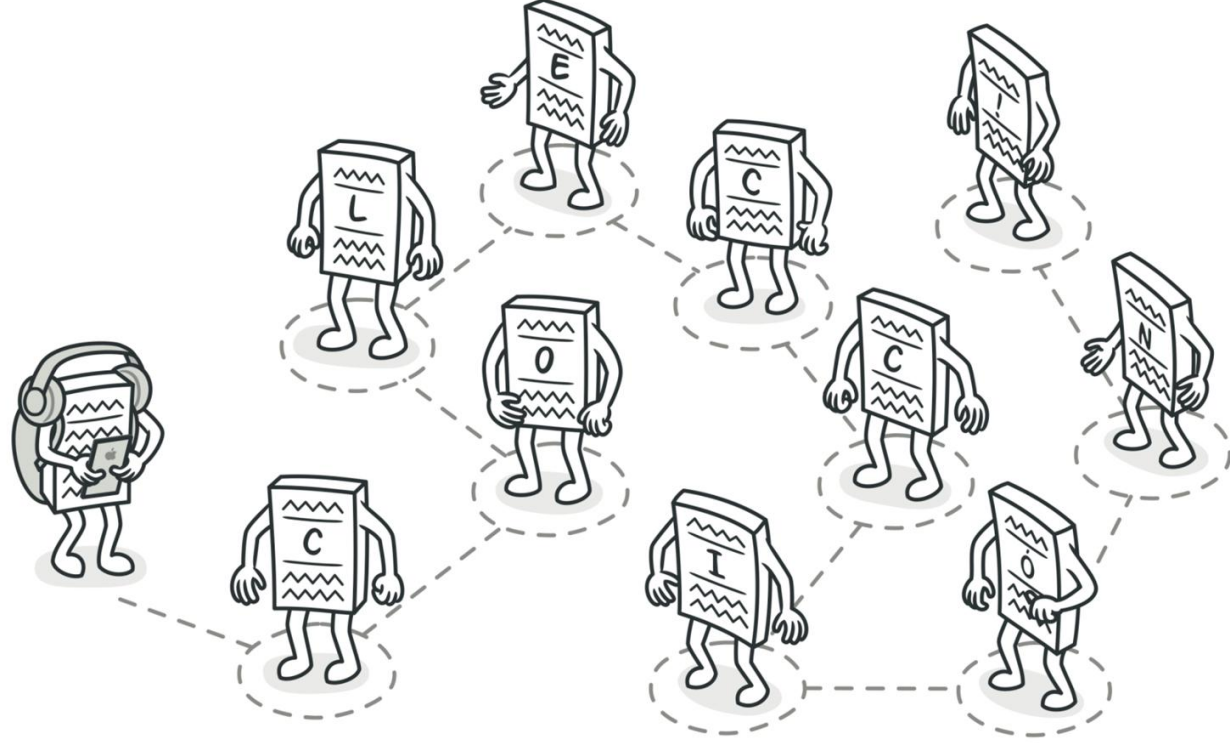
Participantes – Adaptador de Clase

- **Cliente** – clase que contiene lógica de negocio de la aplicación.
- **Objetivo** – define las operaciones que requiere el cliente para interactuar con otras clases.
- **Adaptable** – una clase (normalmente de terceros) que no puede ser utilizada por Cliente por tener una interfaz incompatible.
- **Adapter** – redefine los comportamientos de Objetivo y de Adaptable de manera que se puedan entender.

Desventajas – Herencia

- ❖ Esta implementación sólo es posible con lenguajes que soporten herencia múltiple.
- ❖ La redefinición en Adapter sólo puede hacerse para un Adaptable concreto.

Iterador (Iterator)



Iterator

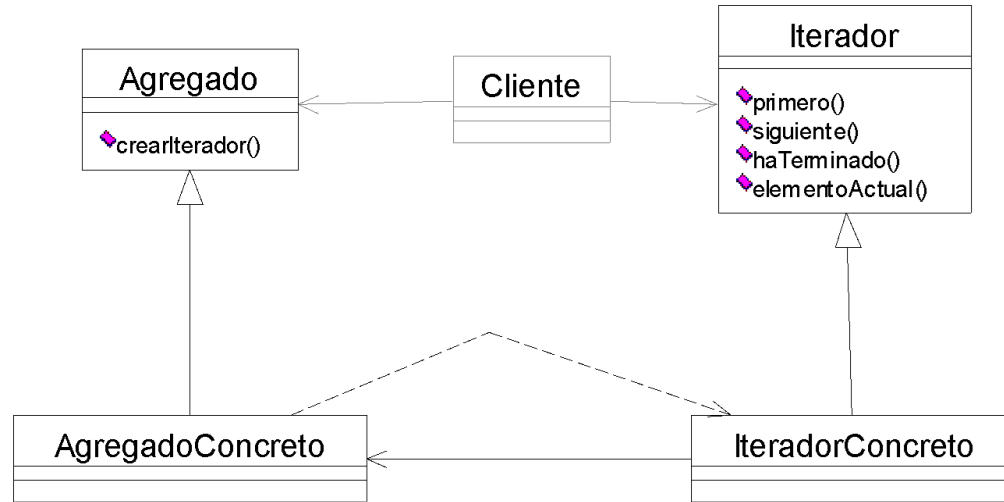
◎ Consecuencias

- ◎ Simplifica la interfaz de un contenedor al extraer los métodos de recorrido
- ◎ Permite varios recorridos, concurrentes
- ◎ Soporta variantes en las técnicas de recorrido

- Patrones de Creación
- Patrones de Estructura
- Patrones de Comportamiento

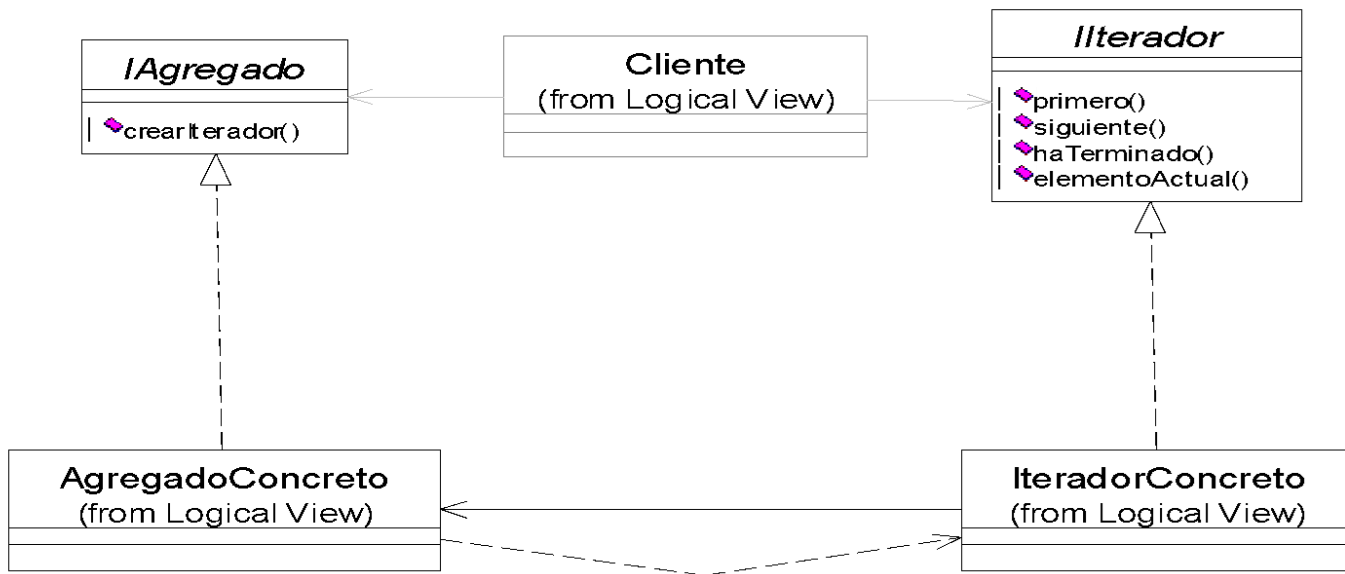
Iterador Externo Polimórfico

Herencia



Iterador Externo Polimórfico

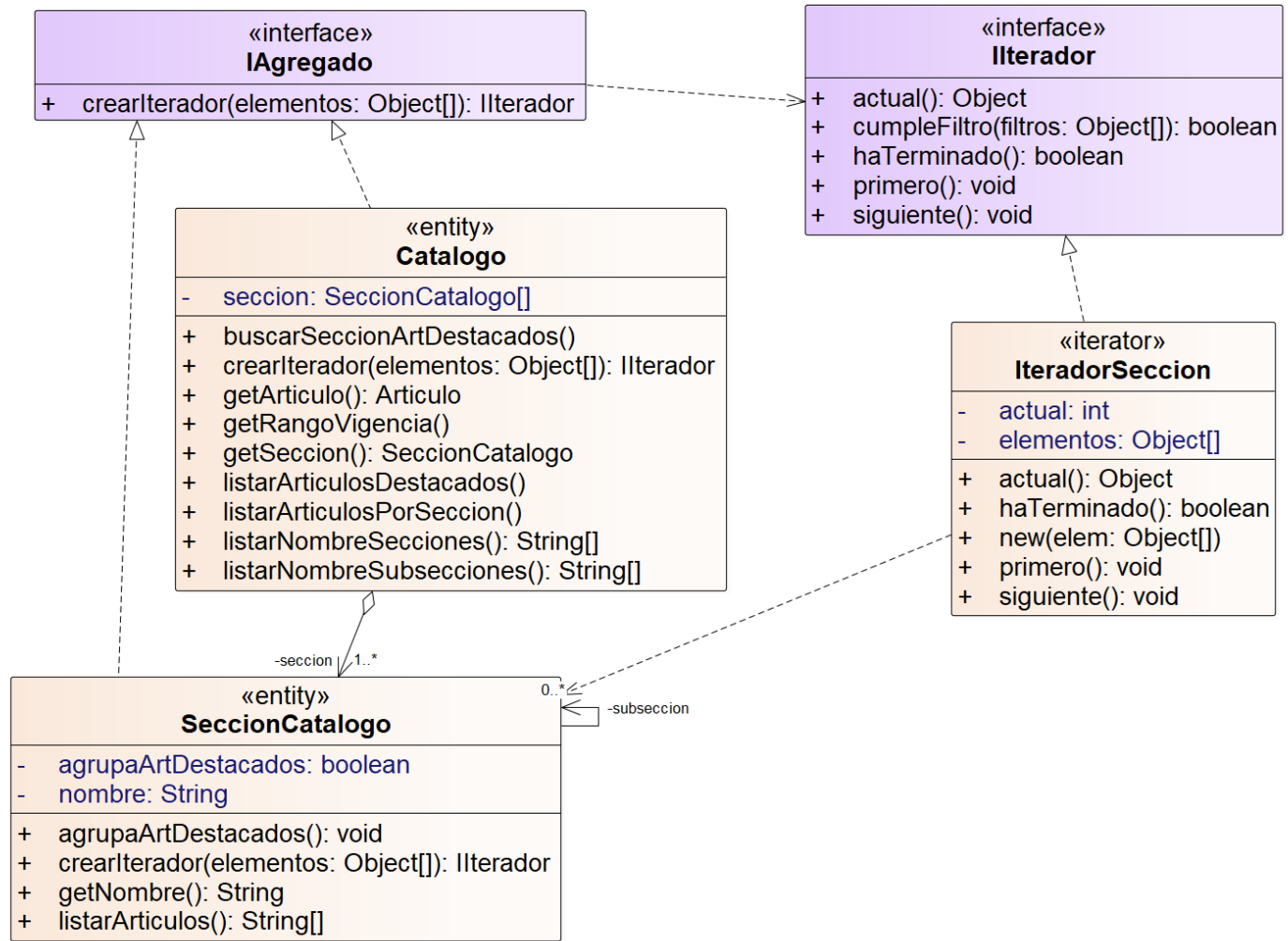
Realización



Iterator: Aplicación

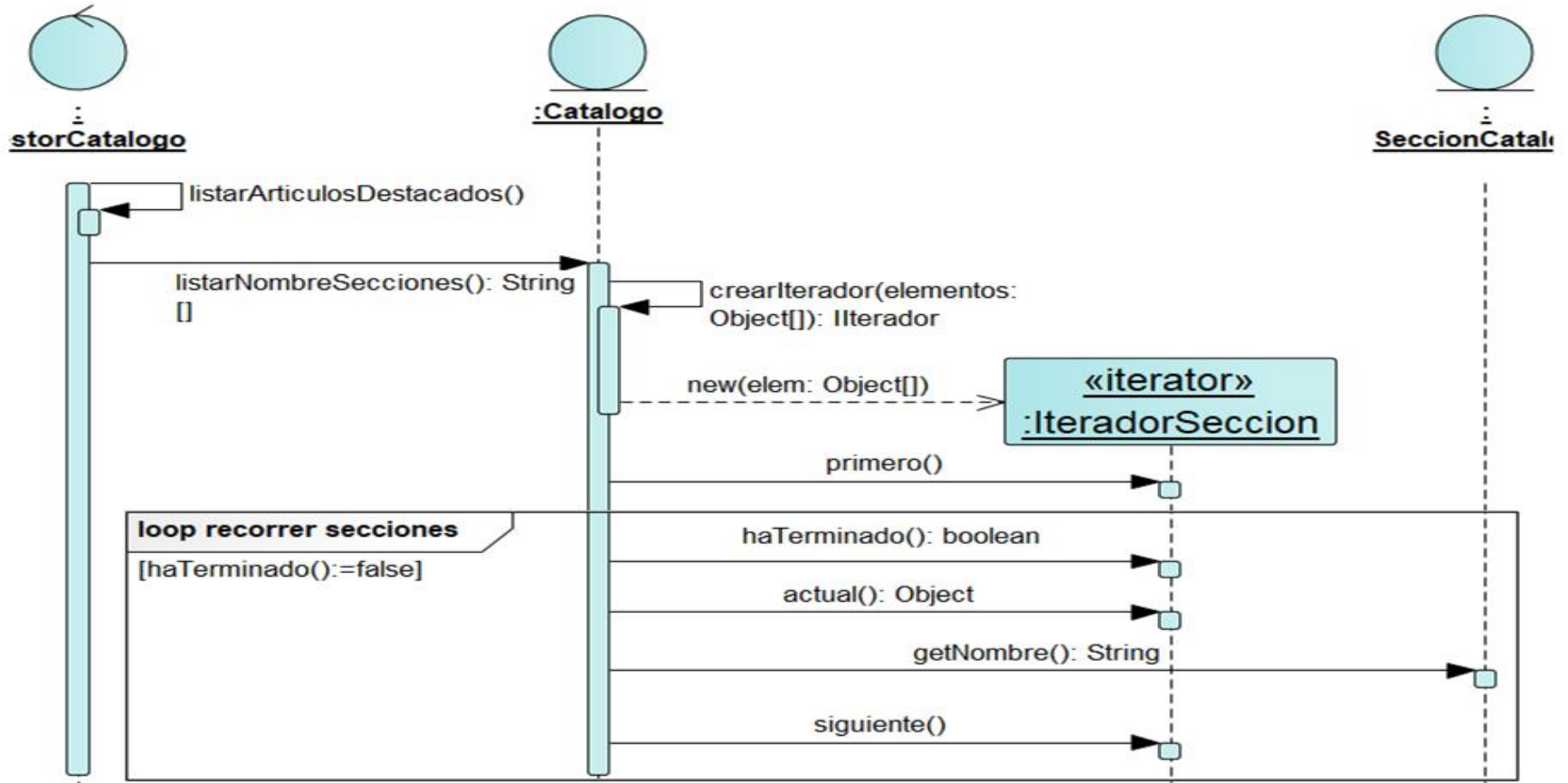
Estructura

class Iterator-Seccion



Dinámica:

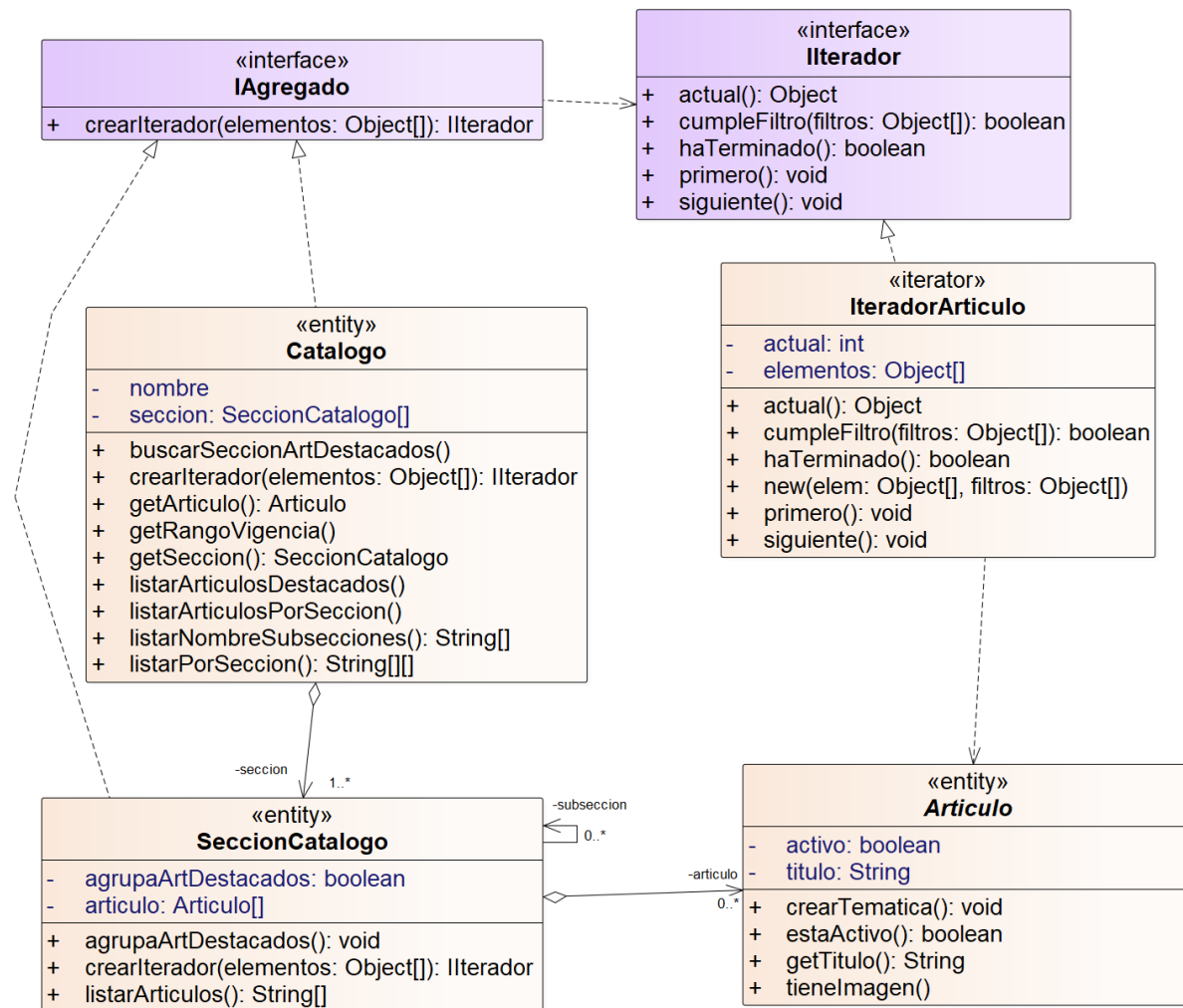
Iterator: Aplicación



Iterator: Aplicación

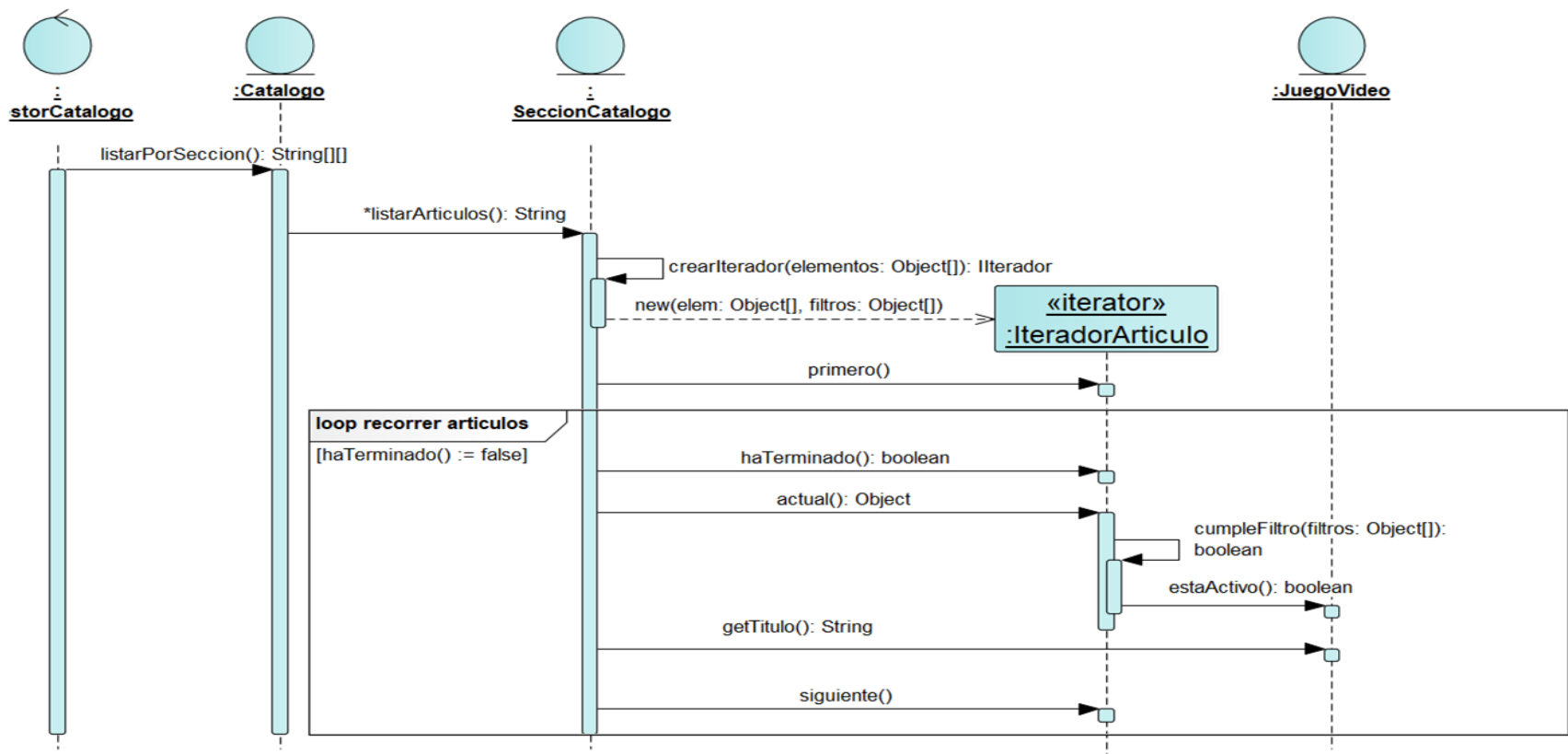
Estructura

class Iterator-Articulo



Dinámica:

Iterator: Aplicación



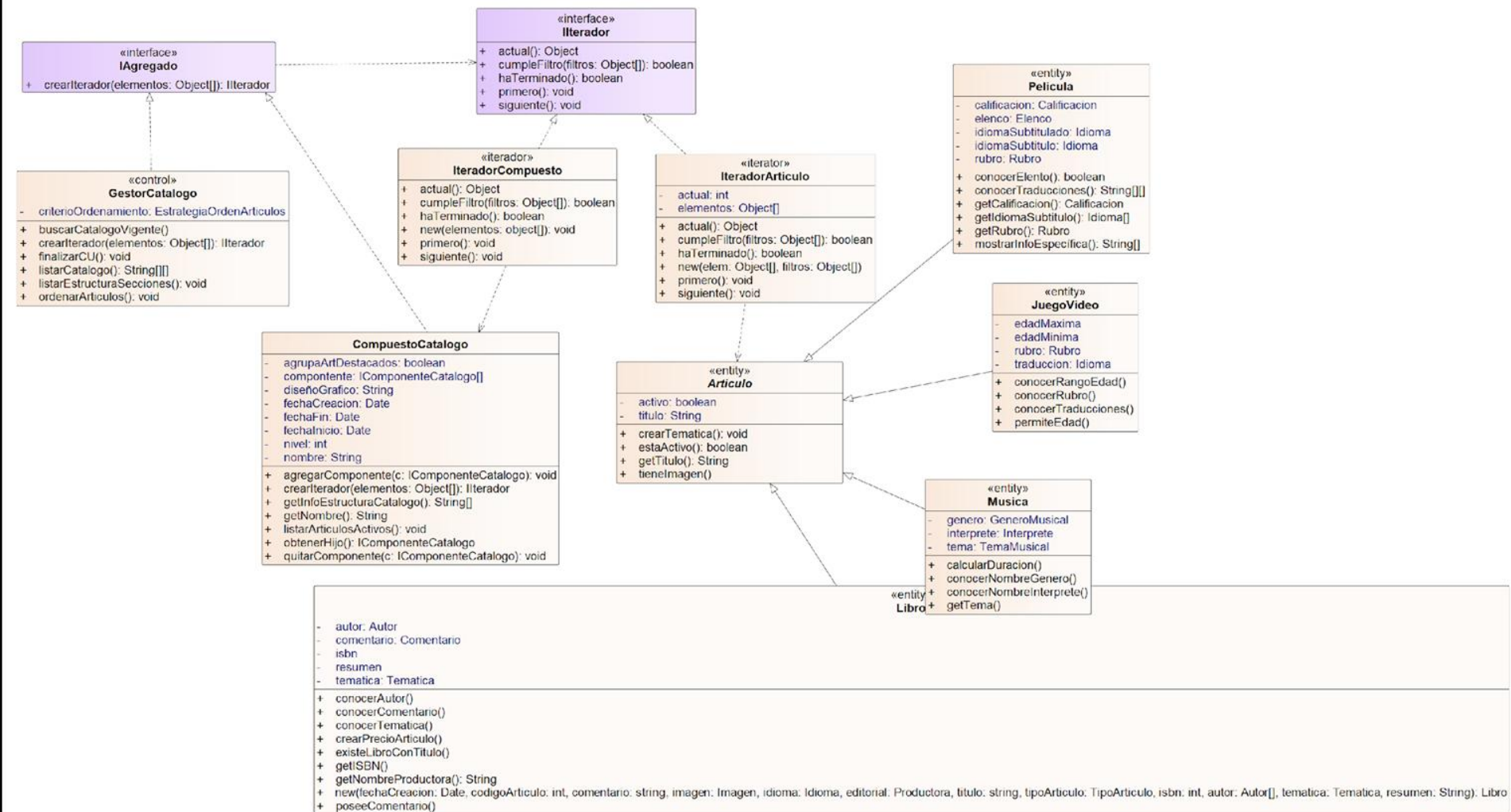
Iterator: Aplicación

◎ Participantes

- ◎ Iterador [**IIterador**]: interfaz que permite recorrer los elementos y acceder a ellos.
- ◎ IteradorConcreto [**IteradorArticulos**, **IteradorSecciones**]: implementa la interfaz Iterador y mantiene la posición actual en el recorrido del Agregado.
- ◎ Agregado [**IAgregado**]: define una interfaz para crear un objeto Iterador
- ◎ AgregadoConcreto [**Catálogo**, **SeccionCatalogo**]: implementa la interfaz de creación de Iterador para devolver una instancia de un **Iterador** concreto (**IteradorArticulos** o **IteradorSecciones**).

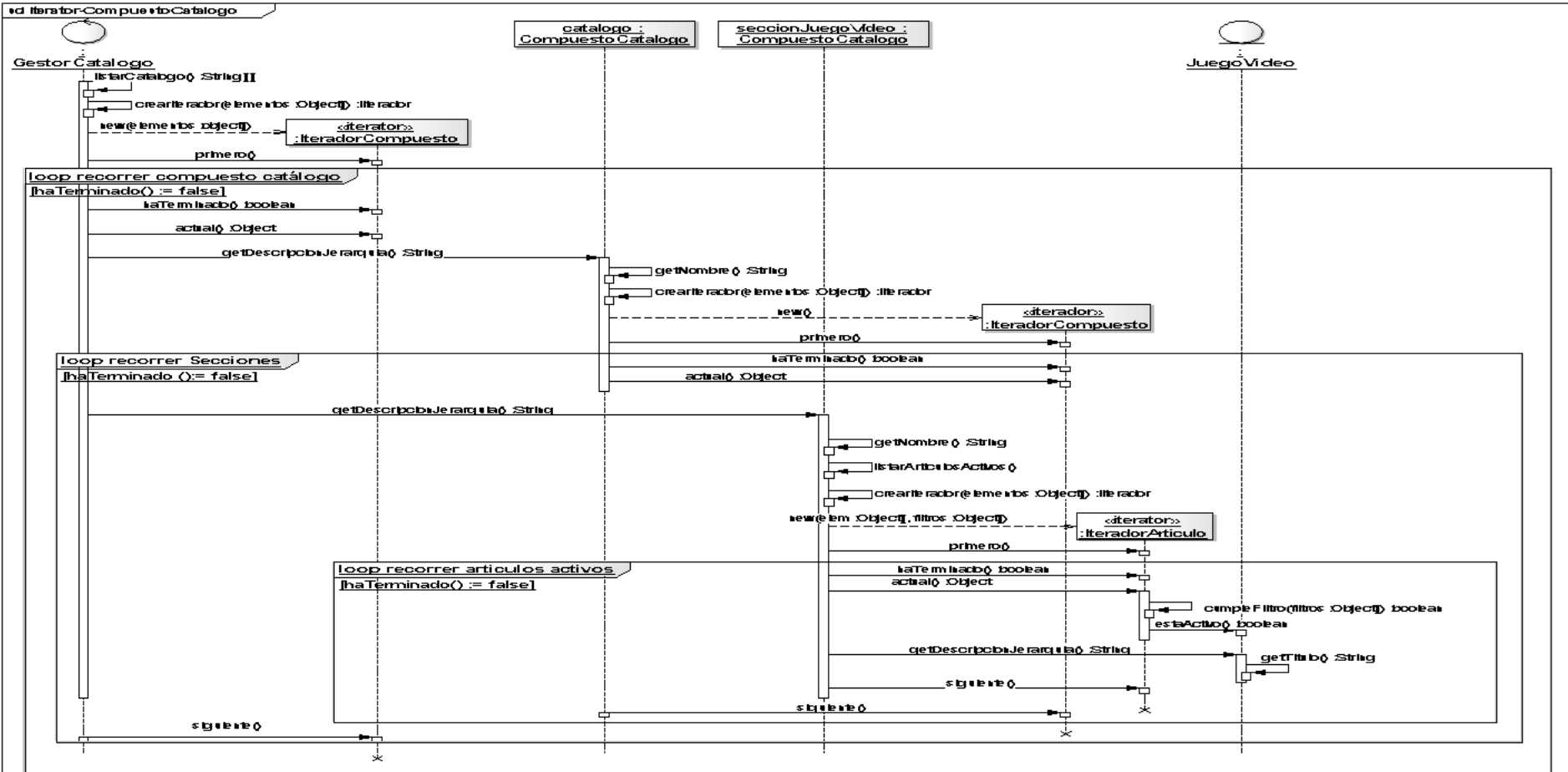
◎ Procedimiento

- ◎ Cuando necesitamos recorrer o mostrar los artículos (libros, películas, música o juegos) o secciones del catálogo se invoca a los métodos **listarArticulosActivos()** y **listarNombreSecciones()**, respectivamente.
- ◎ En la dinámica se mostrará el ejemplo con JuegoVideo.
- ◎ Este método invoca a **crearIterador()** para establecer el Iterador.
- ◎ Finalmente se definen los métodos necesarios para realizar el recorrido de los elementos: **haTerminado()**, **siguiente()**, **cumpleFiltro()**, etc.



Iterator: Aplicación sobre el composite

Dinámica:



Iterator: Aplicación conjunta con Composite

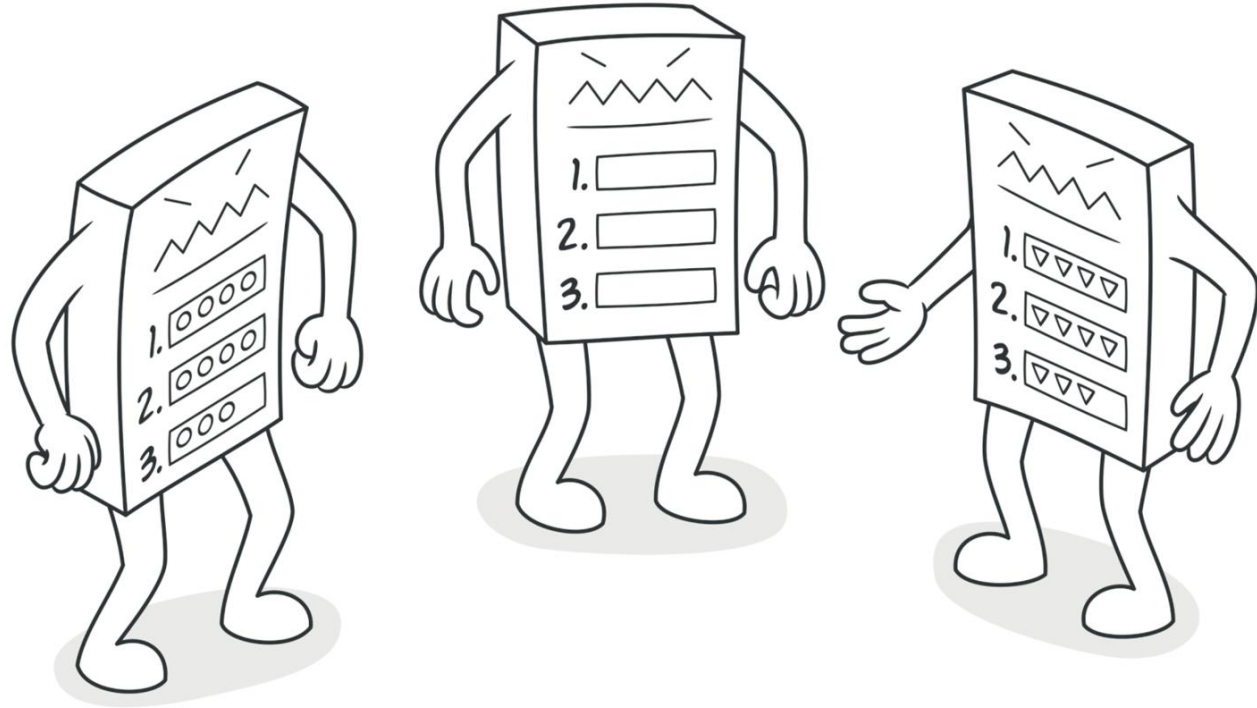
⊙ Participantes

- ⊙ Iterador [**IIterador**]: interfaz que permite recorrer los elementos y acceder a ellos.
- ⊙ IteradorConcreto [**IteradorArticulos**, **IteradorCompuesto**]: implementa la interfaz Iterador y mantiene la posición actual en el recorrido del Agregado.
- ⊙ Agregado [**IAgregado**]: define una interfaz para crear un objeto Iterador
- ⊙ AgregadoConcreto [**GestorCatálogo**, **CompuestoCatalogo**]: implementa la interfaz de creación de Iterador para devolver una instancia de un **Iterador** concreto (**IteradorArticulos** o **IteradorCompuesto**).

⊙ Procedimiento

- ⊙ Cuando necesitamos recorrer o mostrar los artículos (libros, películas, música o juegos) de un catálogo se invoca a los métodos **listarCatalogo()** y **listarArticulosActivos()**, respectivamente.
- ⊙ En la dinámica se mostrará el ejemplo con JuegoVideo.
- ⊙ Este método invoca a **crearIterador()** para establecer el Iterador.
- ⊙ Finalmente se definen los métodos necesarios para realizar el recorrido de los elementos: **haTerminado()**, **siguiente()**, **cumpleFiltro()**, etc.

Template Method



Template Method

◎ Propósito

- Define el esqueleto (esquema, patrón) de un algoritmo en una operación, difiriendo algunos pasos a las subclases. Permite a las subclases redefinir ciertos pasos de un algoritmo sin cambiar la estructura del algoritmo.

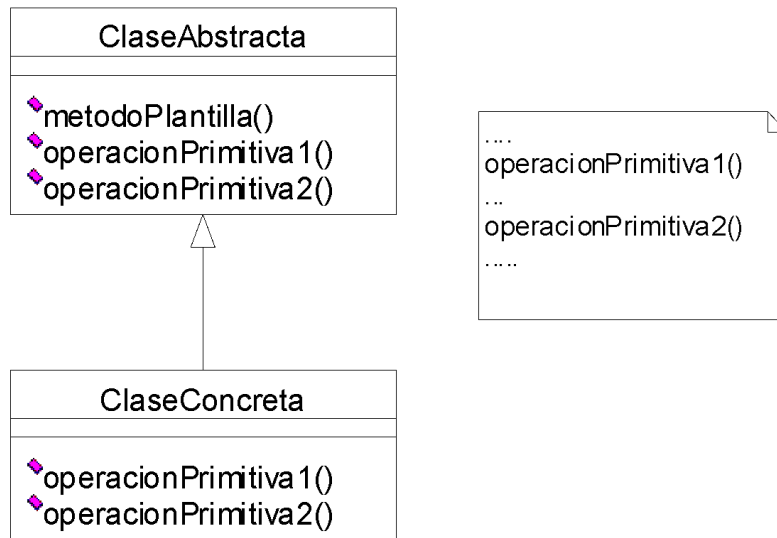
◎ Motivación

- Fundamental para escribir código en un framework.
- Clase *Aplicación* que maneja objetos de la clase *Documento*: método *openDocument*

- Patrones de Creación
- Patrones de Estructura
- Patrones de Comportamiento

Template Method

Estructura



- Patrones de Creación
- Patrones de Estructura
- Patrones de Comportamiento

Template Method

◎ Aplicabilidad

- ◎ Implementar las partes fijas de un algoritmo y dejar que las subclases implementen el comportamiento que puede variar.
- ◎ Cuando el comportamiento común entre varias subclases debe ser factorizado y localizado en una superclase común.
- ◎ Controlar extensiones de las subclases: algoritmos con puntos de extensión

- Patrones de Creación
- Patrones de Estructura
- Patrones de Comportamiento

Template Method

© Consecuencias

- Técnica fundamental para la reutilización: factorizar comportamiento común en librerías de clases
- Estructura de control invertida conocida como “Principio de Hollywood”: “No nos llames, nosotros te llamaremos”.
- Un método plantilla (template) invoca a los siguientes tipos de métodos:
 - operaciones concretas en la subclase concreta o en clientes
 - operaciones concretas en la clase abstracta
 - operaciones abstractas (primitivas)
 - métodos factoría (uso combinado con el Factory Method)
 - métodos hook que proporcionan comportamiento por defecto que se implementa en la clase abstracta y que las subclases pueden utilizar o ignorar en función de lo que necesiten. También puede estar vacío.

- Patrones de Creación
- Patrones de Estructura
- Patrones de Comportamiento

Template Method

◎ Implementación

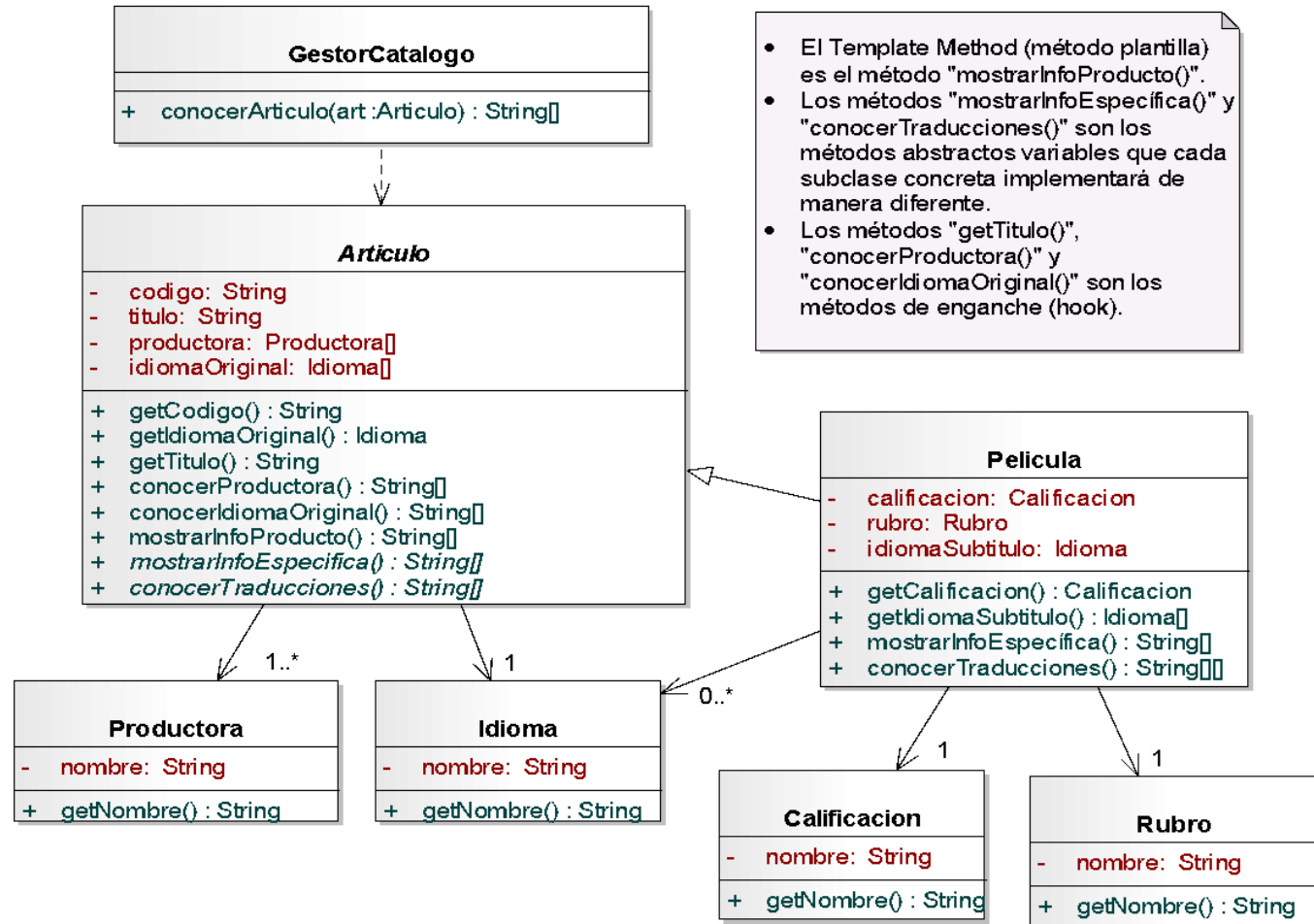
- ◎ Minimizar el número de métodos que es necesario redefinir en las subclases.
- ◎ Nombrar a los métodos que se deben redefinir añadiéndole cierto prefijo, p.e. “do”.
- ◎ En C++, métodos a redefinir son protegidos y virtuales, el método plantilla no será virtual.

- Patrones de Creación
- Patrones de Estructura
- Patrones de Comportamiento

Template Method: Aplicación

Estructura

class Template Method



- El Template Method (método plantilla) es el método "mostrarInfoProducto()".
- Los métodos "mostrarInfoEspecifica()" y "conocerTraducciones()" son los métodos abstractos variables que cada subclase concreta implementará de manera diferente.
- Los métodos "getTitulo()", "conocerProductora()" y "conocerIdiomaOriginal()" son los métodos de enganche (hook).

Template Method: Aplicación

◎ Participantes

◎ Clase abstracta [*Articulo*]:

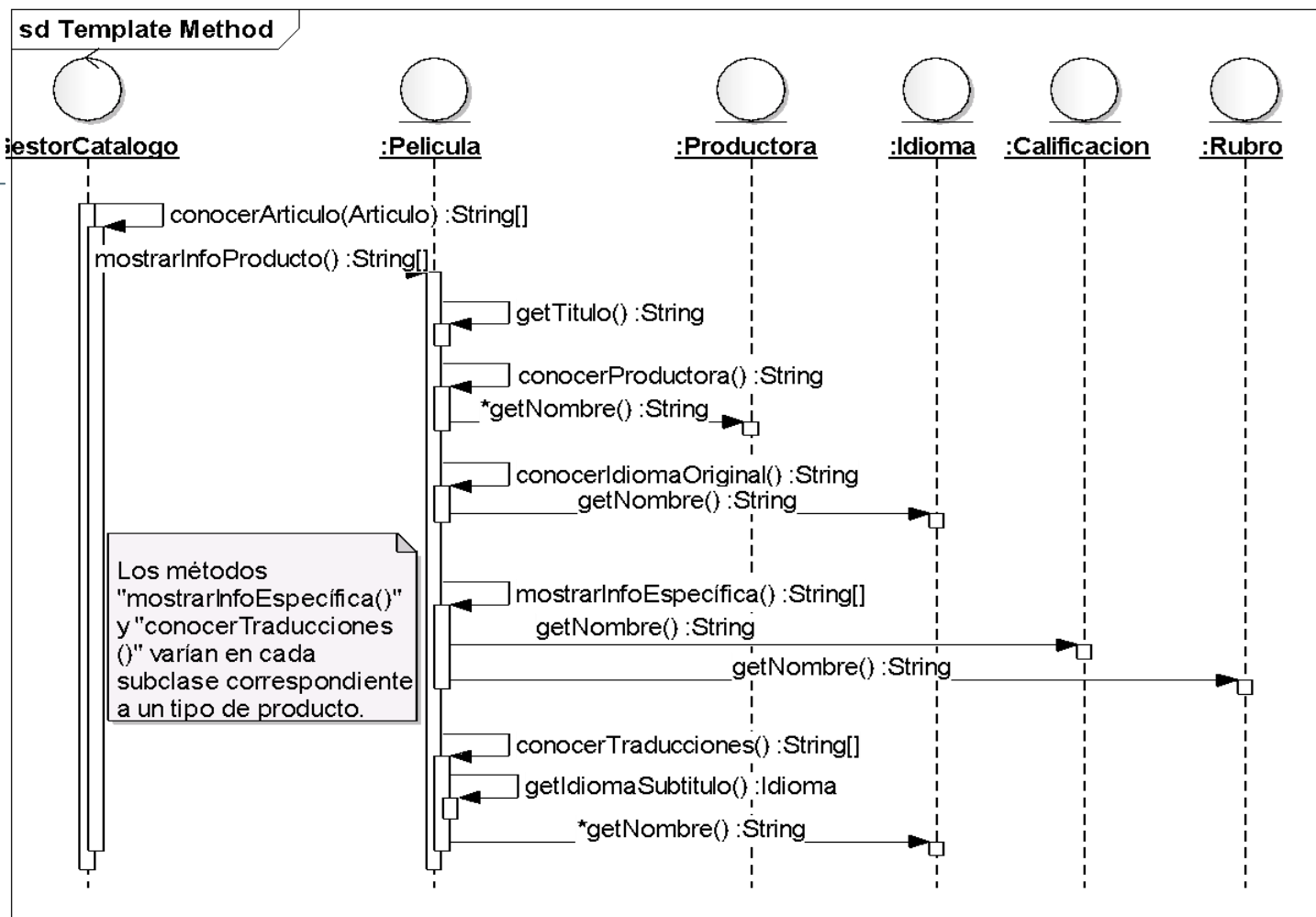
- implementa un método plantilla [**mostrarInfoProducto()**] que define el esqueleto de un algoritmo, que llama a las operaciones primitivas y a las definidas por la clase abstracta o a la de otros objetos
- define las **operaciones primitivas abstractas** que son definidas por las subclasses para implementar los pasos de un algoritmo [**conocerTraducciones()**, **mostrarInfoEspecífica()**, etc.]
- Define los **métodos de enganche** con la implementación trivial que usarán las clases concretas si lo deciden: **getTitulo()**, **conocerProductora()**, **conocerIdiomaOriginal()**.

◎ Clases Concretas [*Pelicula, Musica, Libro y Juego*]:

- implementan la **operaciones primitivas** para realizar los pasos del algoritmo específico de las subclasses (**mostrarInfoEspecífica()**, **mostrarTraducciones()**, etc.)

Template Method: Aplicación

Dinámica

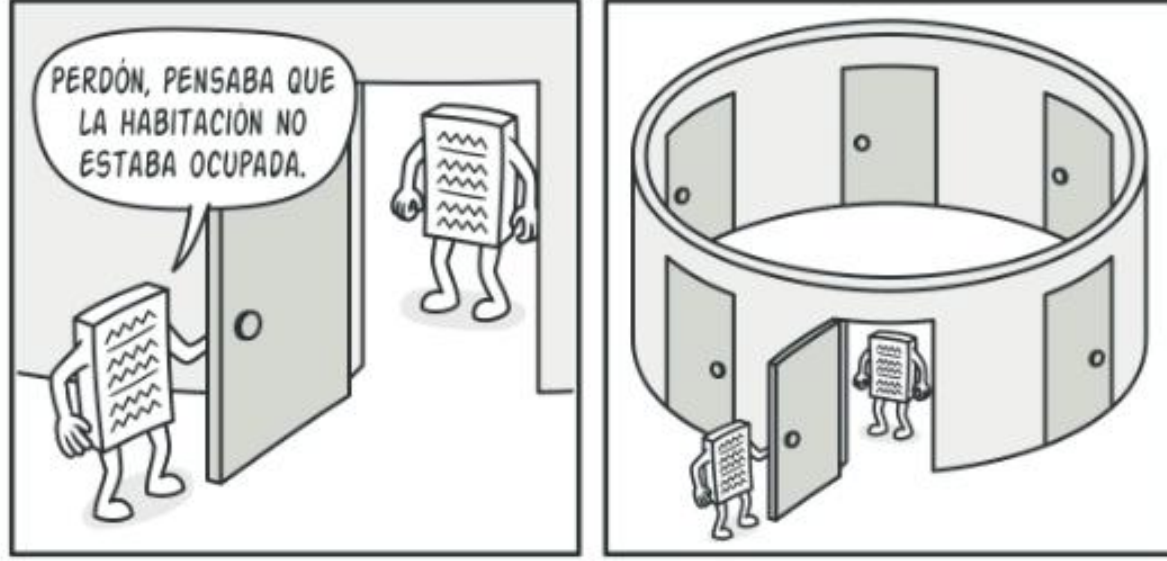


Template Method: Aplicación

⦿ Procedimiento

- ⦿ **Artículo** necesita definir un método que permita mostrar toda la información de cada artículo, de modo de hacerlo lo más independiente posible de las clases que lo heredan
- ⦿ Para eso define el método plantilla **mostrarInfoProducto()** que contiene la estructura de lo que debe hacerse al momento de mostrar la información de un artículo concreto; y define, además las operaciones necesarias para poder “ejecutar” el método. Estas operaciones pueden ser :
 - Concretas de la clase abstracta: **conocerIdiomaOriginal()**...
 - Operaciones primitivas: **mostrarTraducciones()**, **mostrarInfoEspecífica()**...
 - Operaciones enganche: **: getTitulo(), conocerProductora(), conocerIdiomaOriginal()**.
- ⦿ En las subclases se definen las operaciones **concretas** para cada una de ellas, y se sobrescriben las heredadas (**mostrarInfoEspecífica()**, **mostrarTraducciones()**...)

Singleton



Singleton

Propósito

- Asegurar que una clase tiene una única instancia y asegurar un acceso global

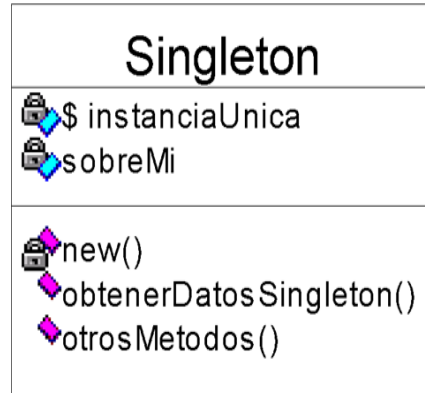
Motivación

- La clase se encarga de asegurar que exista una única instancia y de su acceso.
- Hay muchas circunstancias en las cuales es necesario esto:
 - Caches
 - Cajas de Diálogo
 - Objetos que manejan preferencias y configuraciones de registros.
 - Objetos utilizados para logueo
 - Objetos manejadores de dispositivos como impresoras o tarjetas gráficas.
- Para estos objetos, instanciar más de uno provocaría toda clase de problemas como: funcionamiento incorrecto, sobreuso de recursos, resultados inconsistentes.

- Patrones de Creación
- Patrones de Estructura
- Patrones de Comportamiento

Singleton

Estructura



new:
devuelve
una única
instancia

Singleton

◎ Aplicabilidad

- ◎ Debe existir una única instancia de una clase, accesible globalmente.

◎ Consecuencias

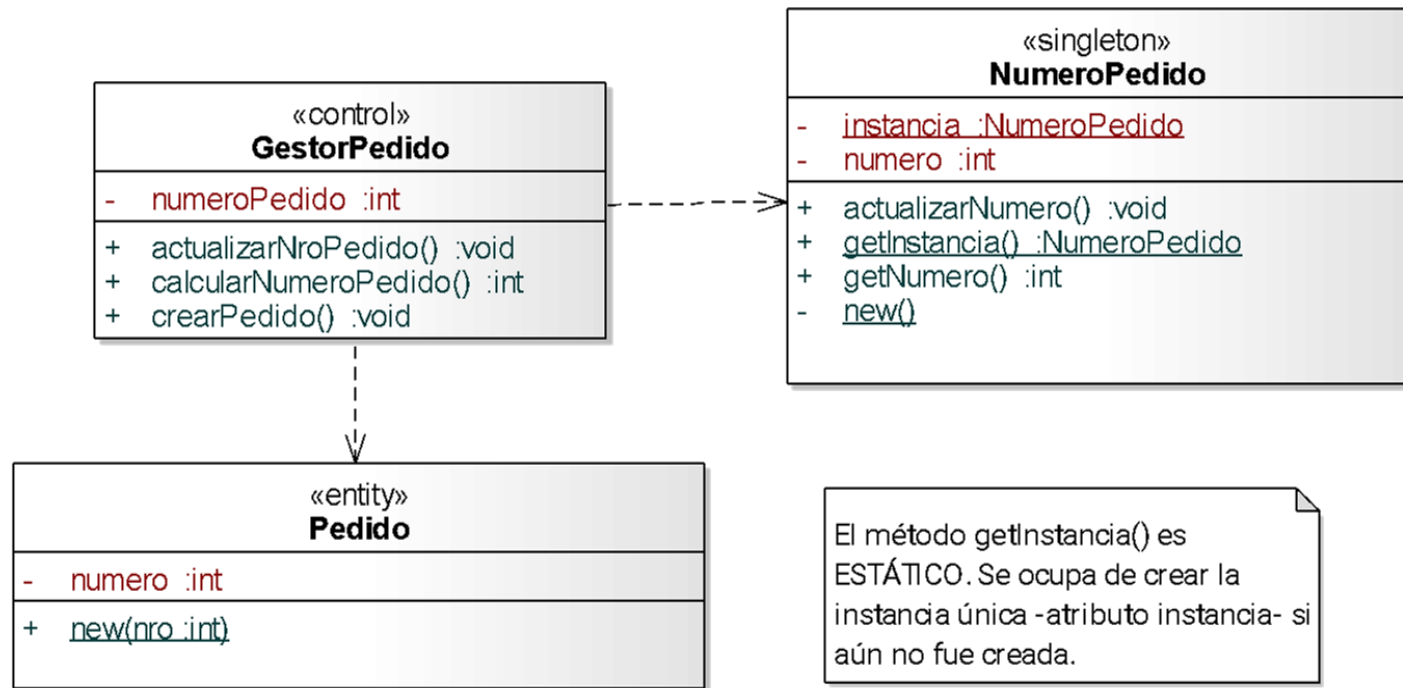
- ◎ Acceso controlado a la única instancia
- ◎ Evita usar variables globales
- ◎ Generalizar a un número variable de instancias
- ◎ Manejar subclases de clase *Singleton*

- Debe validarse que haya una única instancia y que la instancia sea la que queremos

- Patrones de Creación
- Patrones de Estructura
- Patrones de Comportamiento

Singleton: Aplicación

Estructura



Singleton: Aplicación

◎ Participantes

- ◎ Singleton [**NumeroPedido**]: proporciona un constructor privado, mantiene una referencia estática a la única instancia de la clase, y proporciona un método estático de acceso **-getInstancia()-** para devolver una referencia a la misma, permitiendo que los clientes accedan a la única instancia de la clase.

◎ Procedimiento

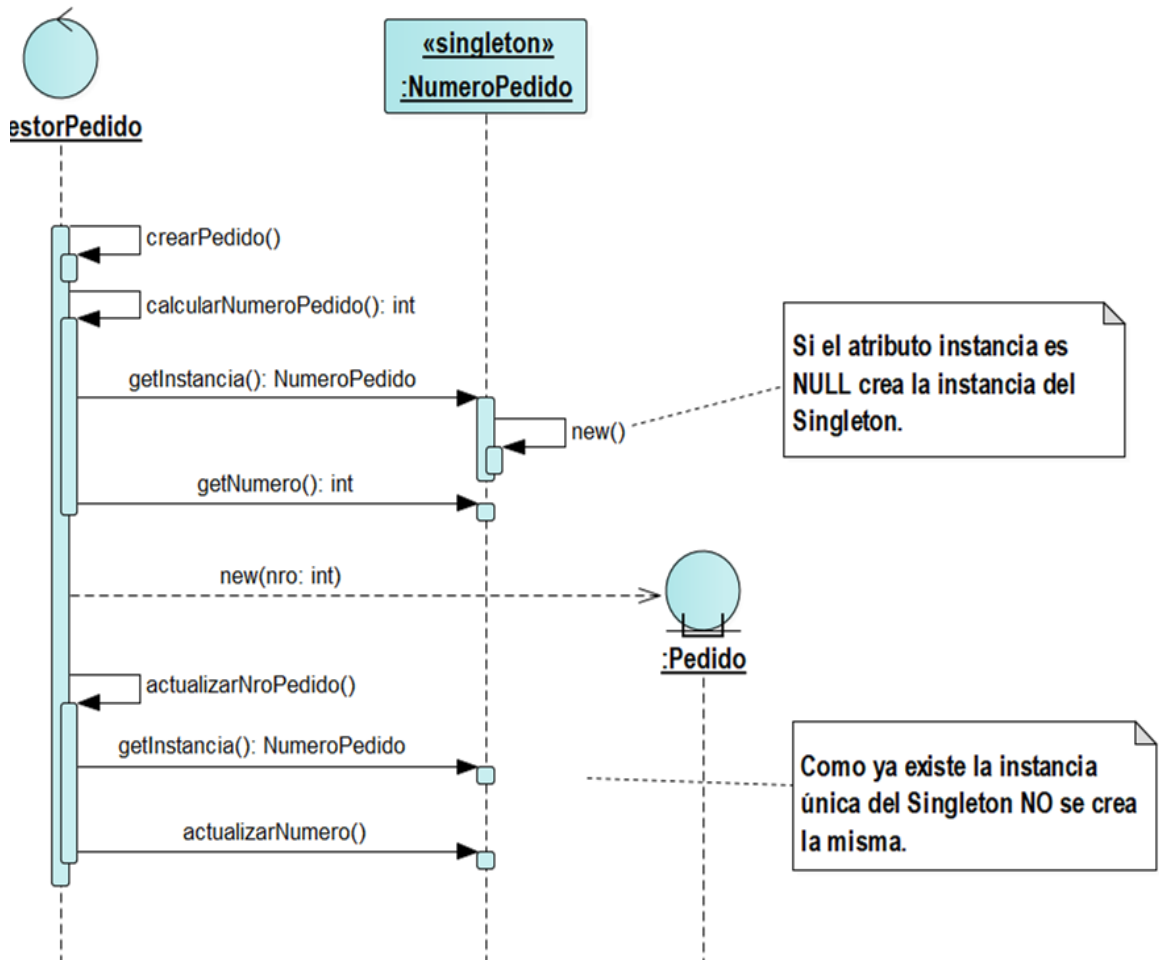
- ◎ Cuando necesitemos crear un nuevo pedido y por lo tanto asignarle su correspondiente número, el gestor de pedido le pediría al **Singleton (NumeroPedido)**, el número de pedido que debe asignar al nuevo pedido.
- ◎ El Singleton tiene un atributo estático para mantener la referencia a la instancia única y uno o más atributos que guardan el estado relevante del objeto -el último número asignado. Crea la instancia única y le devuelve al Gestor el número.
- ◎ El gestor de pedido invoca el método **new()** del pedido pasándole como parámetro entre otras cosas el número de pedido que corresponde y luego le pide al Singleton que actualice el valor del atributo estático.

Singleton: Aplicación

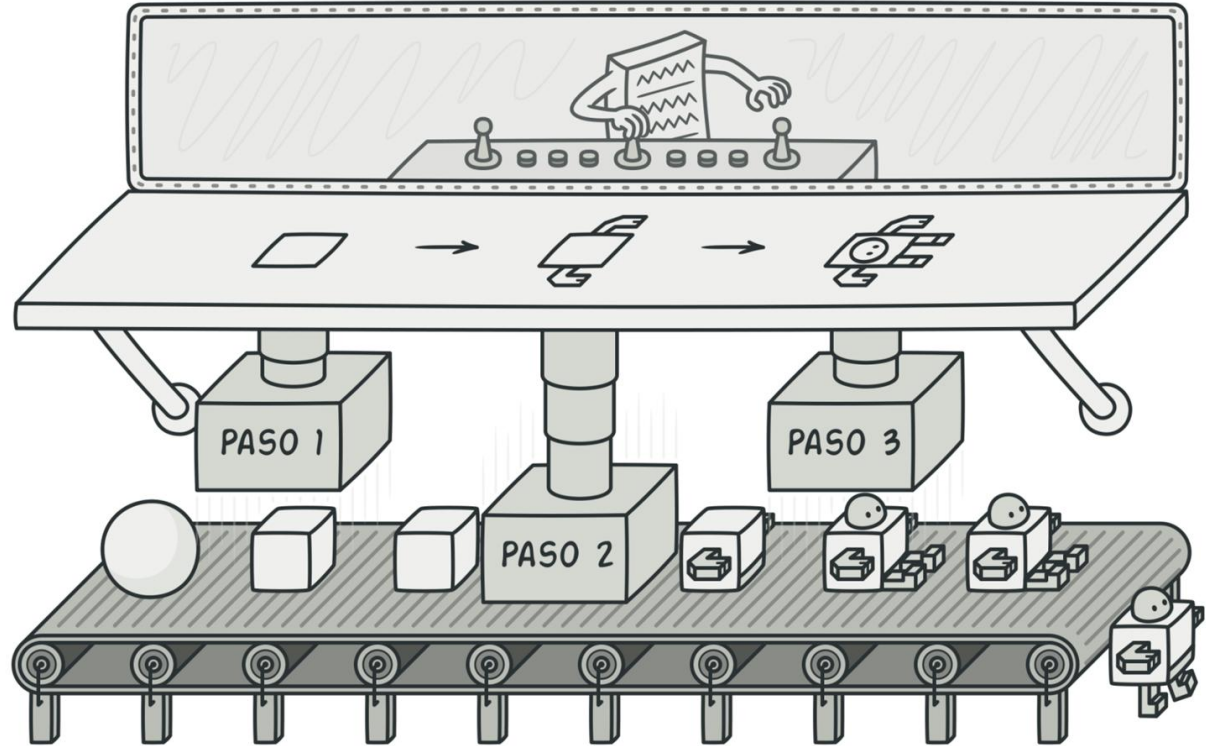
⦿ Dinámica

⦿ Ventaja

Permite un acceso controlado a la única instancia ya que posee un control estricto sobre cómo y cuándo acceden a ella.



Builder



Builder (Constructor)

◎ Propósito

- ◎ Separa la construcción de un objeto complejo de su representación, así que el mismo proceso de construcción puede crear diferentes representaciones.

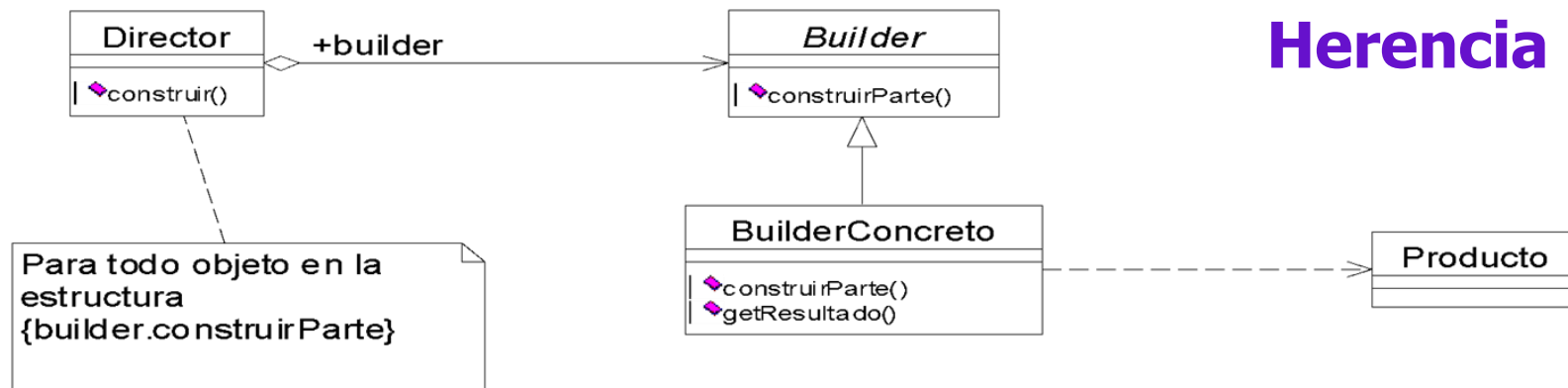
◎ Motivación

- ◎ Un traductor de documentos RTF a otros formatos. ¿Es posible añadir una nueva conversión sin modificar el traductor?

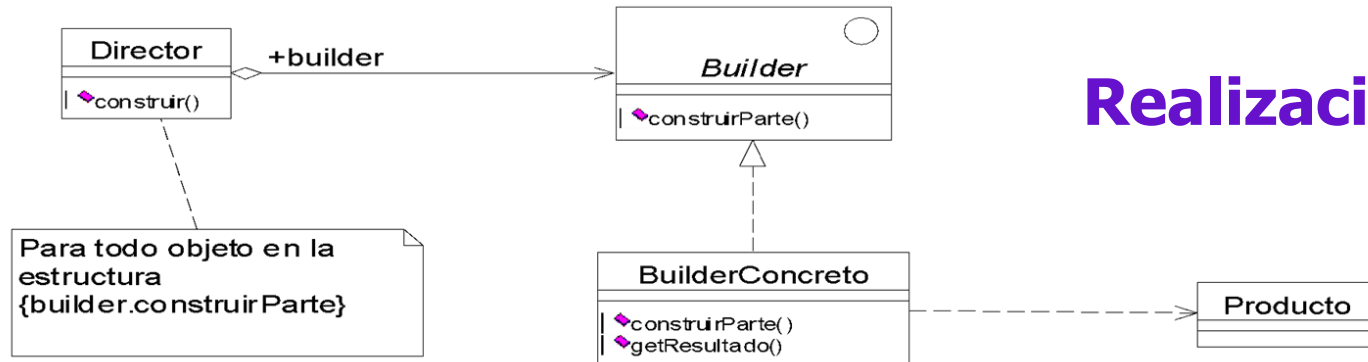
- Patrones de Creación
- Patrones de Estructura
- Patrones de Comportamiento

Builder (Constructor)

Herencia



Realización



Builder (Constructor)

◎ Aplicabilidad

- ◎ Cuando el algoritmo para crear un objeto complejo debe ser independiente de las piezas que conforman el objeto y de cómo se ensamblan.
- ◎ El proceso de construcción debe permitir diferentes representaciones para el objeto que se construye.

- Patrones de Creación
- Patrones de Estructura
- Patrones de Comportamiento

Builder (Constructor)

◎ Consecuencias

- ◎ Permite cambiar la representación interna del producto.
- ◎ Separa el código de la representación del código para la construcción.
- ◎ Proporciona un control fino del proceso de construcción.

- Patrones de Creación
- Patrones de Estructura
- Patrones de Comportamiento

Builder (Constructor)

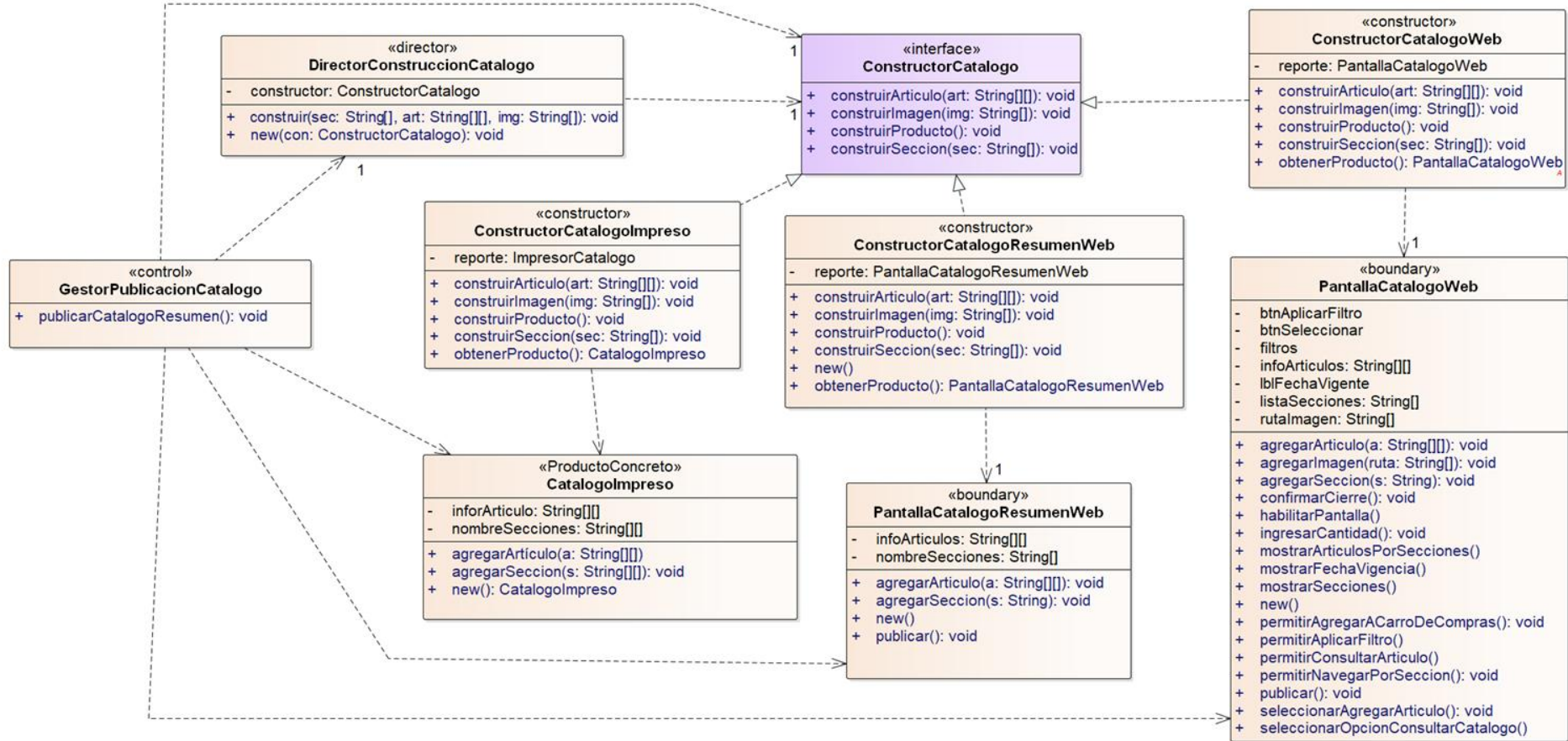
◎ Implementación

- ◎ La interfaz de *Builder* debe ser lo suficientemente general para permitir la construcción de productos para cualquier *Builder* concreto.
- ◎ La construcción puede ser más complicada de añadir el nuevo token al producto en construcción.
- ◎ Los métodos de *Builder* no son abstractos sino vacíos.
- ◎ Las clases de los productos no tienen una clase abstracta común.

- Patrones de Creación
- Patrones de Estructura
- Patrones de Comportamiento

Estructura

Builder: Aplicación



Builder: Aplicación

⊙ Participantes

- ⊙ Constructor [**ConstructorCatalogo**]: especifica una interfaz abstracta para crear las partes de un objeto complejo.
- ⊙ Constructor Concreto [**ConstructorCatalogoWeb, ConstructorCatalogoResumenWeb, ConstructorCatalogoImpreso**]: implementa la interfaz Constructor para construir y ensamblar las partes del objeto producto; define la representación a crear.
- ⊙ Director [**DirectorConstruccionCatalogo**]: es el encargado de crear un objeto usando la interfaz Constructor.
- ⊙ Producto [**PantallaCatalogoWeb, PantallaCatalogoResumenWeb, CatalogoImpreso**]: representa el objeto complejo que es construido.

Builder: Aplicación

⦿ Procedimiento

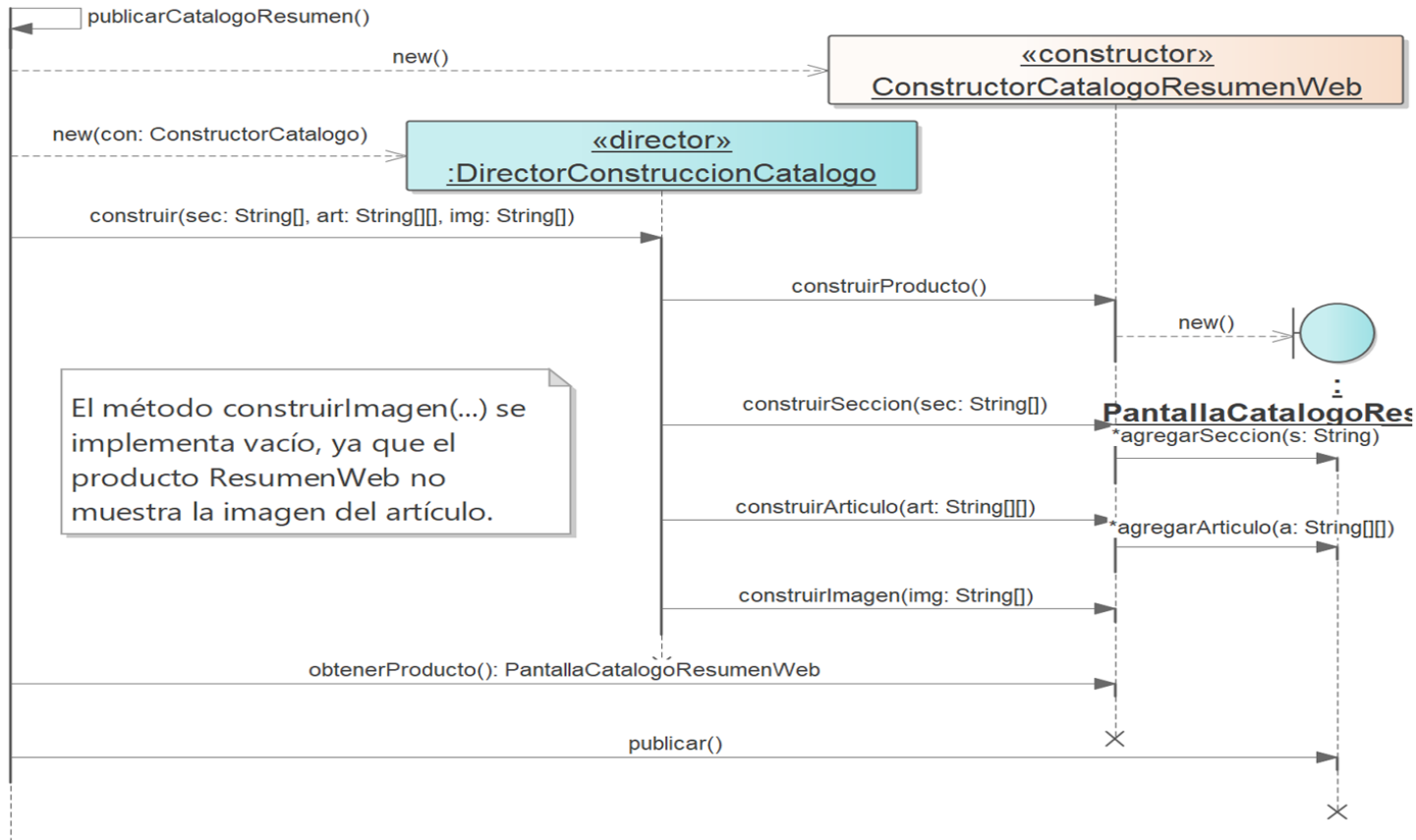
- ⦿ Definir las diferentes partes que deberán construirse para obtener el objeto completo.
- ⦿ Especificar la interfaz Constructor que define los métodos para construir cada una de las partes detallada anteriormente.
- ⦿ Establecer la clase responsable de dirigir, coordinar y controlar la construcción, clase Director.
- ⦿ Crear la o las clases que serán los constructores concretos de los productos a ensamblar, las cuales implementan la interfaz Constructor.



AplicacionCatalogo

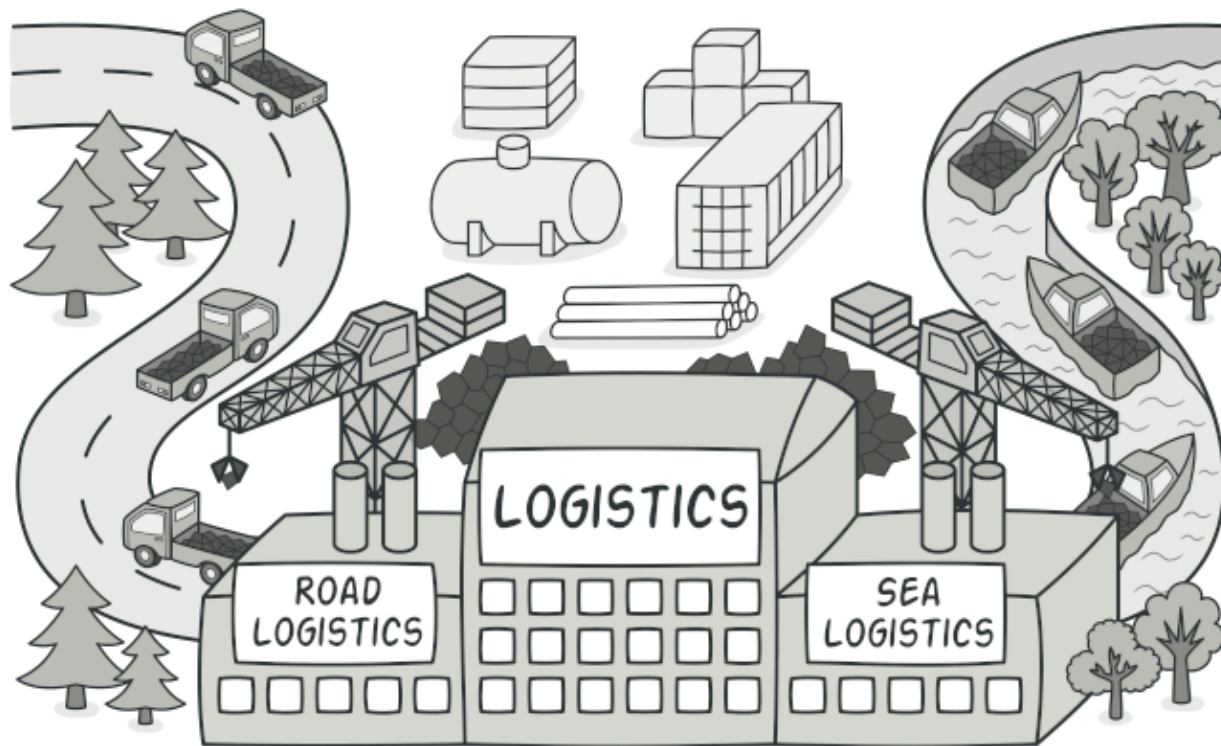
Builder: Aplicación

Dinámica:



Factory Method/ Virtual Constructor

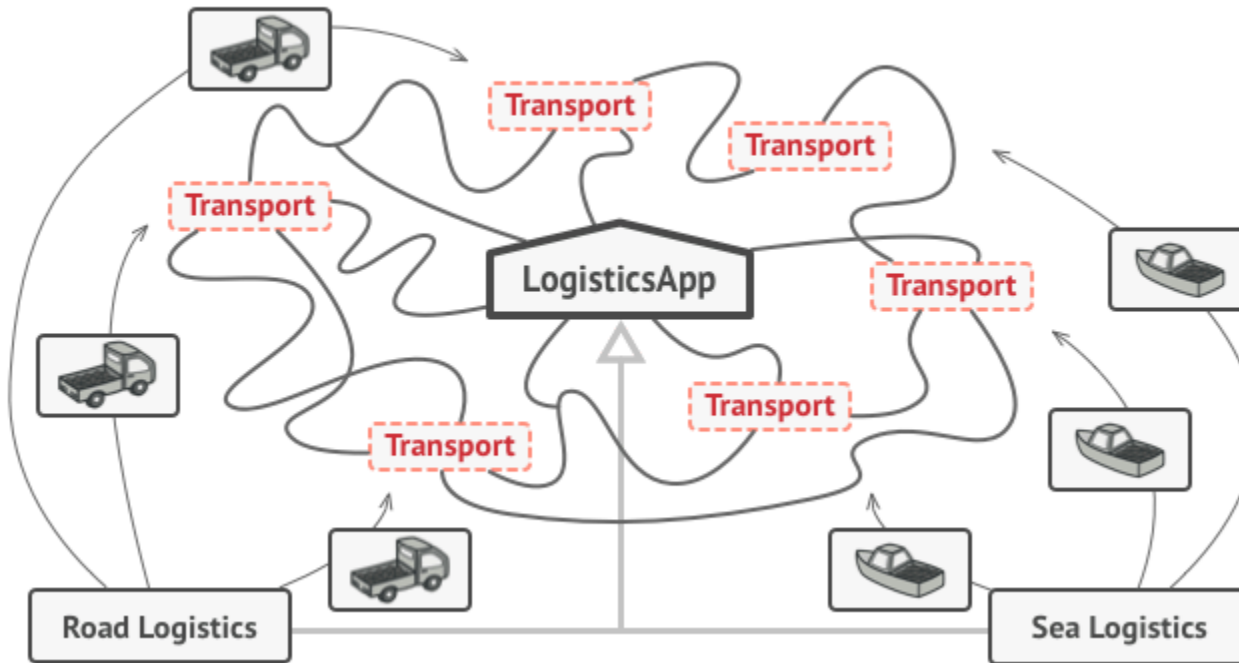
(Método de Factoría/ Constructor Virtual)



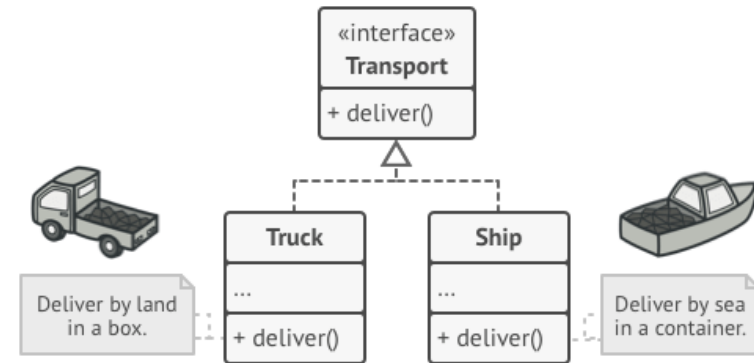
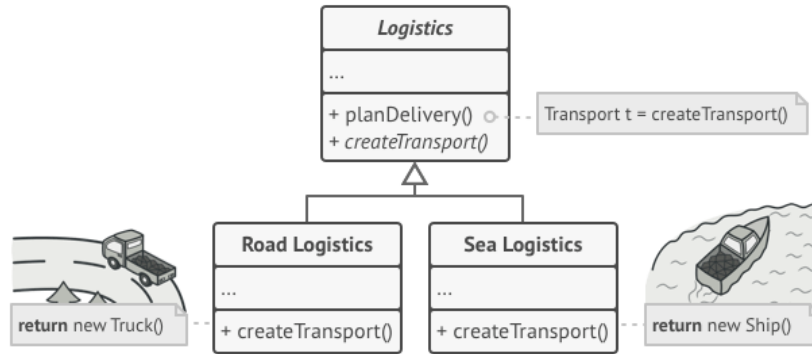
¿Qué problema resuelve?



Solución: Patrón Factory Method



Solución: Patrón Factory Method



Solución: Patrón Factory Method

Propósito

Define una interfaz para la creación de objetos delegando en las **subclases** la **decisión de qué clase instanciar**.

La intención del Factory Method es tener una clase a la cual delegar la responsabilidad de la creación de los objetos, para no decidir en tiempo de compilación que clase instanciará, si no que delegará esta responsabilidad al Factory confiando en que este le regresará la clase adecuada para trabajar.

Ámbito

De Clase

Solución: Patrón Factory Method

Aplicabilidad

- **Para casos en los que no se sabe en tiempo de diseño que subtipo vamos a necesitar**
- Cuando una clase desea que sus subclases especifiquen los objetos que debe crear.
- **Cuando se desea crear tipos de objetos relacionados por una base común sin preocuparnos de los detalles de creación del objeto. La finalidad es que los mismos puedan variar sin afectar la forma en que el cliente interactúa con ellos.**

Solución: Patrón Factory Method

Implementación

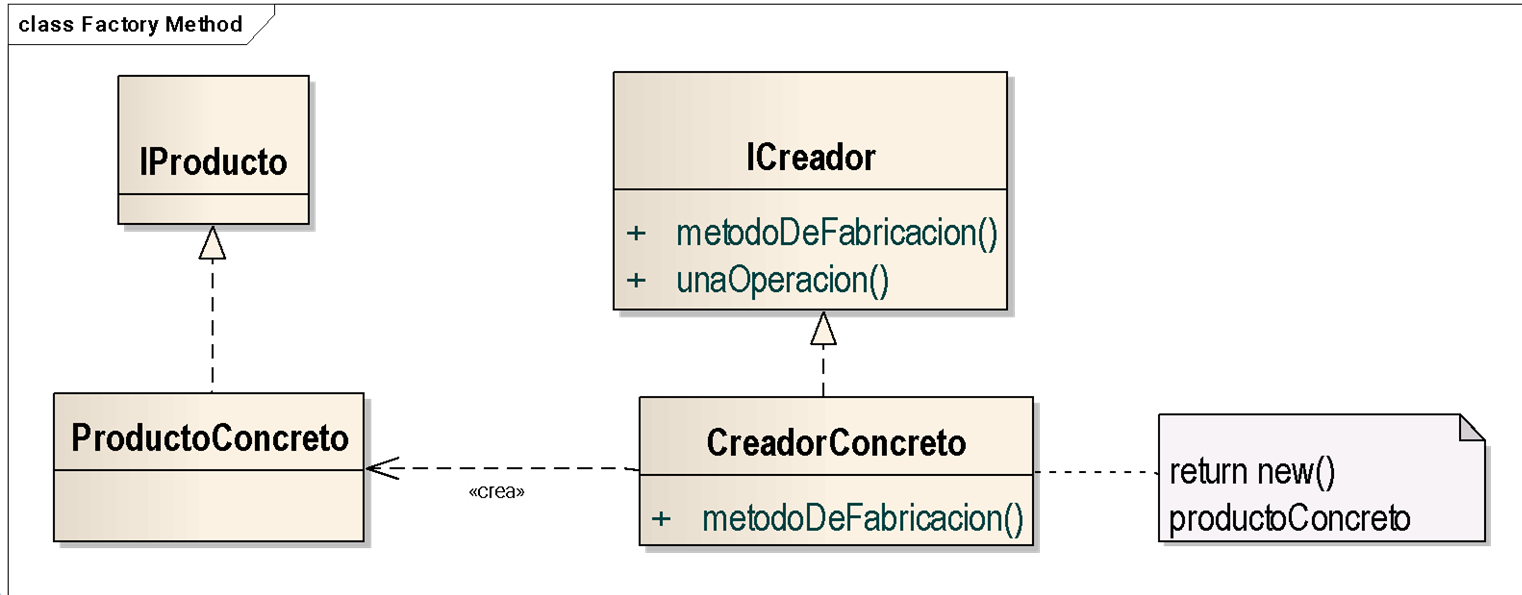
1. Haga que todos los productos sigan la misma interfaz. Esta interfaz debe declarar métodos que tengan sentido en cada producto.
2. Agregue un método de fábrica vacío dentro de la clase creadora. El tipo de retorno del método debe coincidir con la interfaz de producto común
3. En el código del creador encuentre todas las referencias a los constructores de productos. Uno por uno, reemplácelos con llamadas al método de fábrica, mientras extrae el código de creación del producto en el método de fábrica.

Solución: Patrón Factory Method

Implementación (Continuación)

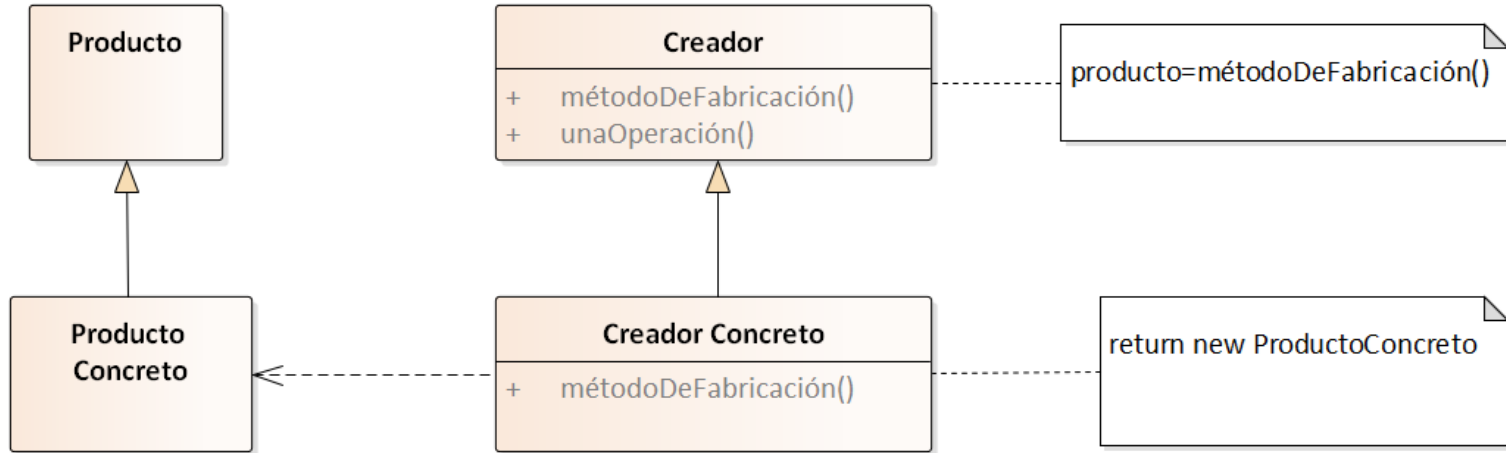
4. Si, después de todas las extracciones, el método de fábrica base se ha quedado vacío, puede hacerlo abstracto. Si queda algo, puede convertirlo en un comportamiento predeterminado del método.

Estructura - Realización



Estructura - Herencia

class Estructura Factory Method Herencia



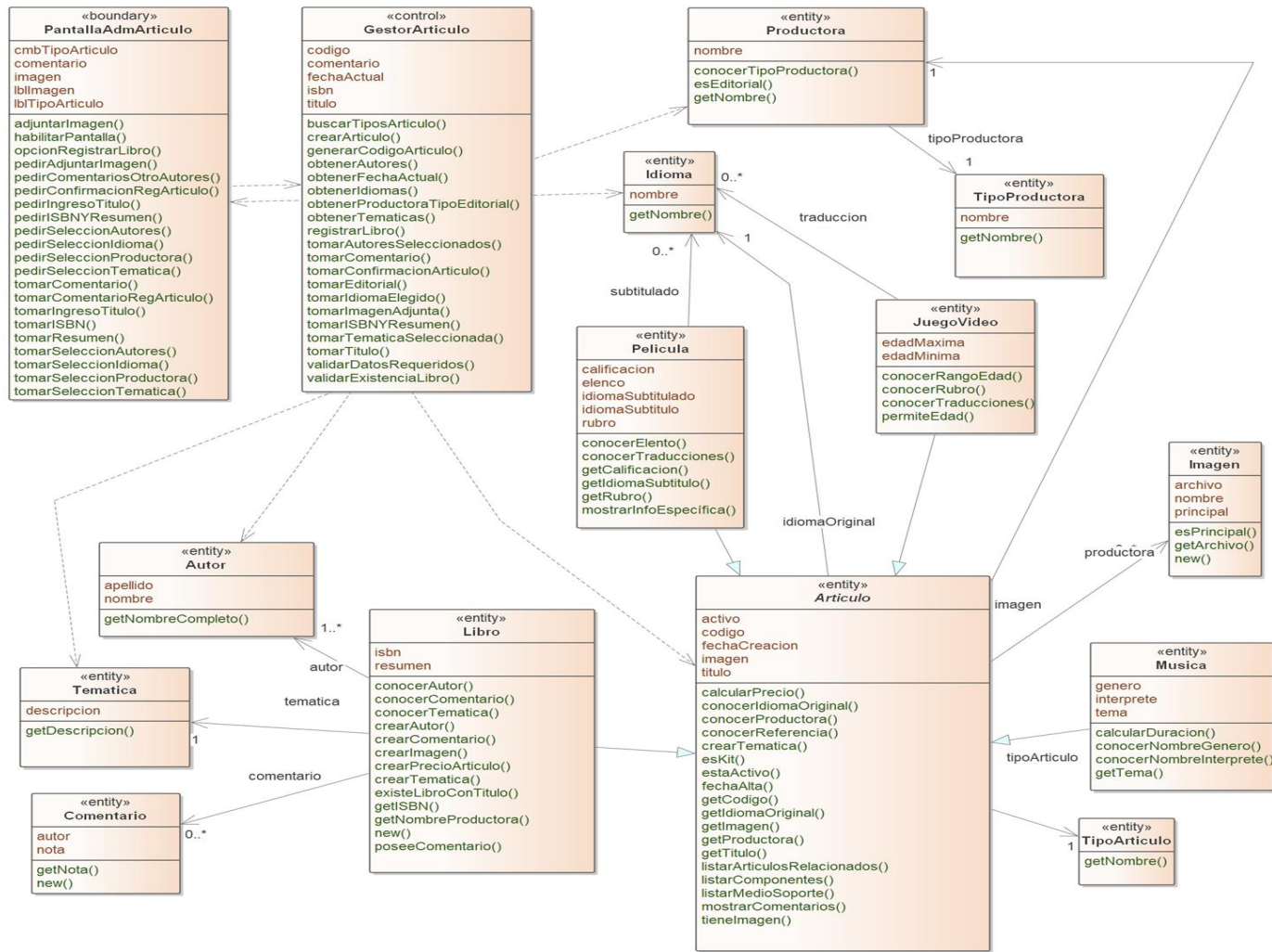
Aplicación Venta Digital

Implementar la creación de un nuevo artículo sin que en tiempo de compilación sea necesario determinar cuál es el tipo de artículo que se va a crear. Deben evitarse condicionales que cambian el comportamiento de la aplicación según el tipo de artículo que desee crearse.

Factory Method

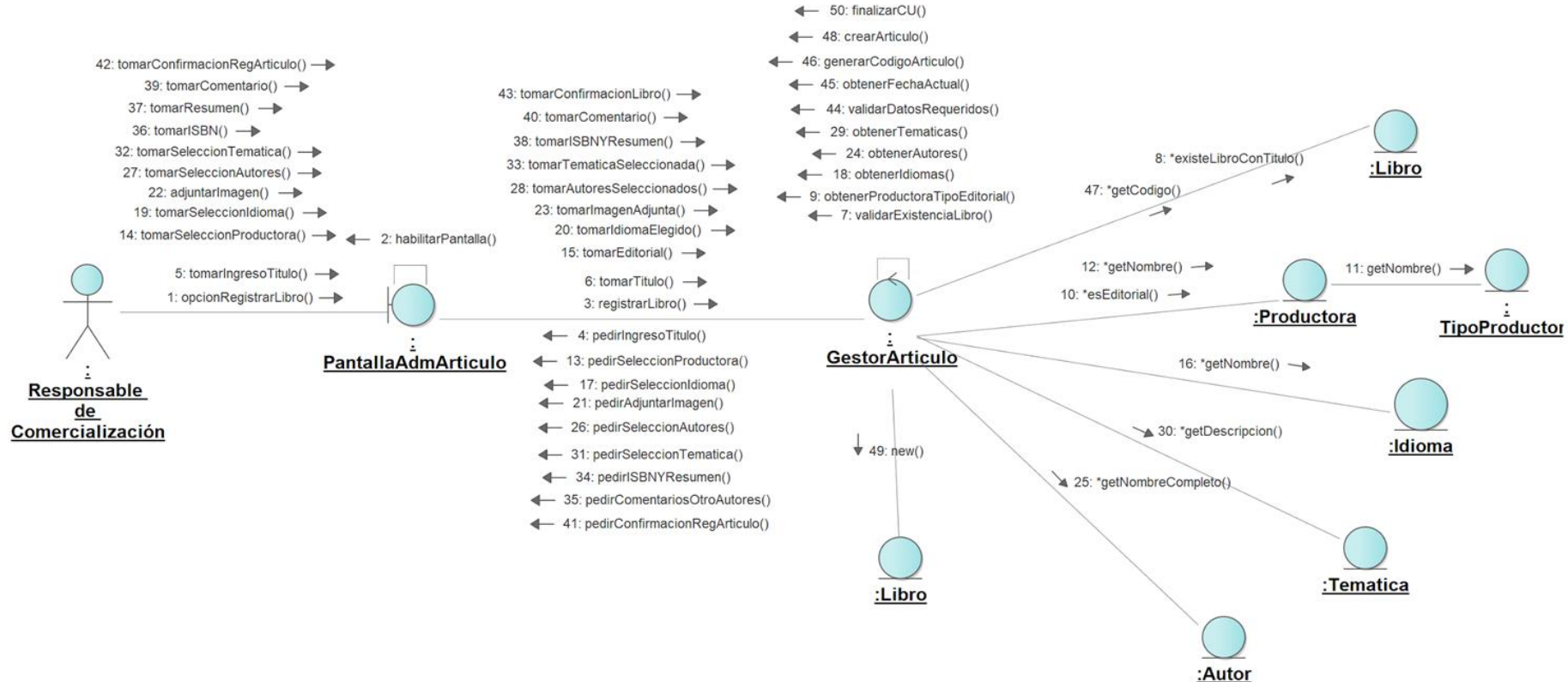
Aplicación Venta Digital Estructura Análisis – CU Registrar Artículo

class UC-Registrar Artículo



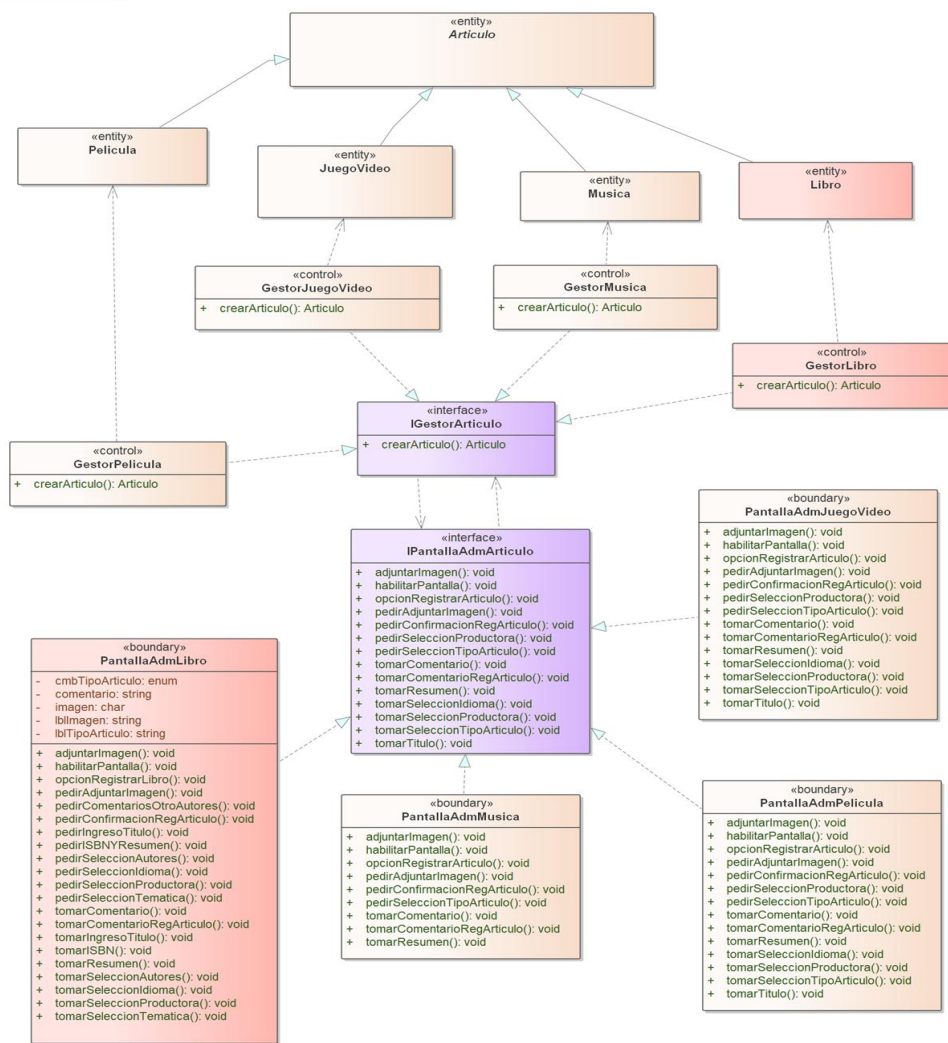
Aplicación Venta Digital

Dinámica Análisis – CU Registrar Artículo



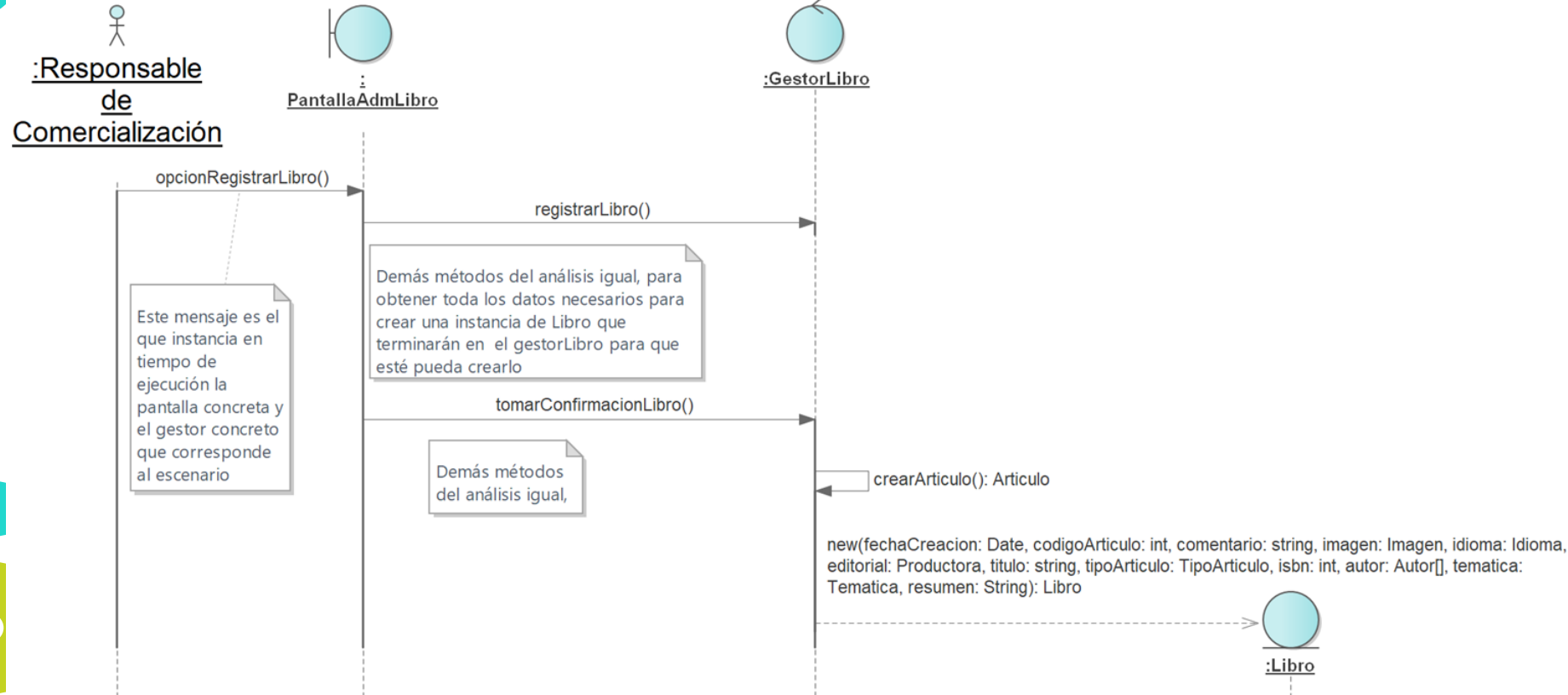
Aplicación Venta Digital Estructura Diseño – CU Registrar Artículo

class Factory Method



Aplicación Venta Digital

Dinámica Diseño – CU Registrar Artículo



Aplicación Venta Digital

Participantes

- **Producto** [*Articulo*]: Declara una interfaz de los objetos que crea el método de Fabricación.
- **ProductoConcreto** [*Libro, JuegoVideo, Pelicula, Musica*]: Implementa las operaciones declaradas en la interfaz de producto.
- **Creador** [*IGestorArticulo*]: Declara el método de fabricación, el cual devuelve un objeto de tipo Articulo.
- También puede definir una implementación predeterminada del método de fabricación que devuelva un objeto del producto concreto (si fuese herencia).
- Puede llamar al método de fabricación para crear un objeto Articulo.
- **Creadores Concretos** [*GestorLibro, GestorPelicula, GestorMusica, GestorJuegoVideo*]: Redefine el método de fabricación para devolver una instancia de un producto concreto.

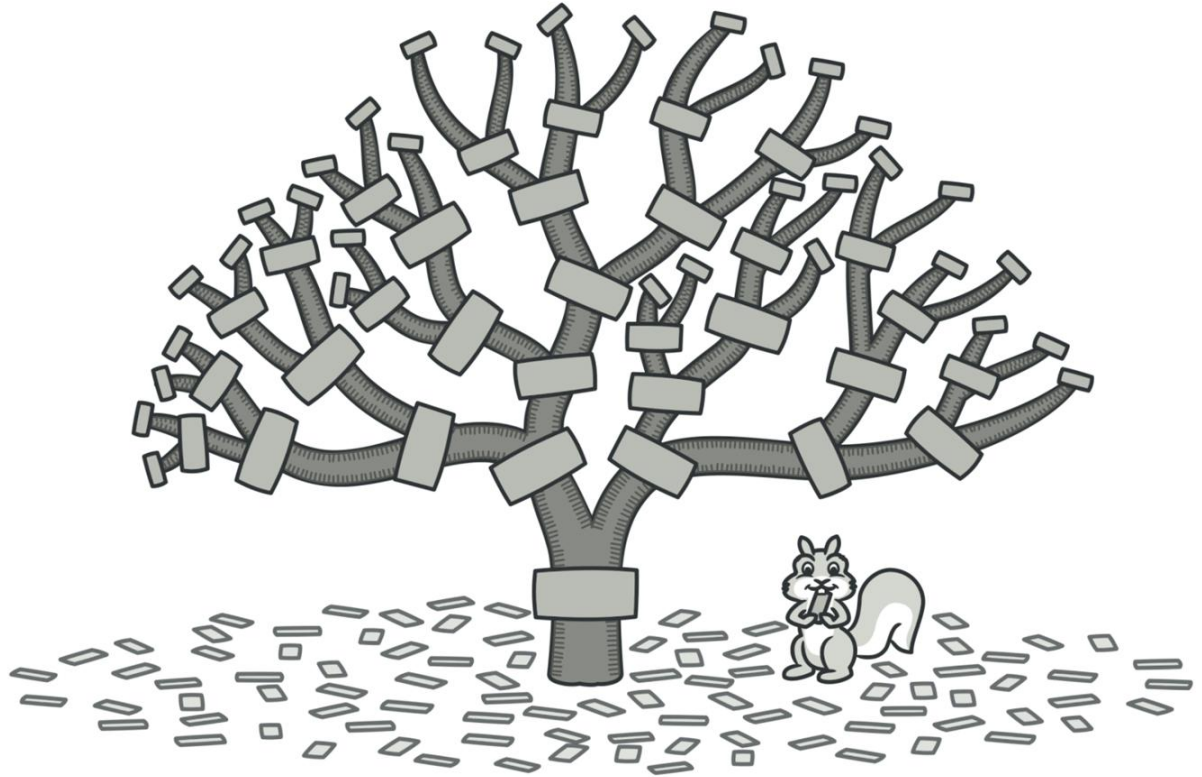
Ventajas

- ❖ Evita el acoplamiento estrecho entre el creador y los productos concretos.
- ❖ Principio de responsabilidad única . Puede mover el código de creación del producto a un lugar en el programa, haciendo que el código sea más fácil de soportar.
- ❖ Principio abierto / cerrado . Puede introducir nuevos tipos de productos en el programa sin romper el código de cliente existente

Desventajas

- ❖ El código puede volverse más complicado ya que necesita introducir muchas subclases nuevas para implementar el patrón.
- ❖ El mejor de los casos es cuando se introduce el patrón en una jerarquía existente de clases de creadores.

Composite



Composite

◎ Propósito

- Componer objetos en estructuras jerárquicas para representar jerarquías todo/parte. Permite a los clientes manejar a los objetos primitivos y compuestos de forma uniforme.

◎ Motivación

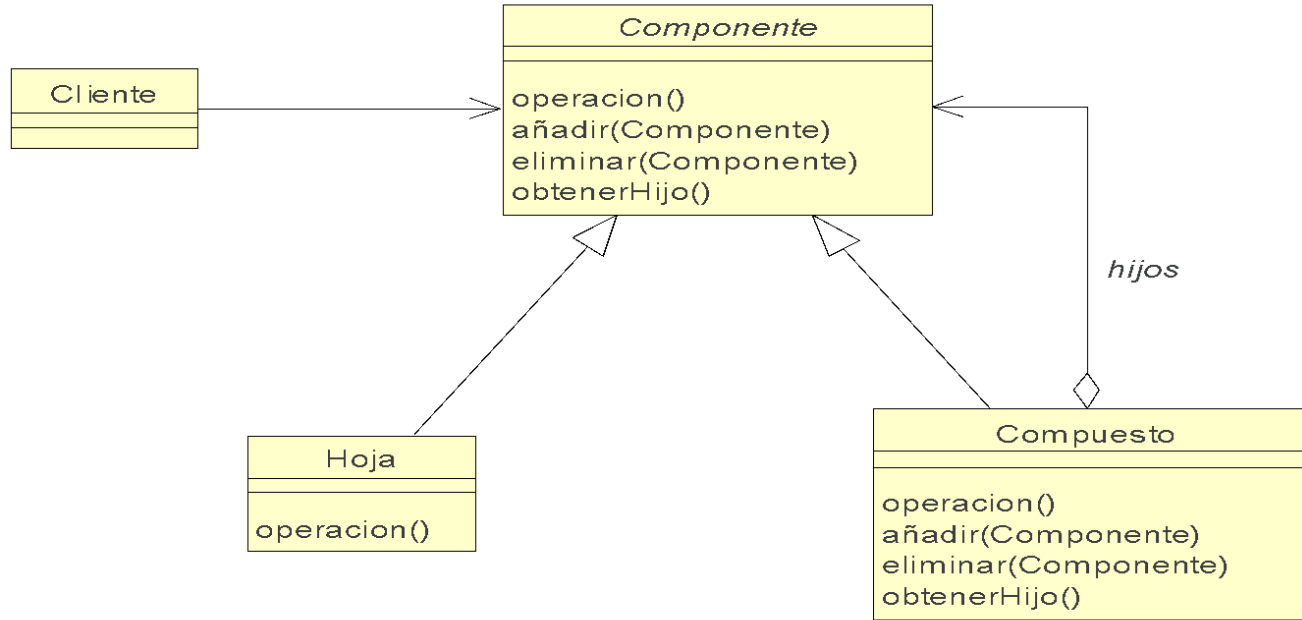
- Modelar figuras compuestas
- Modelar paquetes de valores (acciones, bonos,...)

- Patrones de Creación
- Patrones de Estructura
- Patrones de Comportamiento

Composite

Estructura

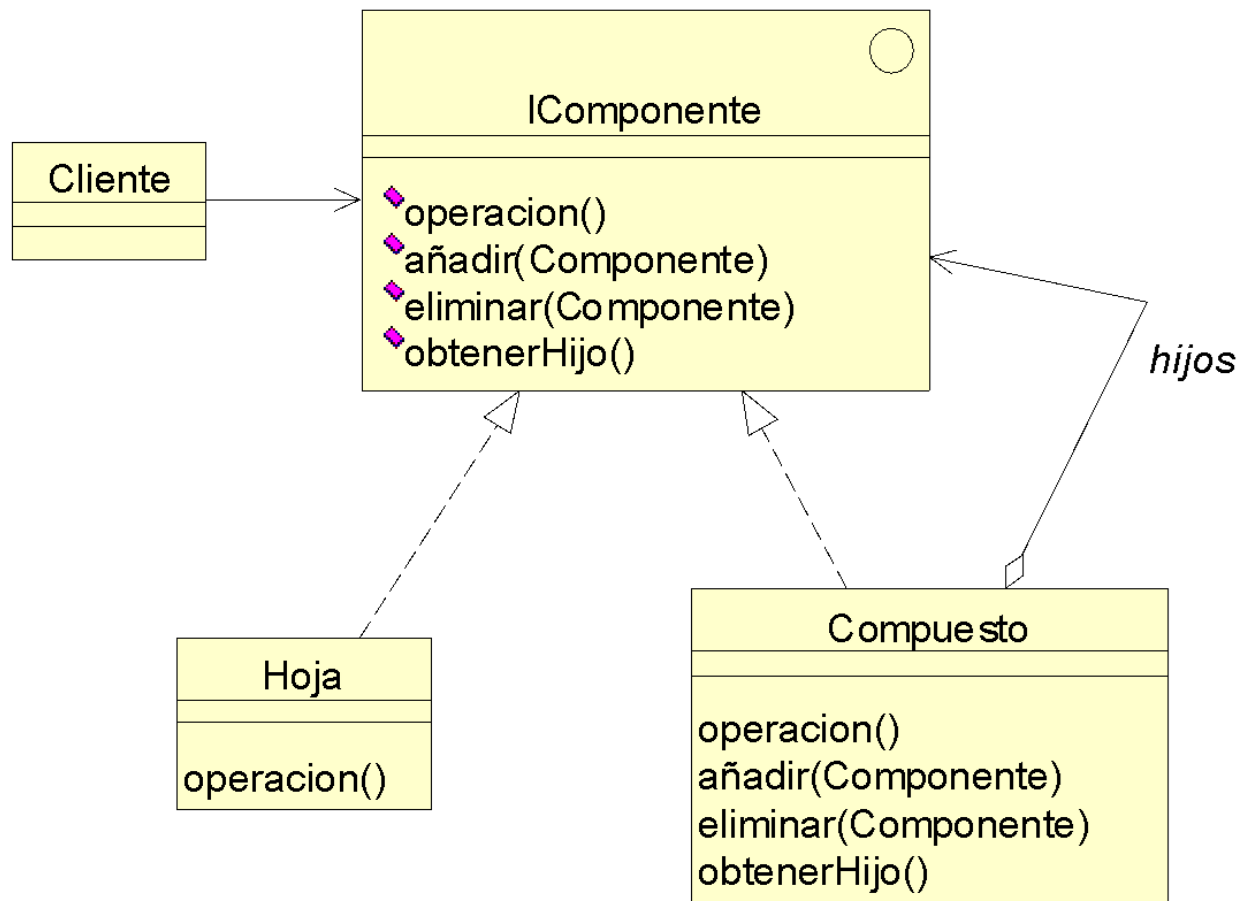
Herencia

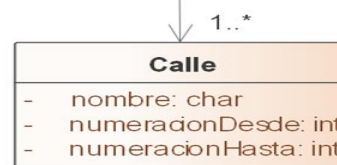
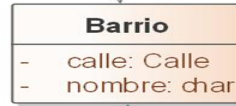
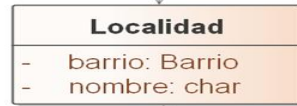
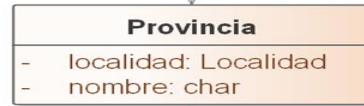
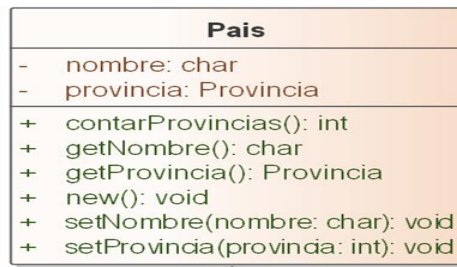


Composite

Estructura

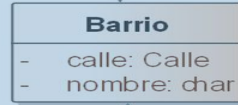
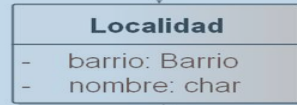
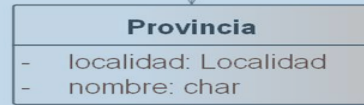
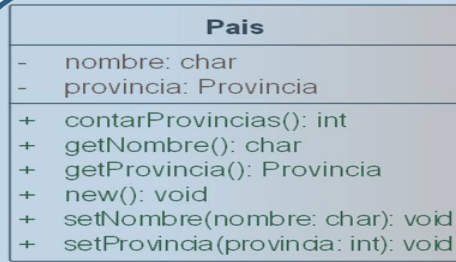
Realización





¿Cómo se
afecta la
estructura al
aplicar el
patrón
Composite?
Un ejemplo con
la estructura
geográfica

**Com-
puesto**



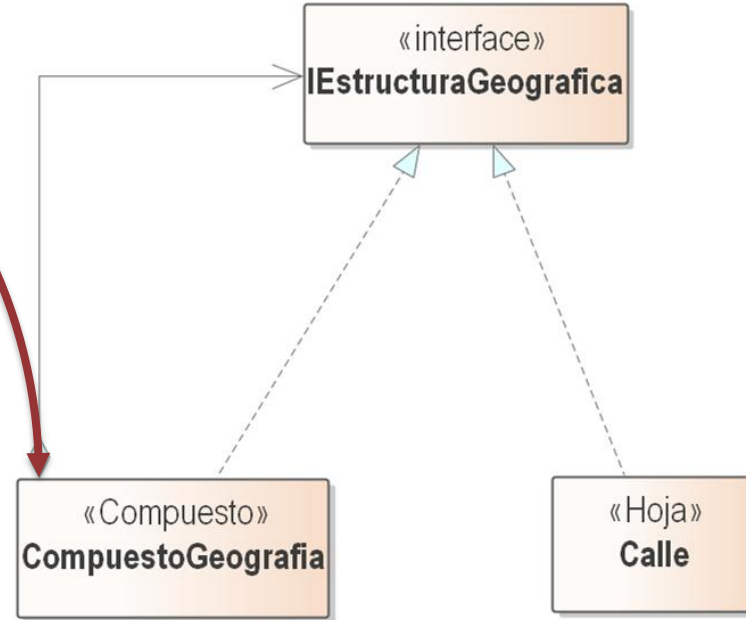
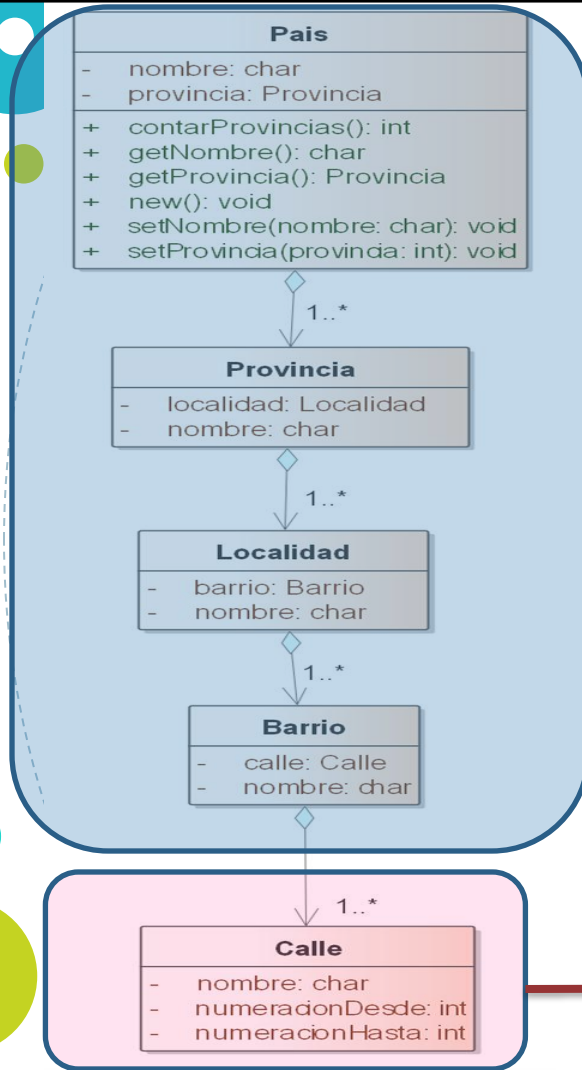
Hoja



¿Cómo se
afecta la
estructura al
aplicar el
patrón
Composite?
Un ejemplo
con la
estructura
geográfica

¿Cómo se afecta la estructura al aplicar el patrón Composite?

Un ejemplo con la estructura geográfica



Composite

◎ Aplicabilidad

- ◎ Se quiere representar jerarquías todo/parte
- ◎ Se quiere que los clientes ignoren la diferencia entre objetos compuestos y los objetos individuales que los forman.

- Patrones de Creación
- Patrones de Estructura
- Patrones de Comportamiento

Composite

◎ Consecuencias

- ◎ Jerarquía con clases que modelan objetos primitivos y objetos compuestos, que permite composición recursiva.
- ◎ Clientes pueden tratar objetos primitivos y compuestos de modo uniforme.
- ◎ Es fácil añadir nuevos tipos de componentes.
- ◎ No se puede confiar al sistema de tipos que asegure que un objeto compuesto sólo contendrá objetos de ciertas clases, necesidad de comprobaciones en tiempo de ejecución.

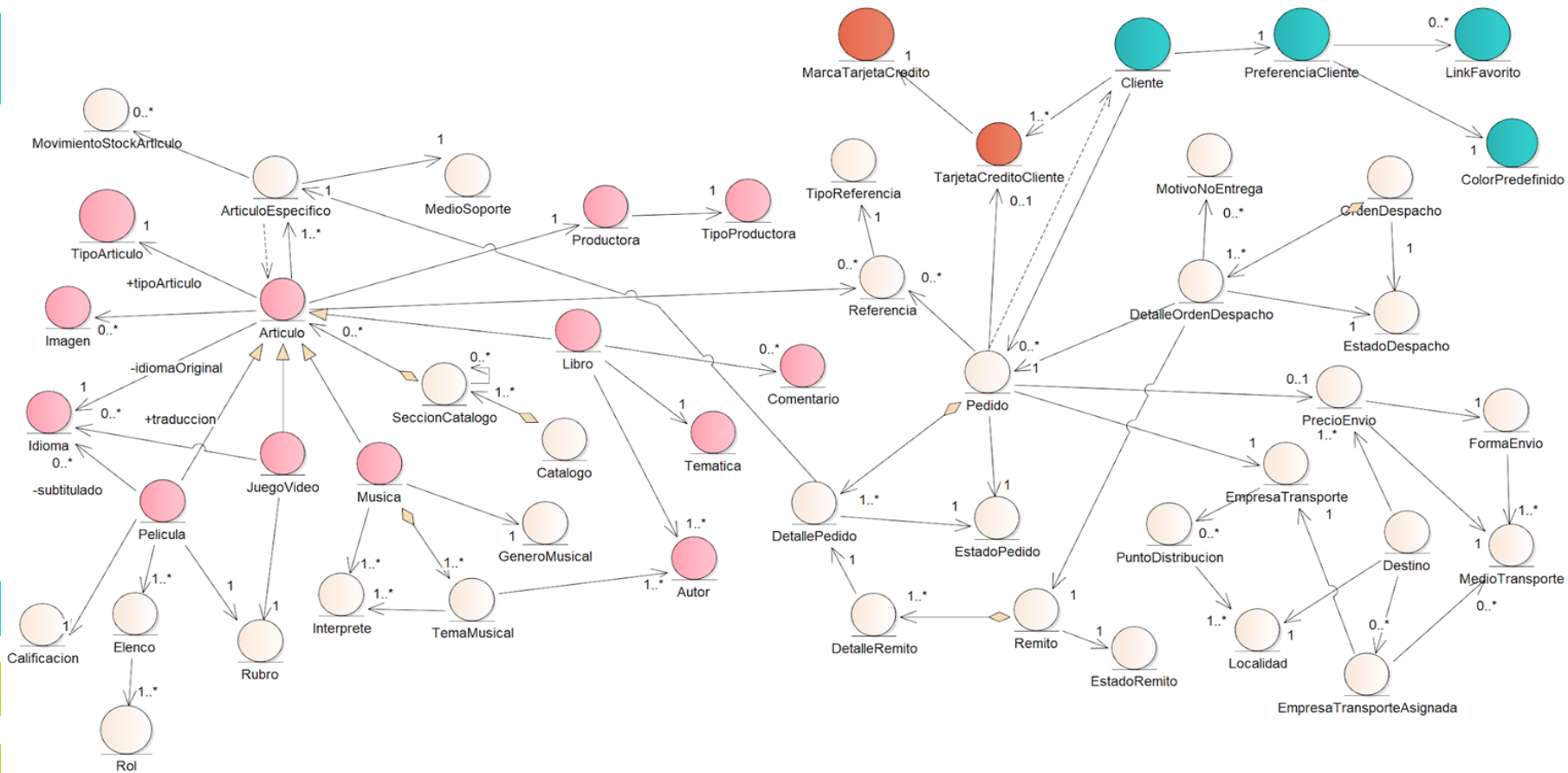
- Patrones de Creación
- Patrones de Estructura
- Patrones de Comportamiento

Composite

◎ Implementación

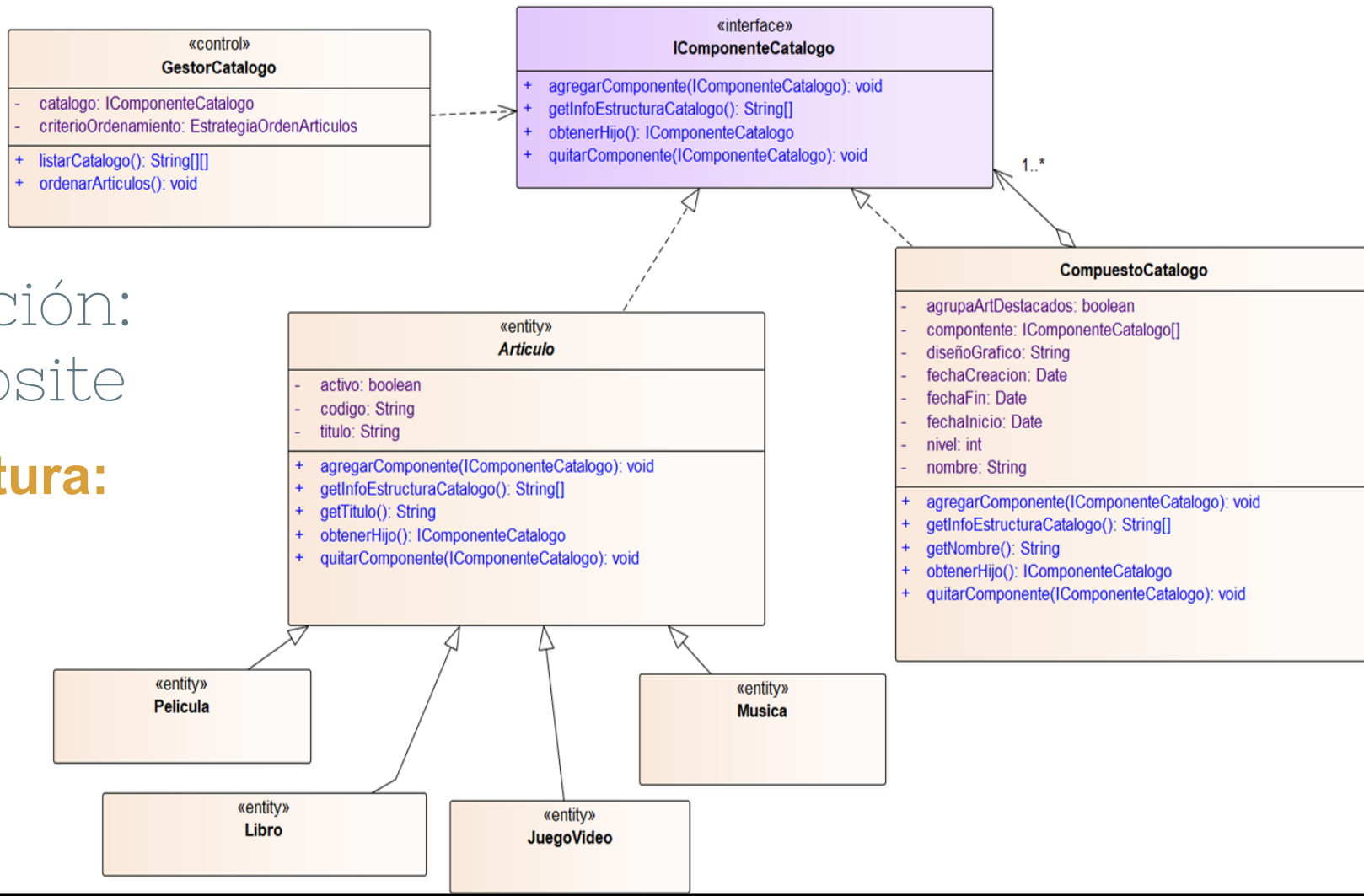
- ◎ Identificar clases de agregación
- ◎ Identificar componentes y hojas.
- ◎ Referencias explícitas al padre, para simplificar el recorrido y la gestión de una estructura compuesta.
- ◎ Maximizar la interfaz Componente, para que los clientes se despreocupen de las clases Hoja o Compuesto.
- ◎ Declarar las operación de gestión de los hijos: equilibrio entre seguridad y transparencia.

177



Aplicación: Composite

Estructura:



Composite

◎ Participantes

- ◎ Componente (**IComponenteCatalogo**)
 - Declara la interfaz de los objetos de la composición.
- ◎ Hoja (**Articulo: Libro, Película, Música, JuegoDeVideo**)
 - Implementa la operación `getDescripciónJerarquía()` que es uniforme a toda la jerarquía.
- ◎ Compuesto (**CompuestoCatalogo**)
 - Implementa todas las operaciones definidas en el Componente (**IComponenteCatalogo**) y las operaciones específicas para la administración de los elementos que lo componen.
- ◎ Cliente (**GestorCatalogo**)
 - Realiza la petición al objeto que implementa la interfaz Componente

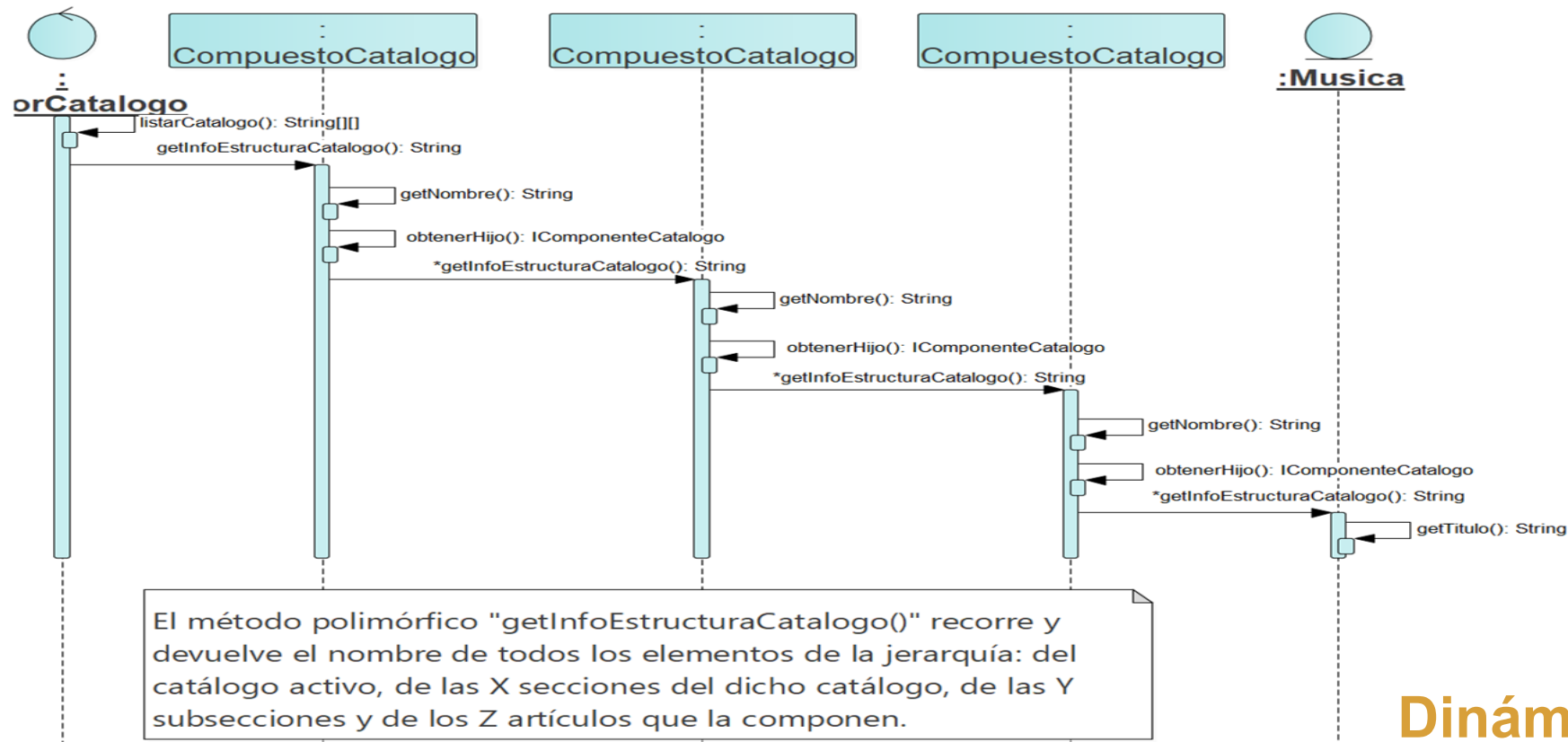
Aplicación

Catálogo

Secciones (donde está Música Internacional)

Subsecciones (donde está The Beatles)

Artículos entre los
que está el álbum Let
it Be



Dinámica

Composite

◉ Procedimiento

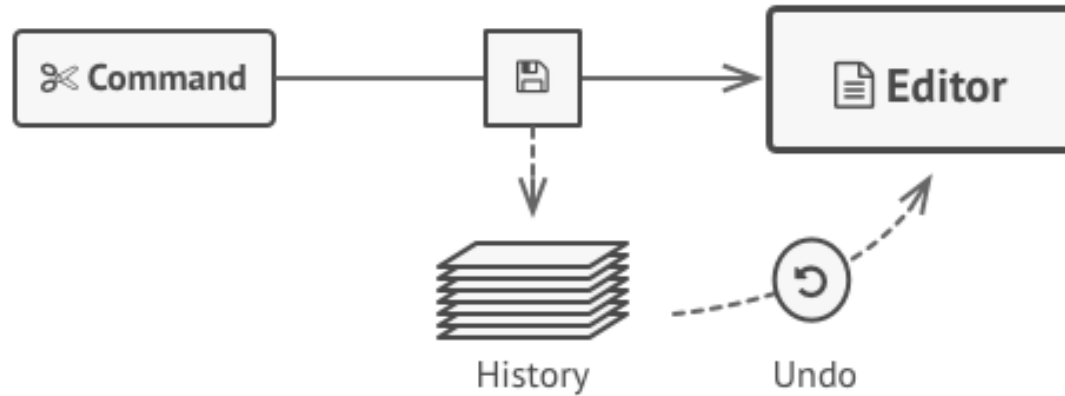
- ◉ El **GestorCatalogo** debe armar y mostrar toda la jerarquía del catálogo activo (*X* secciones, *Y* subsecciones y *Z* artículos) cuando el usuario desea consultarlo, para lo cual le pide a la interfaz (**InterfazComponenteCatalogo**) que lo resuelva a través del método: `getDescripciónJerarquía()`.
- ◉ Las clases (**Articulo** y **CompuestoCatalogo**) implementan la interfaz **InterfazComponenteCatalogo**.
- ◉ Para el caso de **Articulo** (hoja) los métodos de manipulación de hijos `agregarComponente()`, `quitarComponente()` y `obtenerHijo()` se implementan VACIOS.
- ◉ El **Compuesto** (**CompuestoCatalogo**), es quien resuelve la petición recursivamente hasta armar todo el árbol.
- ◉ En el caso del ejemplo, el Catálogo está compuesto por Secciones entre las cuales está Música Internacional, Subsecciones entre las subsecciones de la sección Música Internacional está The Beatles y debe recorrer los artículos que la componen entre los cuáles está el álbum Let It Be.
- ◉ En la segunda parte del ejemplo, se debe recuperar el nombre de un artículo (hoja), para ello no es necesario que intervenga el objeto **Compuesto**.
- ◉ De esa manera el **GestorCatalogo** obtiene el nombre de cada elemento del catálogo y su ruta de ubicación, sin necesidad de conocer la cantidad de niveles de división del catálogo.

Memento/ Token

(Recuerdo)

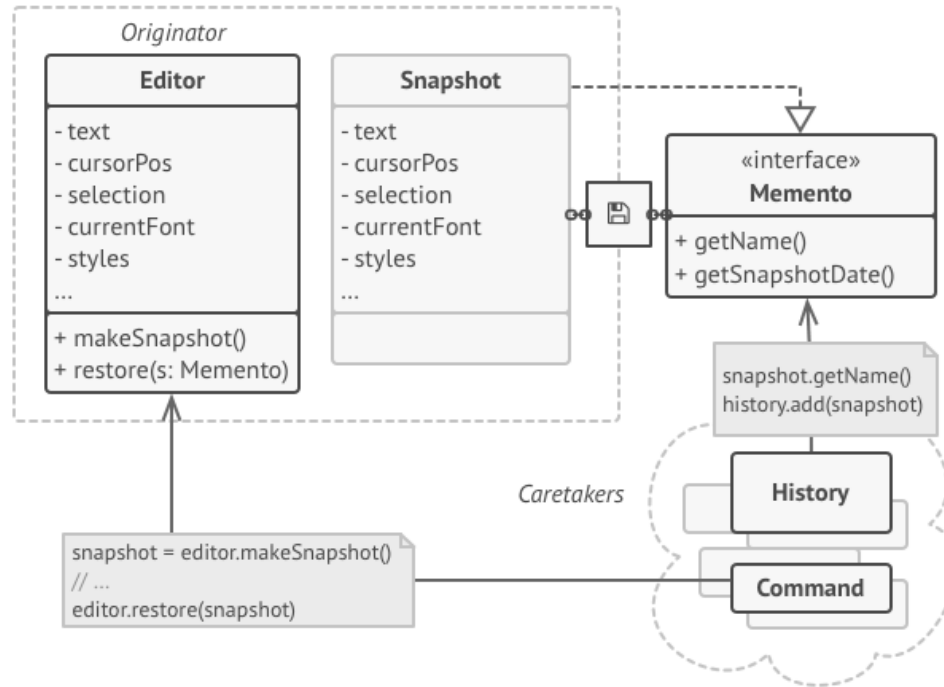


¿Qué problema resuelve?



Antes de ejecutar una operación, la aplicación guarda una instantánea del estado de los objetos, que luego se puede utilizar para restaurar los objetos a su estado anterior.

Solución: Patrón Memento



Memento

◎ Propósito

- Captura y exterioriza el estado interno de un objeto, sin violar el encapsulamiento, de modo que el objeto puede ser restaurado a ese estado más tarde.

◎ Motivación

- Algunas veces es necesario registrar el estado interno de un objeto: mecanismos *checkpoints* y deshacer cambios que permiten probar operaciones o recuperación de errores.
- Mecanismo de transacciones
 - Patrones de Creación
 - Patrones de Estructura
 - Patrones de Comportamiento

Memento

◎ Aplicabilidad

- ◎ Una parte del estado de un objeto debe ser guardado para que pueda ser restaurado más tarde.
- ◎ Una interfaz para obtener el estado de un objeto podría romper el encapsulamiento exponiendo detalles de implementación.

- Patrones de Creación
- Patrones de Estructura
- Patrones de Comportamiento

Memento

◎ Consecuencias

- ◎ Mantiene el encapsulamiento
- ◎ Simplifica la clase *Creador* ya que no debe preocuparse de mantener las versiones del estado interno.
- ◎ Podría incurrir en un considerable gasto de memoria: encapsular y restaurar el estado no debe ser costoso.
- ◎ Puede ser difícil en algunos lenguajes asegurar que sólo el *Creador* tenga acceso al estado del *Memento*.

- Patrones de Creación
- Patrones de Estructura
- Patrones de Comportamiento

Solución: Patrón Memento

Aplicabilidad

- Use el patrón Memento cuando desee producir instantáneas del estado del objeto para poder restaurar un estado anterior del objeto.
- Cuando una interfaz para obtener el estado de un objeto podría romper el encapsulamiento exponiendo detalles de implementación.

Solución: Patrón Memento

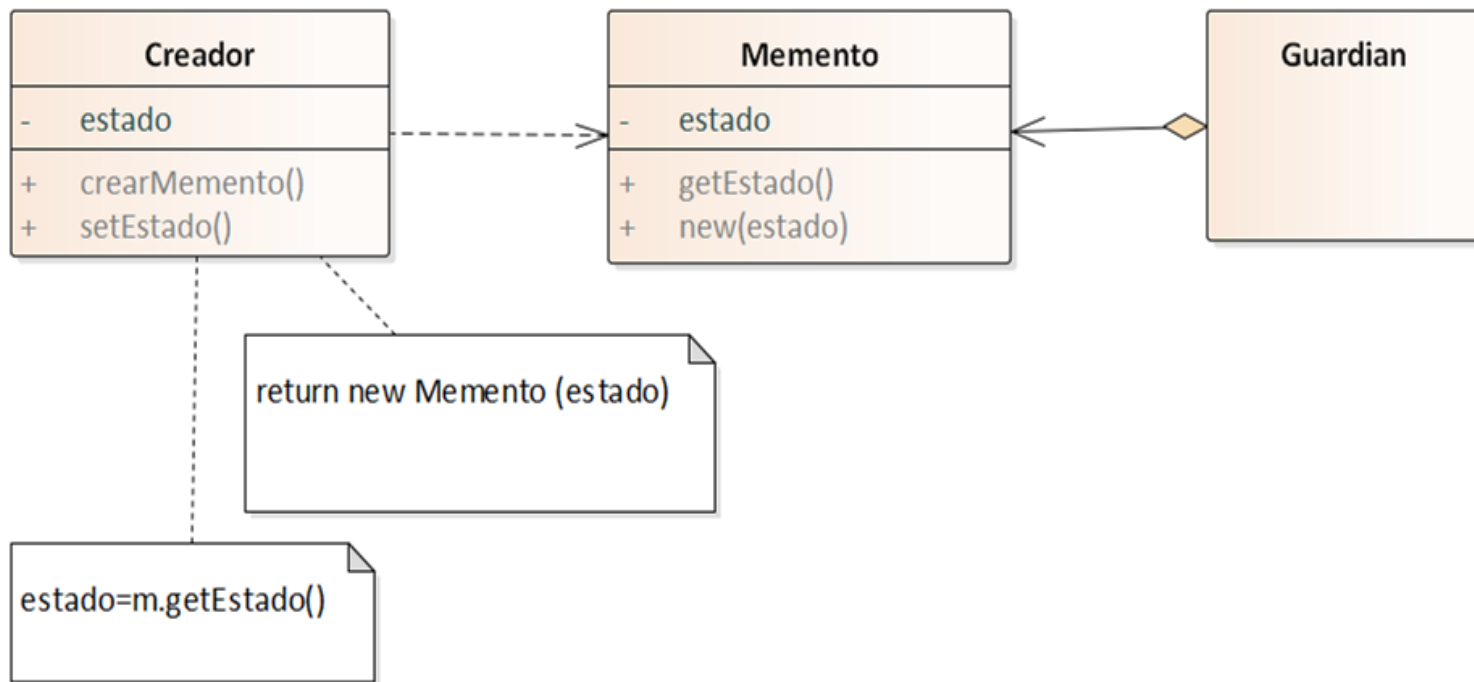
Implementación

1. **Determine qué clase desempeñará el papel del Creador.**
1. **Crea la clase Memento. Uno por uno, declare un conjunto de campos que reflejen los campos declarados dentro de la clase Creador.**
1. **Haz que la clase Memento sea inmutable. Un Memento debe aceptar los datos solo una vez, a través del constructor. La clase no debe tener métodos de seteo.**

Solución: Patrón Memento

Implementación (Continuación)

- 4. **Agregue un método para producir recuerdos a la clase Creador y otro para restaurar el estado del originador a su clase. Debe aceptar un objeto de Memento como argumento.**
- 4. **El Guardián, ya sea que represente un historial o algo completamente diferente, debe saber cuándo solicitar nuevos recuerdos al Creador, cómo almacenarlos y cuándo restaurar el Creador con un recuerdo particular.**
- 4. **El vínculo entre los Guardianes y los creadores puede trasladarse a la clase Memento. En este caso, cada Memento debe estar conectado al creador que lo creó. El método de restauración también pasaría a la clase memento**

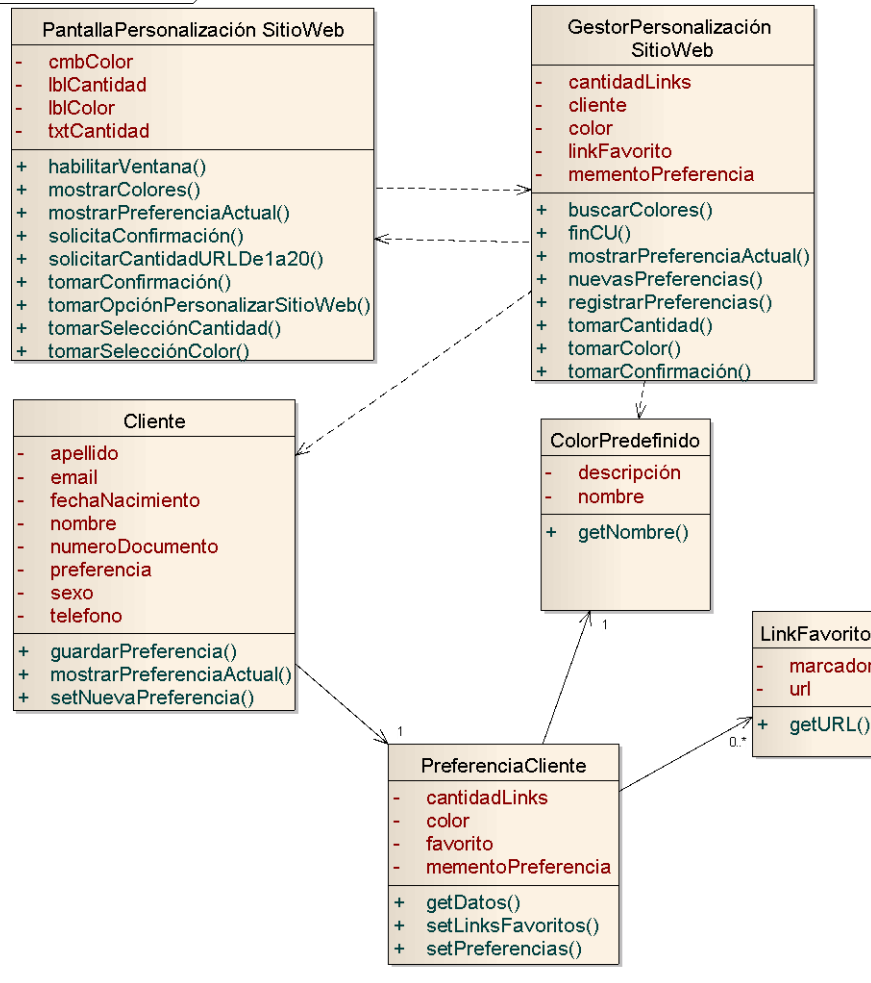


Aplicación Venta Digital

Permitir **guardar en el sitio las preferencias del cliente y restaurarlas en el momento en que el cliente lo desee sin revelar los detalles de su implementación**

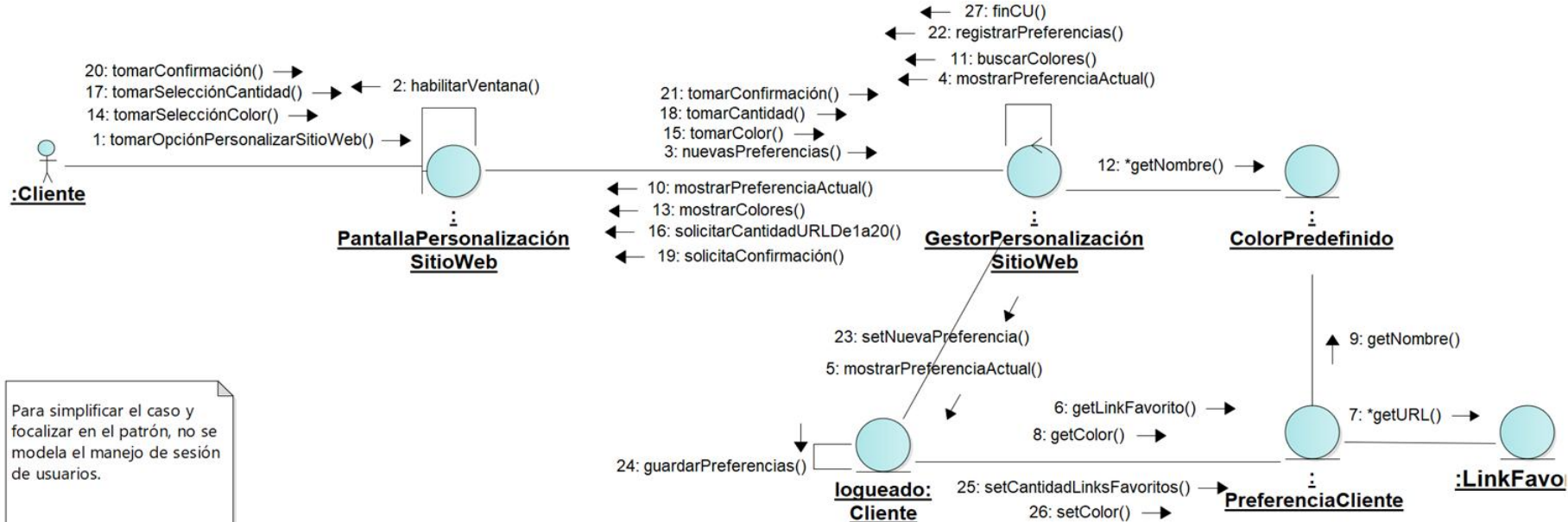
Aplicación Venta Digital Estructura Análisis – CU Personalizar sitio web

class UC - Personalizar sitio w...

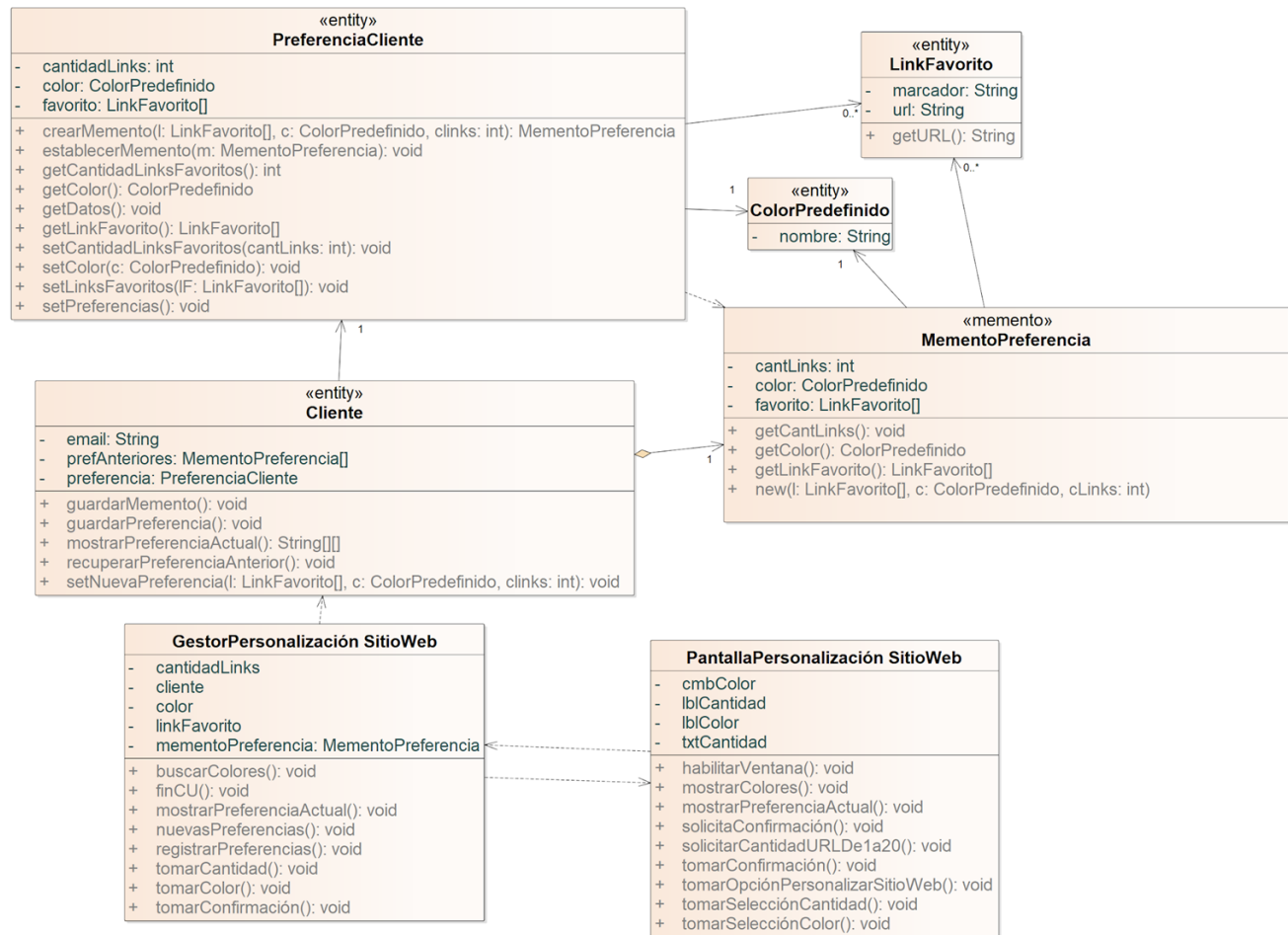


Aplicación Venta Digital

Dinámica Análisis – CU Personalizar sitio web

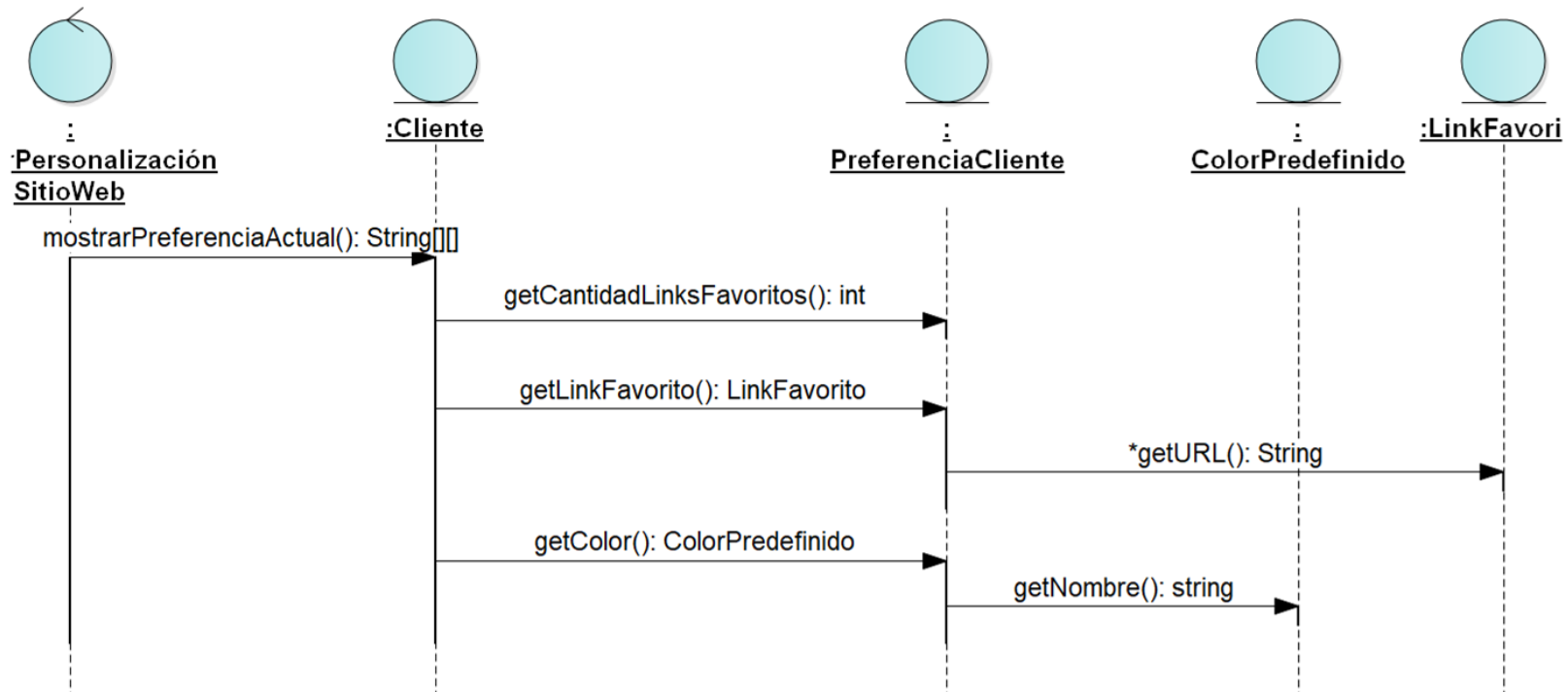


Aplicación Venta Digital Estructura – CU Personalizar sitio web



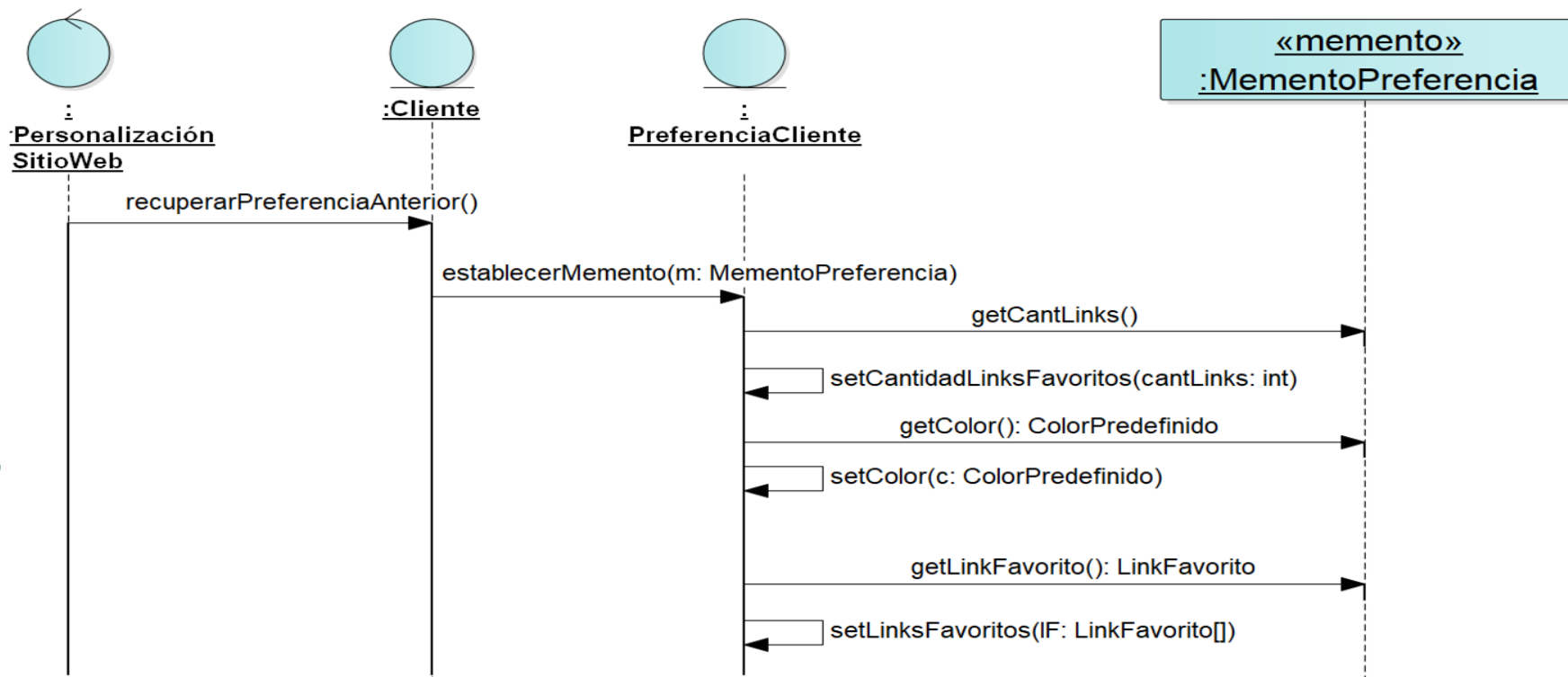
Aplicación Venta Digital

Dinámica – Mostrar Preferencias Actuales



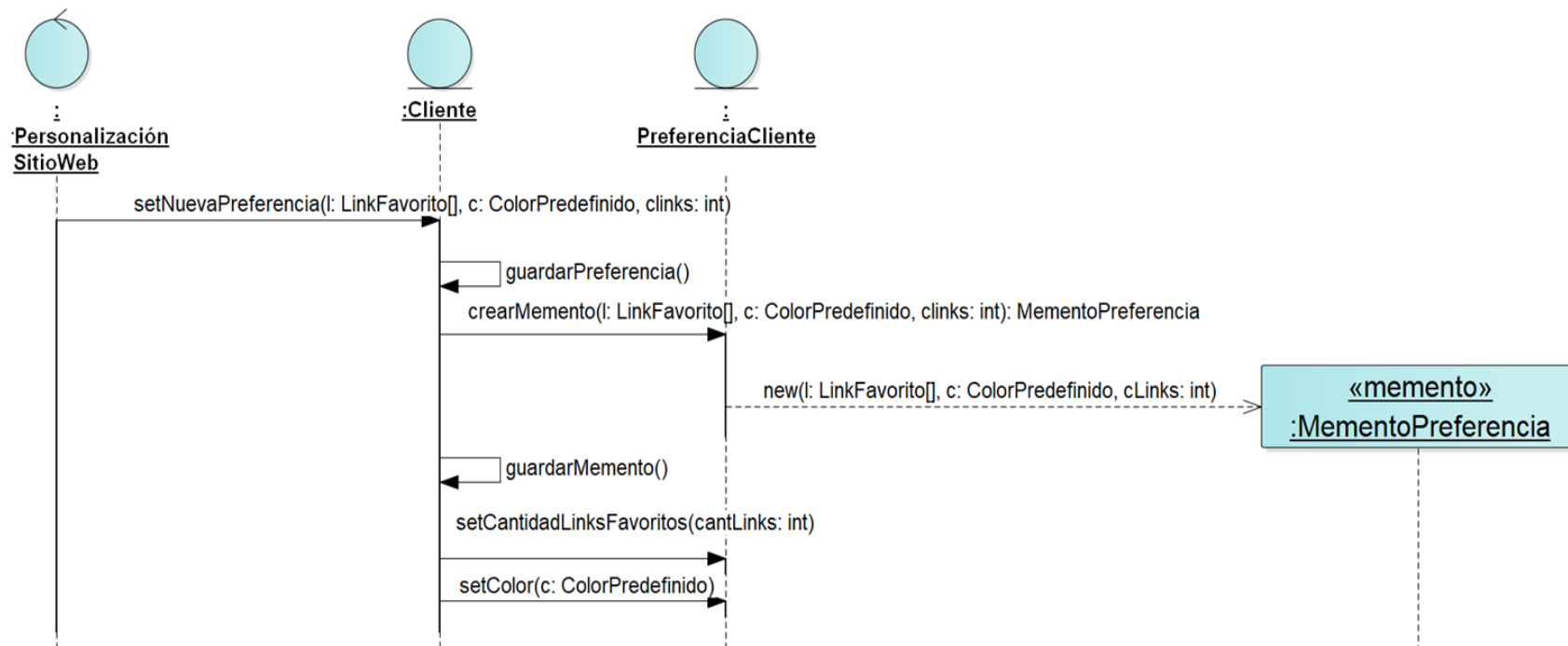
Aplicación Venta Digital

Dinámica – Recuperar Preferencias anteriores



Aplicación Venta Digital Dinámica – Creación del Memento

Memento 198



Aplicación Venta Digital

Participantes - Realización

- **Creador** [**PreferenciasCliente**]: es quien va a crear y/o restaurar el Memento de preferencias del cliente.
- **Memento** [**MementoPreferencia**]: cada instancia contiene las preferencias de un cliente, en cuanto a color, links, etc. que tuvo el cliente en un momento determinado.
- **Guardián** [**Cliente**]: Es quien contiene a todas las instancias de MementoPreferencia.
- **Cliente** [**GestorPersonalizacionSitioWeb**]: Es quién solicita las preferencias actuales o anteriores del cliente y también la creación de las mismas.

Ventajas

- ❖ Puede producir instantáneas del estado del objeto sin violar su encapsulamiento.
- ❖ Puede simplificar el código del autor dejando que el cuidador mantenga el historial del estado del autor.

Desventajas

- ❖ La aplicación puede consumir mucha RAM si los clientes crean recuerdos con demasiada frecuencia.
- ❖ Los guardianes deben rastrear el ciclo de vida del creador para poder destruir recuerdos obsoletos.
- ❖ Puede ser difícil en algunos lenguajes asegurar que sólo el Creador tenga acceso al estado del Memento.

Façade / Fachada

◎ Propósito

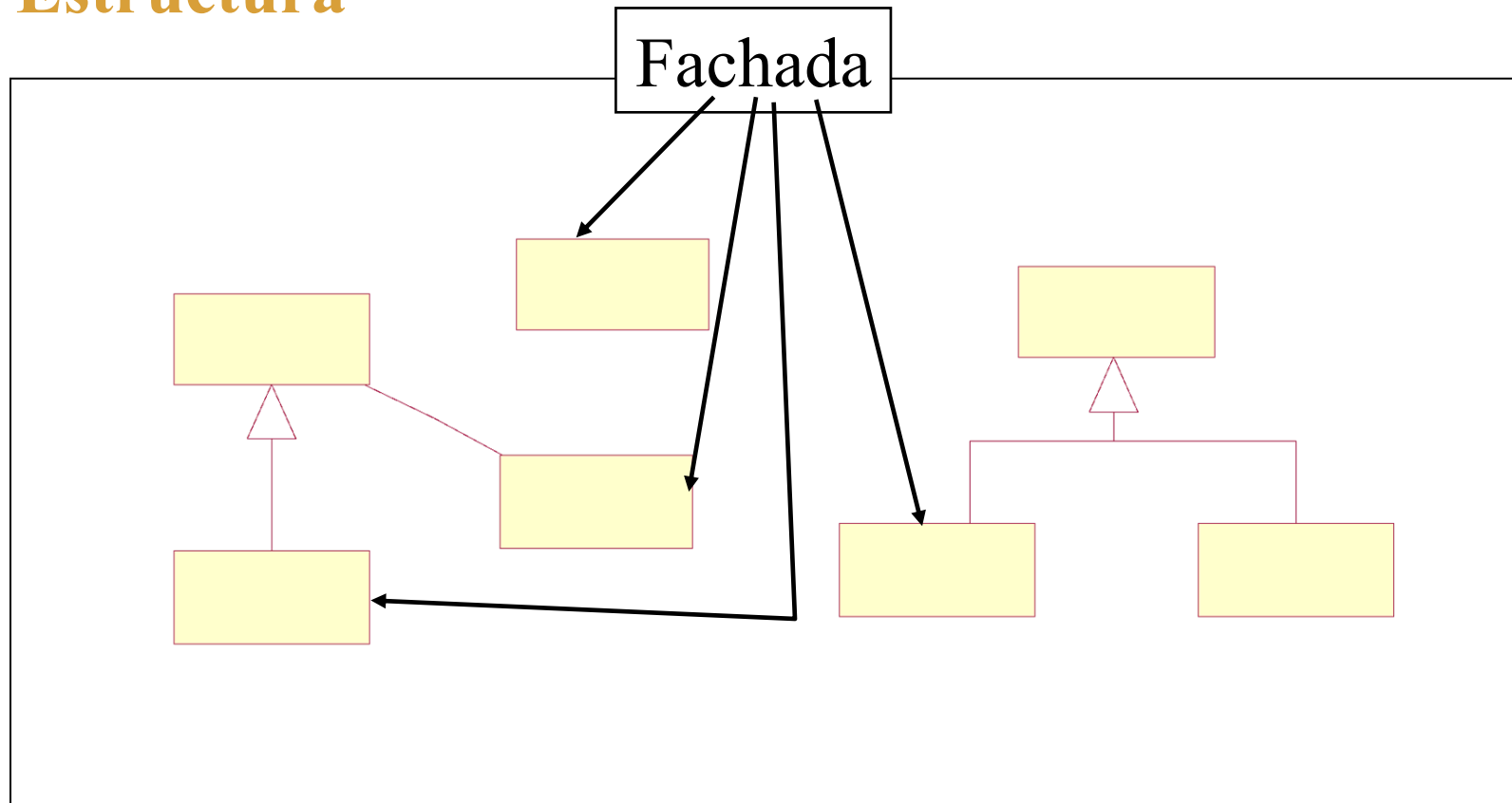
- Proporciona una única interfaz a un conjunto de interfaces de un subsistema. Define una interfaz de más alto nivel que facilita el uso de un subsistema.

◎ Motivación

- Reducir las dependencias entre subsistemas.
- Un entorno de programación que ofrece una librería de clases que proporcionan acceso a su subsistema compilador: *Scanner*, *Parser*, *ProgramNode*, *ByteCodeStream* y *ProgramNodeBuilder*. Clase *Compiler* actúa como fachada.

- Patrones de Creación
- Patrones de Estructura
- Patrones de Comportamiento

Estructura



Façade / Fachada

◎ Aplicabilidad

- ◎ Proporcionar una interfaz simple a un subsistema.
- ◎ Hay muchas dependencias entre clientes y las clases que implementan una abstracción.
- ◎ Se desea una arquitectura de varios niveles: una fachada define el punto de entrada para cada nivel subsistema.

- Patrones de Creación
- Patrones de Estructura
- Patrones de Comportamiento

Façade / Fachada

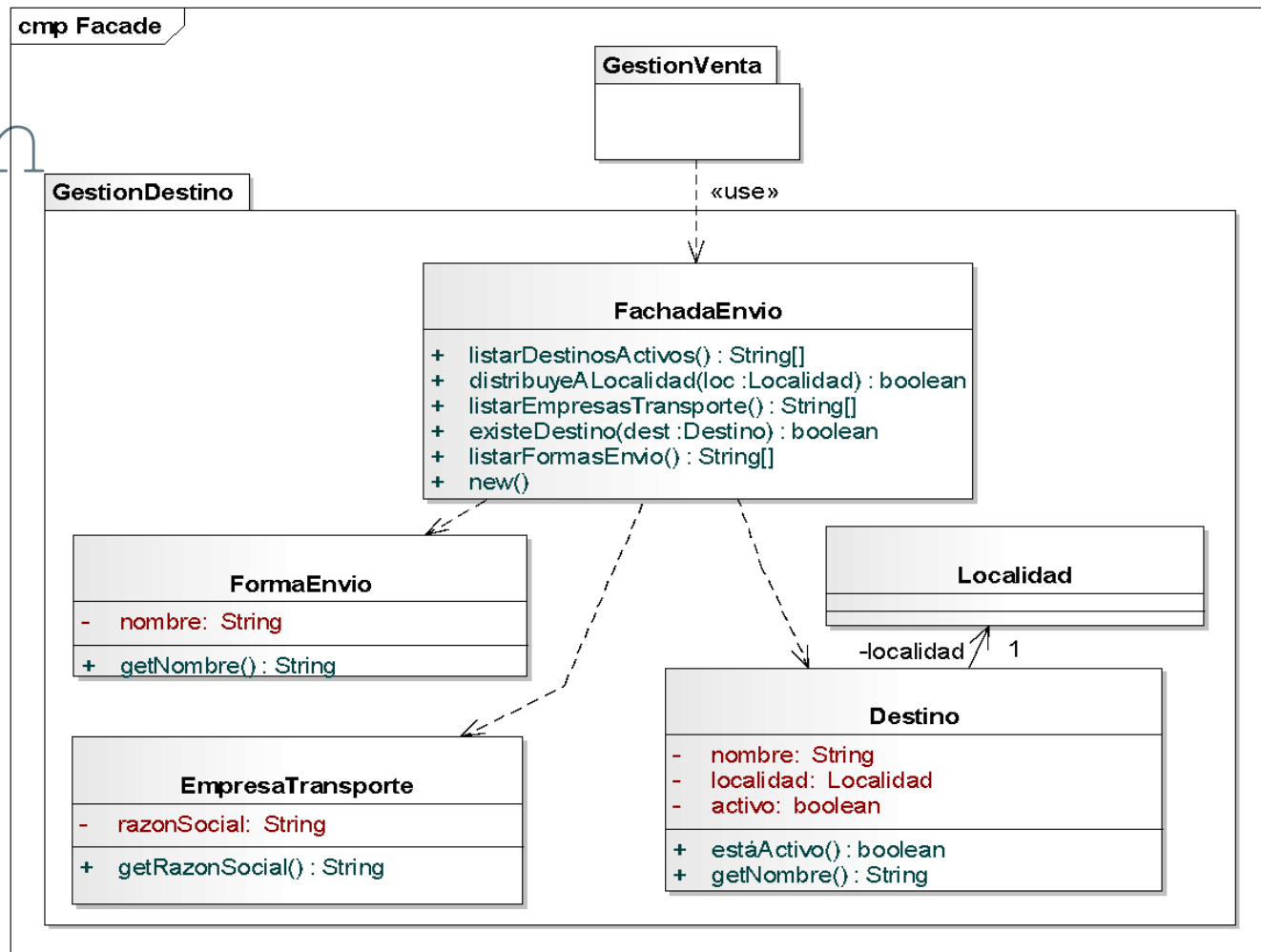
◎ Consecuencias

- ◎ Facilita a los clientes el uso de un subsistema, al ocultar sus componentes.
- ◎ Proporciona un acoplamiento débil entre un subsistema y los clientes: cambios en los componentes no afectan a los clientes.
- ◎ No se impide a los clientes el uso de las clases del subsistema si lo necesitan. Se puede elegir entre facilidad y funcionalidad

- Patrones de Creación
- Patrones de Estructura
- Patrones de Comportamiento

Façade: Aplicación

Estructura



Façade / Fachada

⦿ Participantes

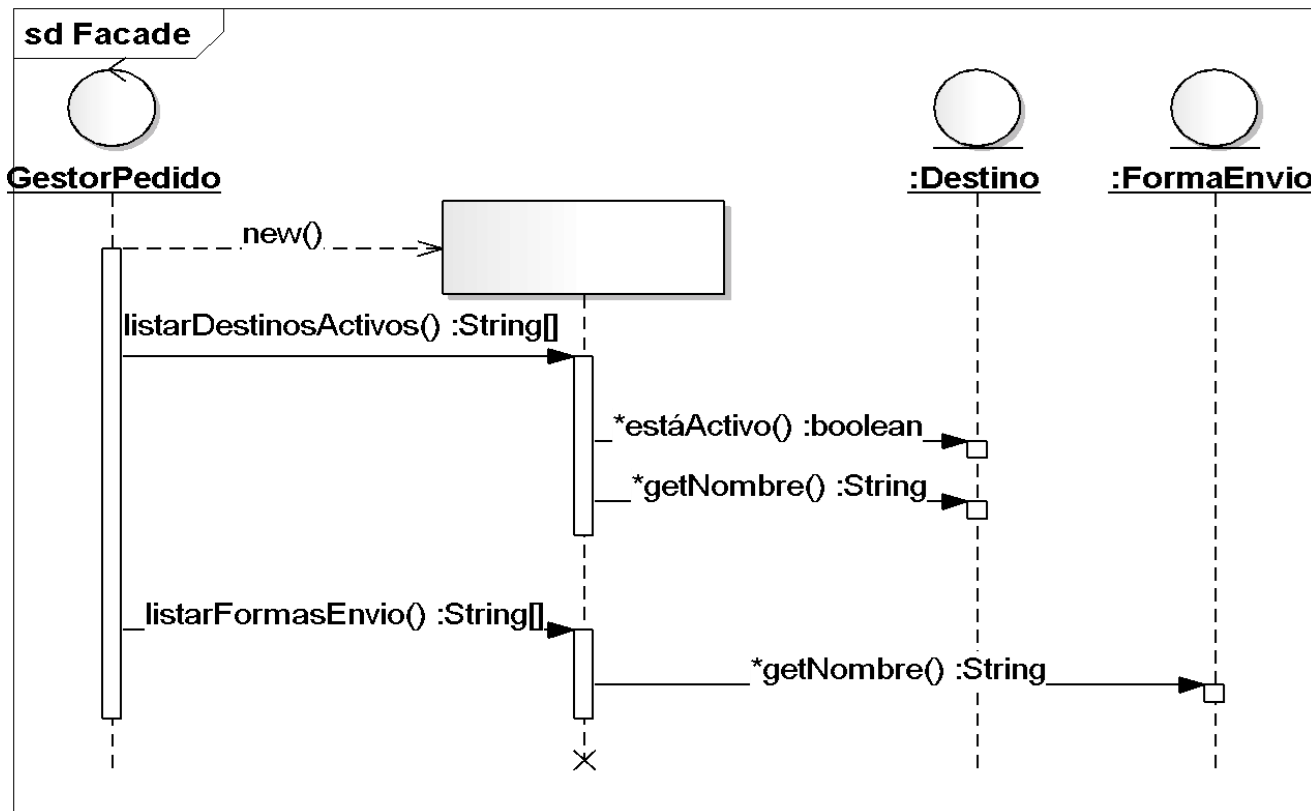
- ⦿ Fachada [**FachadaEnvio**]: Conoce las clases del subsistema que son responsables ante una petición. Se ocupa de delegar las solicitudes de los clientes en los objetos apropiados.
- ⦿ Clases del Subsistema: implementan la funcionalidad del subsistema y realizan las tareas solicitadas por la fachada.

⦿ Procedimiento

- ⦿ Definir los servicios que serán brindados por la Fachada.
- ⦿ Crear una clase de fabricación pura de acceso público que proporcione a los clientes los servicios definidos (FachadaEnvio).
- ⦿ Implementar los métodos que realizan la petición a las clases del subsistema.
- ⦿ Establecer las relaciones requeridas con los objetos propios del subsistema .

Façade: Aplicación

⦿ Dinámica:



⦿ Ventajas:

- ⦿ Disminuye el acoplamiento entre un subsistema y sus clientes.
- ⦿ Ocultar a los clientes los componentes de un subsistema.

Abstract Factory

(Factoría Abstracta)

◎ Propósito

- Proporcionar una interfaz para crear familias de objetos relacionados o dependientes sin especificar la clase concreta

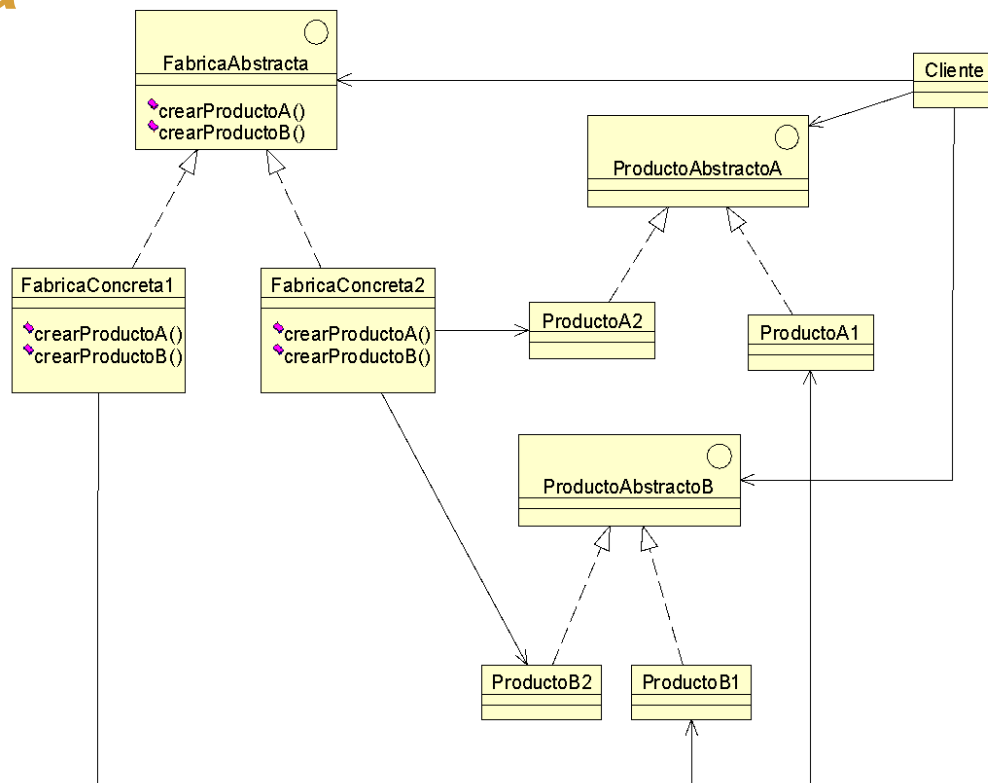
◎ Motivación

- Un *toolkit* interfaz de usuario que soporta diferentes formatos: Windows, Motif, X-Windows,...

- Patrones de Creación
- Patrones de Estructura
- Patrones de Comportamiento

Abstract Factory

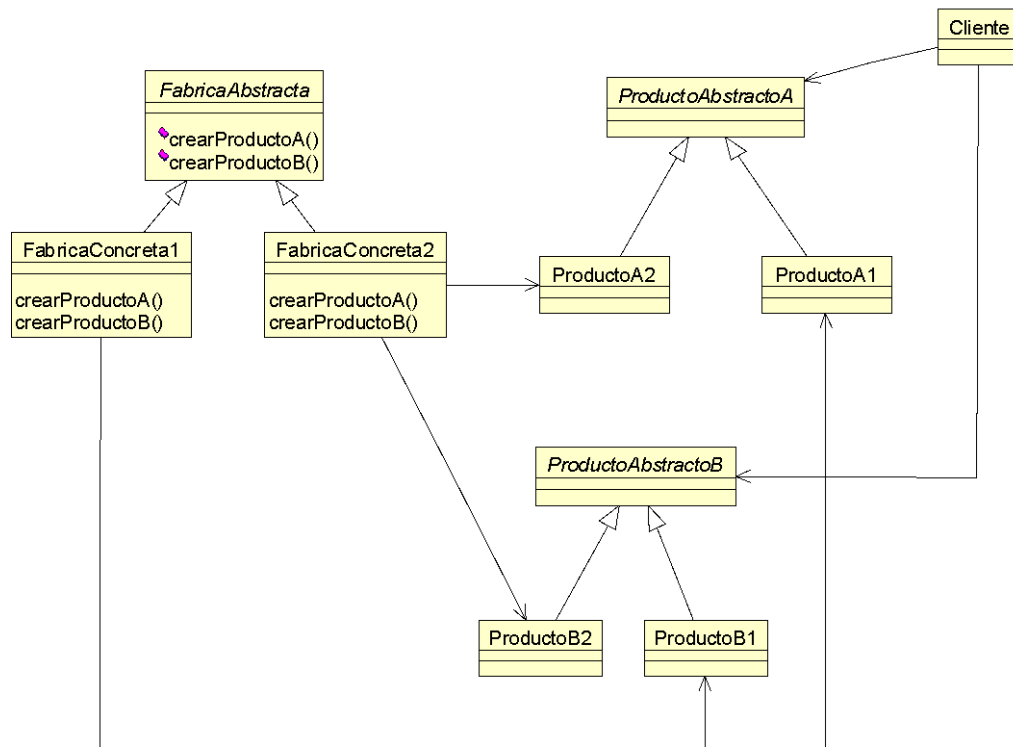
Estructura



Realización

Abstract Factory

Estructura



Herencia

Abstract Factory

◎ Aplicabilidad

- ◎ Un sistema debería ser independiente de cómo sus productos son creados, compuestos y representados
- ◎ Un sistema debería ser configurado para una familia de productos.
- ◎ Una familia de objetos productos relacionados es diseñado para ser utilizado juntos y se necesita forzar la restricción.
- ◎ Se quiere proporcionar una librería de clases de productos y se quiere revelar sólo la interfaz, no la implementación.

- Patrones de Creación
- Patrones de Estructura
- Patrones de Comportamiento

Abstract Factory

◎ Consecuencias

- ◎ Aísla a los clientes de las clases concretas de implementación.
- ◎ Facilita el intercambio de familias de productos.
- ◎ Favorece la consistencia entre productos
- ◎ Es difícil soportar nuevas clases de productos.

- Patrones de Creación
- Patrones de Estructura
- Patrones de Comportamiento

Abstract Factory

◎ Implementación

- ◎ Factorías como *singleton*.
- ◎ Se necesita una subclase de *AbstractFactory* por cada clase de producto que redefina un *método factoría*.
- ◎ Posibilidad de usar el patrón *Prototype*.
- ◎ Definir factorías extensibles: *AbstractFactory* sólo necesita un método de creación. DEPENDE DEL LENGUAJE

- Patrones de Creación
- Patrones de Estructura
- Patrones de Comportamiento

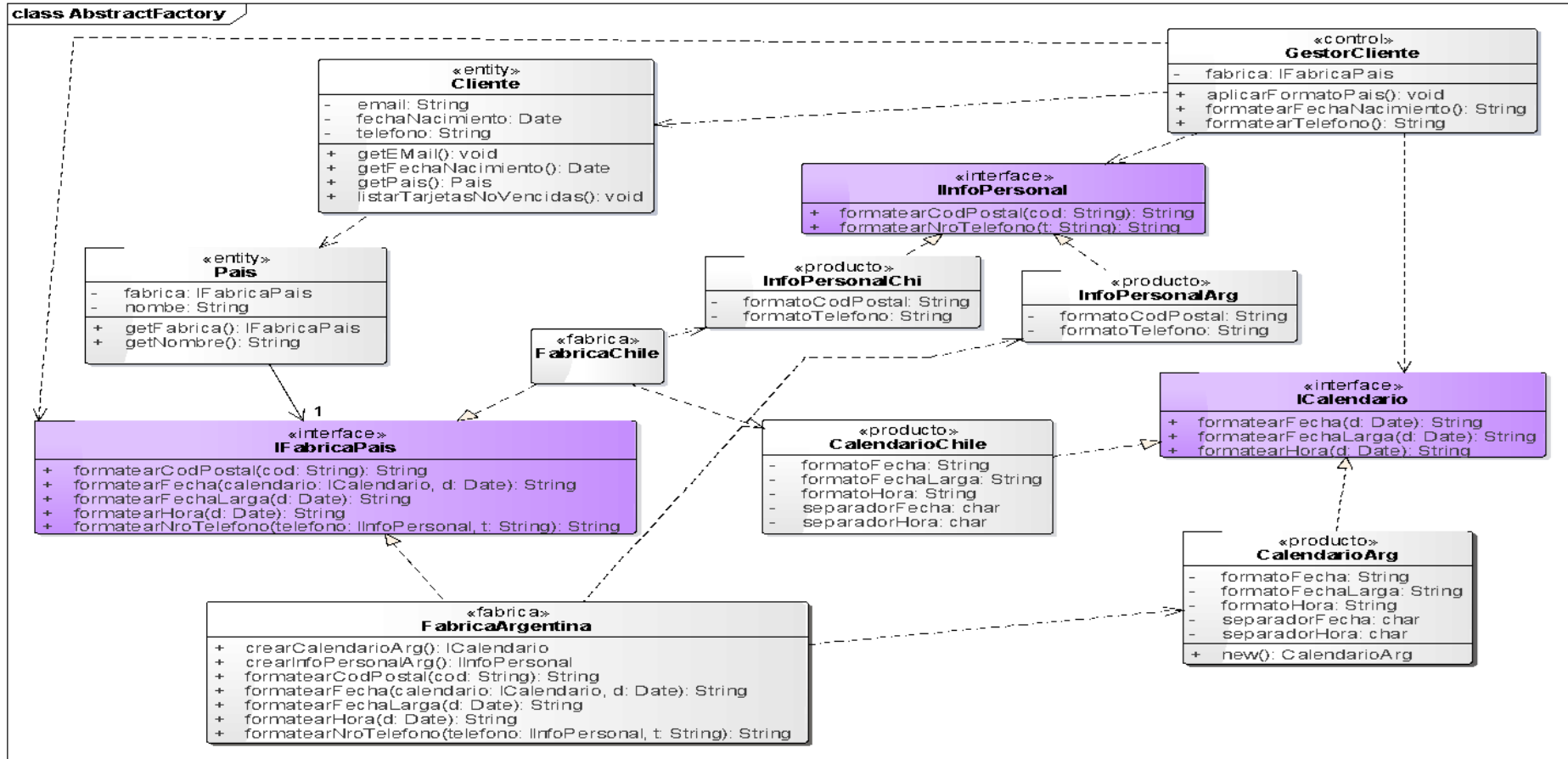
Abstract Factory

⦿ Procedimiento

- ⦿ Analizar la situación determinando elementos que siendo del mismo tipo varían en su comportamiento (Productos).
- ⦿ Identificar el patrón que da origen a las variaciones en los productos (fábricas).
- ⦿ Implementar las características que varían de los productos es decir: implementar los productos concretos.
- ⦿ Implementar las fábricas concretas que crean los productos concretos.
- ⦿ Establecer en el cliente la relación de conocimiento con la fábrica abstracta y el producto abstracto.
- ⦿ Entendemos que el cliente recibirá por parámetros la identificación de la fábrica concreta. Y a partir de esta última se determinarán los productos concretos que corresponda.

Abstract Factory: Aplicación

Estructura

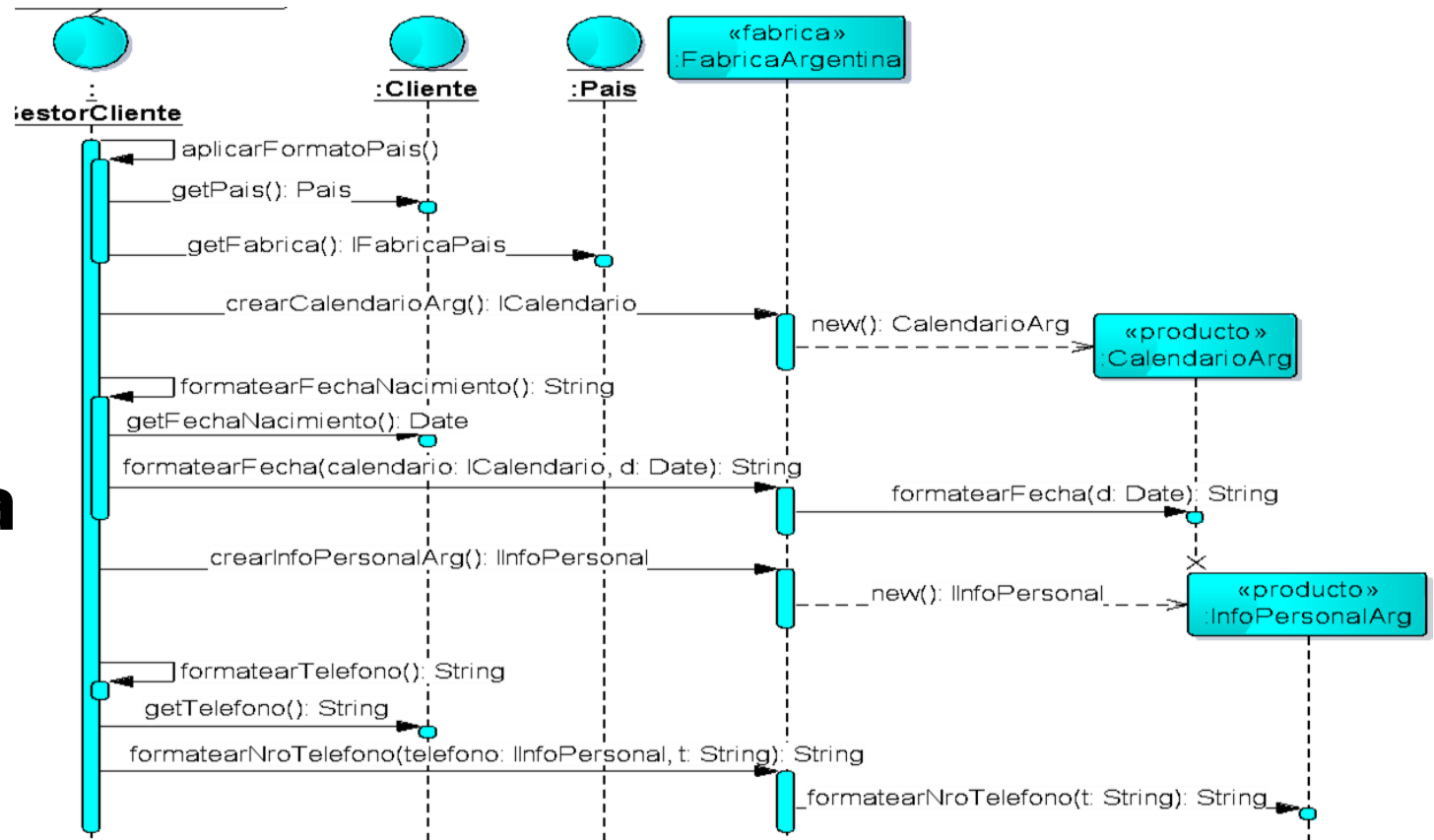


Abstract Factory

◎ Participantes

- ◎ **Fábrica Abstracta** [/FabricaPais]:
 - Declara una interfaz para las operaciones que crean los objetos que determinar las preferencias de los usuarios para moneda, fecha y hora.
- ◎ **Fábricas Concreta** [/FabricaArgentina y FabricaChile]:
 - Implementa las operaciones que crean los objetos que definen las preferencias concretas de cada país.
- ◎ **Producto Abstracto** [/Calendario, InfoPersonal]:
 - Declara la interfaz para un crear cada producto determinado.
- ◎ **Producto Concreto** [/CalendarioArg, CalendarioChile, InfoPersonalArg, InfoPersonalChi]:
 - Define un objeto para que sea creado por la fábrica correspondiente al país.
- ◎ **Cliente** [/GestorCliente]:
 - Sólo usa interfaces declaradas por las clases fábrica abstracta y producto abstracto.

Abstract Factory: Aplicación



Dinámica

Abstract Factory: Aplicación

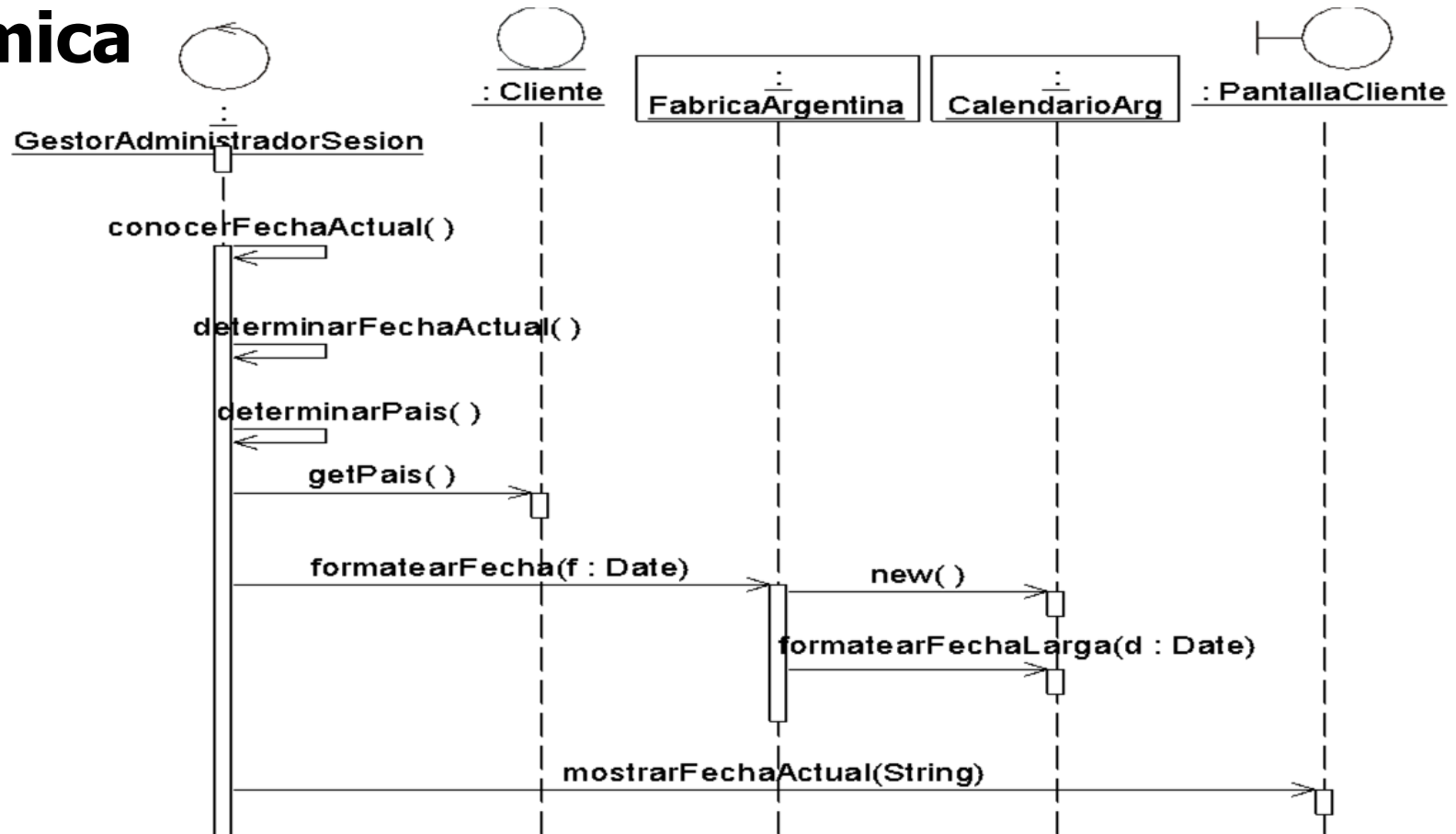
© Procedimiento

Obtener Formato Fecha

- El **GestorCliente** necesita configurar el formato de fecha en función del país, para ello le pide al **Cliente** la referencia del país al que pertenece.
- Luego le pide a la **FabricaArgentina** que cree la **InfoPersonalArg** y que obtenga la máscara de teléfono.
- Luego el **GestorCliente** le envía a la **PantallaCliente** la información para configurar campo de número de teléfono, según el país.

Abstract Factory: Aplicación

Dinámica



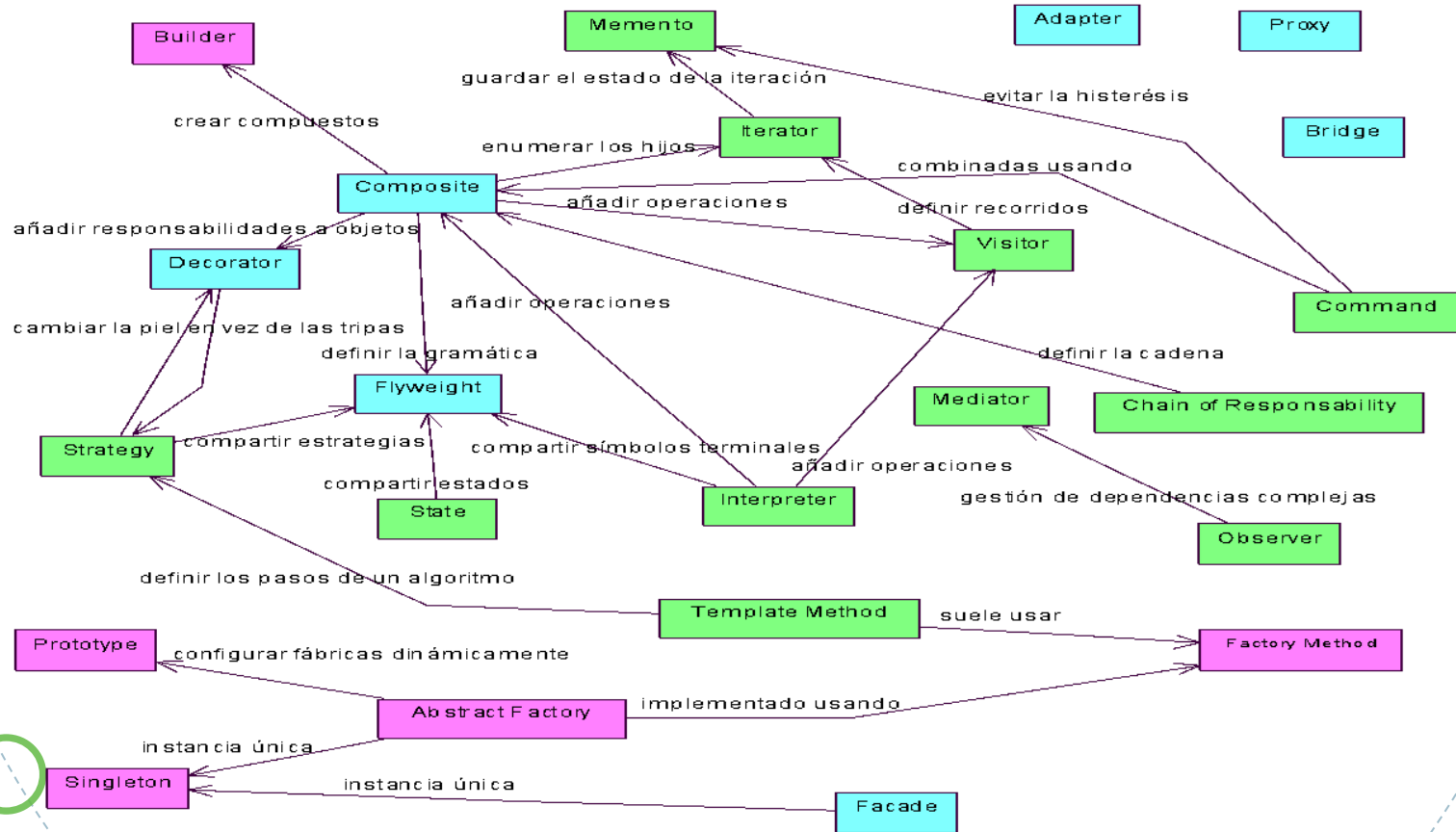
Abstract Factory

© Procedimiento

Obtener Número de Teléfono

- El **GestorCliente** necesita configurar el formato del número de teléfono en función del país, para ello le pide al **Cliente** la referencia del país al que pertenece.
- Luego le pide a la **FabricaArgentina** que cree el **CalendarioArg** y que obtenga el formato de fecha.
- Luego el **GestorCliente** le envía a la **PantallaCliente** la información para configurar campo de fecha según el formato elegido.

Relaciones entre los patrones



Causas comunes de rediseño

Crear un objeto indicando la clase.	<i>Abstract factory, Factory Method, Prototype</i>
Dependencia de operaciones específicas	<i>Chain of Responsibility, Command</i>
Dependencia de plataformas Hw o Sw	<i>Abstract Factory, Bridge</i>
Dependencia sobre representación de objetos.	<i>Abstract factory, Bridge, Memento, Proxy</i>
Dependencias de algoritmos	<i>Builder, Iterator, Strategy, Template Method, Visitor</i>
Acoplamiento fuerte entre clases	<i>Abstract factory, Bridge, CoR, Command, Facade, Mediator, Observer</i>
Extender funcionalidad mediante subclases	<i>Bridge, CoR, Composite, Decorator, Observer, Strategy, State</i>
Incapacidad de cambiar clases convenientemente	<i>Adapter, Decorator, Visitor</i>

Bibliografía

“

- © Design Patterns: Elements of Reusable Object-oriented software. Gamma, Helm, Johnson, Vlissides
- © Applying UML and Patterns. Craig Larman
- © Head First Design Patterns. Freeman & Freeman
- © Dive into Design Patterns. Alexander Shvets
- © <https://refactoring.guru/>

